

Cross-Repository Code Intelligence

Cross-Repository Code Intelligence with a Graph Database: Modeling Database Operations and Role-Specific Use Cases on Codebases

Überarbeitete wissenschaftliche Lernfassung und
Implementierungsmanual

Internes Graphmodell: Code-Database Evidence Graph

Autorin	Fatemeh Hajimohammadi
Studiengang	B.Sc. Angewandte Informatik
Betreuung	Prof. Dr. Ute Schmid
Institution	Otto-Friedrich-Universität Bamberg
Arbeitsstand	Juli 2026, überarbeitete Fassung
Dokumentfunktion	Lernfassung, Forschungsnotiz, Vorlage für eine spätere Kurzfassung und überarbeitete Version nach Review

Bestätigter Projektrahmen

Der verbindliche Kern besteht aus Cross-Repository Code Intelligence, einer Graphdatenbank, der Modellierung von Datenbankoperationen und rollenbezogenen Use Cases. Das interne Graphmodell heißt Code-Database Evidence Graph.

Funktion dieser Fassung

Die Hauptsprache ist Deutsch. Eine dreisprachige Kurzorientierung und ein deutsch-englisch-persisches Glossar unterstützen das Sprachlernen, ohne den gesamten Text zu verdreifachen.

Status, Leseregel und Farbsystem

Dieses Dokument ist gleichzeitig wissenschaftliches Exposé, technische Spezifikation, Forschungsnotiz und persönliche Lernunterlage. Es unterscheidet systematisch zwischen Literaturbefunden, bestätigten Projektentscheidungen, geplanten Implementierungen, Annahmen und offenen Fragen. Diese überarbeitete Fassung verdichtet Wiederholungen, trennt Lerntext von späterer Kurzfassung klarer und hält die Terminologie konsistenter. Wissenschaftliche Aussagen werden paraphrasiert und mit Quellen belegt; direkte Zitate werden vermieden oder ausdrücklich als Zitat markiert.

Explizit markierte Forschungsfragen, Hypothesen und Entscheidungen (Schnellprüfung)

Forschungsfragen (nummeriert)

1. RQ1 (Extraktion): Wie präzise und vollständig lassen sich READ- und WRITE-Operationen aus heterogenen C#- und SQL-Artefakten statisch extrahieren?
2. RQ2 (Retrieval + Verifikation): Verbessert ein evidenzzentrierter Graph mit expliziter Pfadprüfung die Antwortqualität gegenüber reinem Snippet-/Text-Retrieval?
3. RQ3 (Antwortbarkeit): Führt die Trennung von Trefferfindung und Evidenzverifikation zu weniger unbelegten Antworten und zu mehr korrekten Verweigerungen?

Arbeits-Hypothesen (prüfbar)

1. H1: Graphbasierte Evidenzpfade erhöhen Precision und Traceability gegenüber textbasiertem Baseline-Retrieval.
2. H2: Explizite Answerability-Regeln senken false supported claims und reduzieren Halluzinationen.
3. H3: Die getrennte Bewertung von Extraktion und Retrieval macht Fehlerursachen klarer lokalisierbar.

Projektentscheidungen (explizit markiert)

1. D1: Fokus auf statische Analyse (keine Laufzeitinstrumentierung im Pflichtkern).
2. D2: Speicherunabhängiges Evidenzmodell; konkrete Graph-Engine ist Implementierungsdetail.
3. D3: SUPPORT nur bei gültigem Evidence Path; sonst INCOMPLETE/UNRESOLVED oder begründete Verweigerung.
4. D4: Evaluation auf eingefrorenem Korpus-Snapshot mit manuell geprüftem Goldstandard.

Mini-Definitionsliste (B1-B2, sehr kurz)

1. Retrieval-Treffer: Eine Code-Stelle, die durch Suche gefunden wurde.
2. Evidence Record: Ein strukturierter Eintrag mit Code-Stelle, Typ und Metadaten.
3. Evidence Path: Eine Kette von Evidence Records, die einen Zusammenhang zeigt.
4. Unterstützter Claim: Eine Aussage, die durch mindestens einen gültigen Evidence Path belegt ist.

Scope-Status auf einen Blick (klarer Rand des Projekts)

Betreuungsabstimmung zur Fokusreduktion (Bachelor, verbindlich)

Explizite Einordnung für die Bachelor-Bewertung

Diese Arbeit ist wissenschaftlich und methodologisch stark aufgestellt (Problemdefinition, Trennung von Extraktion/Retrieval, Evidenzlogik, Goldstandard, Reproduzierbarkeit).

Für eine hohe Bachelor-Bewertung ist die zentrale Bedingung die konsequente Einhaltung des reduzierten Implementationsumfangs.

Klartext für die Betreuung:

1. Hohe wissenschaftliche Qualität ist gegeben.
2. Hohe Note ist realistisch, wenn der Pflichtkern ohne Scope Drift vollständig umgesetzt, evaluiert und sauber verschriftlicht wird.

3. Optionale Erweiterungen verbessern nicht automatisch die Note, wenn sie die Fertigstellung des Pflichtkerns gefährden.

Ein-Satz-Ziel für die Betreuung:

Der Plan ist wissenschaftlich tragfähig; für die Bachelor-Umsetzung wird der Implementationsumfang bewusst reduziert, damit ein sauberer Goldstandard, belastbare Kernmetriken und eine vollständige schriftliche Auswertung sicher fertig werden.

Vorgeschlagene Reduktion (konkret):

1. Forschungsachsen fokussieren: RQ1 (Extraktion) und RQ2 (Retrieval + Verifikation) als Pflichtkern; RQ3 nur als kurze Ergänzungsfrage in der Diskussion.
2. Datenzugriffsarten begrenzen: Im Pflichtkern nur drei klar prüfbare Zugriffstypen: (a) Entity Framework LINQ (READ/WRITE), (b) Raw SQL im C#-Code, (c) Stored-Procedure-Aufrufe. Weitere Sonderfälle werden als Future Work dokumentiert.
3. Design-Science explizit abgrenzen: In der Praxis werden nur die Zyklen Build -> Evaluate -> Refine für den Pflichtkern vollständig durchgeführt; weiterführende DSR-Varianten bleiben theoretische Einordnung.
4. Retrieval-Komplexität deckeln: Keine breite Embedding-Exploration im Pflichtkern; Baseline + eine graphbasierte Retrievalvariante reichen für den wissenschaftlichen Vergleich.
5. Goldstandard klein und machbar halten: Zielgröße ca. 120 bis 180 Evidence Records, ausgewogen über READ/WRITE und über die drei Zugriffstypen verteilt.
6. Metriken priorisieren: Primärmetriken sind Precision, Recall, Path Validity und Supported-Claim-Rate; alle weiteren Kennzahlen werden als sekundäre Diagnosemetriken geführt.

Explizite Fragen an Prof. Schmid (zur schnellen Entscheidung):

1. Können wir den Pflichtkern offiziell auf RQ1 + RQ2 begrenzen und RQ3 als Diskussionsteil führen?
2. Ist die Begrenzung auf drei Zugriffstypen (EF LINQ, Raw SQL, Stored Procedures) für die Bachelorarbeit ausreichend?
3. Ist eine Goldstandard-Zielgröße von 120 bis 180 Records methodisch akzeptabel?
4. Reicht es, im Pflichtkern Baseline + eine graphbasierte Retrievalvariante zu evaluieren?
5. Soll die ausführliche DSR-Darstellung auf die tatsächlich ausgeführten Zyklen beschränkt werden, mit kurzen theoretischen Verweisen für den Rest?

Verbindliche Arbeitsregel:

Alles, was nicht im abgestimmten Pflichtkern liegt, wird nicht nebenbei implementiert, sondern als Future Work dokumentiert.

Sicher im Pflichtumfang (Muss):

1. Roslyn-basierte statische Extraktion für relevante C#-Artefakte.
2. Typisierte Evidence Records mit Source Locations und Provenienz.
3. Code-Database Evidence Graph mit Pfadprüfung für Claims.
4. Getrennte Evaluation von Extraktion und Retrieval.
5. Goldstandard-basierte Messung auf eingefrorenem Korpus.

Nur bei ausreichender Zeit (Kann):

1. Zusätzliche semantische Retrievalvarianten (z. B. Embeddings).
2. Erweiterte UI/Agent-Interaktion über den Kernworkflow hinaus.
3. Breitere Multi-Repository-Ausweitung über den bestätigten Korpusrahmen hinaus.
4. Umfangreichere Nutzerstudie als ergänzende Evaluation.

Didaktische Erklärung - einfaches Deutsch

Du sollst nach jedem Kapitel nicht nur wissen, was dort steht, sondern erklären können: Was ist das Problem? Warum reichen vorhandene Werkzeuge nicht? Was baut das Projekt? Wie wird geprüft, ob die Lösung funktioniert?

Bestätigte Projektentscheidung

Projektname: Cross-Repository Code Intelligence. Offizieller Titel: Cross-Repository Code Intelligence with a Graph Database: Modeling Database Operations and Role-Specific Use Cases on Codebases. Internes Graphmodell: Code-Database Evidence Graph.

Noch zu prüfen

Alle Aussagen über den konkreten Corpus, die endgültige Zahl der Use Cases, die verpflichtende Spaltenebene und eine mögliche Nutzerstudie bleiben vorläufig, bis Corpus Census, Pilot und Betreuungsgespräch abgeschlossen sind.

Frage für Prof. Schmid

Genügt für „Cross-Repository“ ein reproduzierbares Setup mit mehreren logisch getrennten Projekten oder Repositories aus Danphe, oder soll mindestens ein zweites unabhängiges Open-Source-Repository in die Evaluation aufgenommen werden?

Lernziel

Erkläre ohne Nachschlagen in höchstens zwei Minuten den Unterschied zwischen einem Retrieval-Treffer, einem Evidence Record, einem gültigen Evidence Path und einem unterstützten Claim.

Projektüberblick in drei Sprachen

Deutsch

Cross-Repository Code Intelligence untersucht, wie Datenbankoperationen in mehreren C#-Codebasen statisch erkannt, mit ihrer Provenienz in einem Code-Database Evidence Graph modelliert und für rollenbezogene Fragen genutzt werden können. Das System trennt Kandidatenfindung von Verifikation: Text- oder Embedding-Suche darf mögliche Stellen finden, aber ein Claim wie „Methode X schreibt in Tabelle Y“ gilt nur dann als unterstützt, wenn ein nachvollziehbarer Pfad über Code, Mapping, Operation und Source Locations vorliegt. Die Evaluation verwendet einen eingefrorenen Corpus, einen manuell geprüften Goldstandard, Baselines, kontrollierte Konfigurationen und eine explizite Antwortbarkeitslogik.

English

Cross-Repository Code Intelligence investigates how database operations can be extracted statically across C# codebases, represented with provenance in a Code-Database Evidence Graph, and queried through role-specific use cases. Candidate retrieval is separated from evidence verification: lexical or neural search may identify relevant locations, but a claim such as “method X writes to table Y” is supported only when a traceable path connects code, mappings, operations, database objects, and source locations. The evaluation relies on a frozen corpus, manually verified gold evidence, baselines, controlled configurations, and explicit answerability rules.

فارسی

C# بررسی می‌کند که چگونه می‌توان عملیات پایگاه‌داده را در چند کدبیس Cross-Repository Code Intelligence پروژه نگه داشت و برای پرسش‌های وابسته به Code-Database Evidence Graph به‌صورت ایستا استخراج کرد، منشأ هر شاهد را در نقش‌های مختلف استفاده کرد. در این سیستم، پیدا کردن نامزدها از تأیید شواهد جداست: جستجوی متنی یا عصبی فقط محل‌های می‌نویسد «فقط زمانی پشتیبانی شده است که یک مسیر قابل ردیابی میان کد، Y در جدول X احتمالی را پیدا می‌کند، اما ادعایی مانند «متد نگاشت، عملیات، شیء پایگاه‌داده و مکان منبع وجود داشته باشد. ارزیابی بر پایه یک پیکره ثابت، استاندارد طلایی بررسی شده، خط مبنا، پیکربندی‌های کنترل‌شده و قوانین روشن پاسخ‌پذیری انجام می‌شود.

Das Projekt in einer Kette: Problem - Lösung - Ergebnis

Die folgende Kette ist der rote Faden des gesamten Dokuments. Jede technische Komponente muss mindestens einen Schritt dieser Kette unterstützen, und jede Evaluation muss sich auf einen beobachtbaren Übergang beziehen.

Rollenbezogene Mini-Szenarien (konkret, alltagsnah)

1. Developer: Welche Methode schreibt in Patient und warum?

Output: konkrete Methode, WRITE-Nachweis, Pfad und Source Locations.

2. Architect: Welche Module hängen über Datenbankzugriffe zusammen?

Output: übergreifender Pfad zwischen Komponenten statt isolierter Snippets.

3. QA/Compliance: Ist der Claim X schreibt in Y wirklich belegt?

Output: SUPPORTED nur mit gültigem Evidence Path; sonst begründete Verweigerung.

Diese Szenarien zeigen den realen Nutzen: weniger Raten, schnellere Ursachenanalyse und überprüfbare Antworten für verschiedene Rollen.

Schritt	Baustein	Bedeutung
1	C#, EF- und SQL-Codebasen	Verteilte, heterogene Artefakte
2	Deterministische statische Extraktion	Syntax, Symbole, Mappings, Mutationen, SQL, Stored Procedures
3	Normalisierte Evidence Records	Typ, Source Location, Regel, Status, Provenienz
4	Code-Database Evidence Graph	Codeelemente, Datenbankoperationen, Tabellen, Spalten und Evidence Paths
5	Role-specific Query Contracts	Developer-, Architect-, QA-/Compliance-Fragen
6	Verifier und Answerability	SUPPORTED, INCOMPLETE, UNRESOLVED oder begründete Verweigerung
7	Kontrollierte Evaluation	Goldstandard, Baselines, Precision, Recall, Traceability, Path Validity, False Refusal

Mini-Beispiel

Frage	Welche Methoden schreiben in die Tabelle Patient?
Kandidaten	Methoden mit Entity-Mutation, DbSet-Nutzung, SQL- oder Procedure-Aufrufen.
Verifikation	Mutation + Mapping + Persistenzereignis + Source Locations.
Ausgabe	Methodenliste, Evidence Path, Quellen und Status. Fehlt ein notwendiger Schritt, wird nicht geraten.

Lernnavigation: Was steht wo und wozu?

Diese Navigation ist kein bloßes Inhaltsverzeichnis. Sie erklärt für jedes Hauptkapitel Thema, Problem, Lösungsbeitrag und erwartetes Ergebnis. Die Seitenzahlen werden nach dem finalen Rendering automatisch eingesetzt.

1. Ausgangslage und Motivation Seite 18	Thema: Warum das Thema relevant ist Problem: Die Verbindung zwischen C#-Methoden und Datenbankobjekten ist über Dateien, Projekte, Frameworks und SQL-Artefakte verteilt. Lösung: Statische Extraktion und explizite Evidenzpfade verbinden lokale Beobachtungen. Ergebnis: Ein klar abgegrenztes, praktisch relevantes Forschungsproblem.
2. Wissenschaftliche Problemstellung Seite 23	Thema: Welche wissenschaftliche Schwierigkeit untersucht wird Problem: Treffer sind nicht automatisch Evidenz, und plausible Antworten können falsch oder unvollständig sein. Lösung: Extraktion, Repräsentation, Retrieval und Antwortbarkeit werden getrennt operationalisiert. Ergebnis: Messbare Untersuchungsdimensionen und prüfbare Systementscheidungen.
3. Begriffe und konzeptionelle Grundlagen Seite 28	Thema: Welches Wissen für das Projekt benötigt wird Problem: Ohne präzise Begriffe werden Designentscheidungen, Messgrößen und Ergebnisse vermischt. Lösung: Die zentralen Konzepte aus statischer Analyse, Datenbankzugriff, Graphen, Evidenz und Evaluation werden definiert. Ergebnis: Ein verbindliches Vokabular für Implementierung und Präsentation.
4. Forschungsstand Seite 50	Thema: Welche Ansätze bereits existieren Problem: Einzelne Arbeiten behandeln Codegraphen, Retrieval, SQL-Analyse oder Repository-QA, aber nicht exakt dieselbe Kombination. Lösung: Verwandte Arbeiten werden nach Problem, Methode, Ergebnis und Übertragbarkeit verglichen. Ergebnis: Eine belastbare Positionierung der Arbeit.
5. Forschungslücke Seite 53	Thema: Was trotz vorhandener Werkzeuge offen bleibt Problem: Moderne Agents können Code durchsuchen, liefern aber nicht automatisch reproduzierbare, typisierte Code-Datenbank-Evidenz mit Source Locations. Lösung: Die Arbeit fokussiert einen engeren, empirisch prüfbaren Evidence-Contract. Ergebnis: Eine begründete wissenschaftliche Nische statt bloßer Tool-Konkurrenz.

Lernnavigation - Fortsetzung

6. Zielsetzung und Forschungsziele Seite 55	Thema: Was gebaut und untersucht wird Problem: Ein zu breites Ziel würde gleichzeitig Parser, GraphRAG, Agent, UI und Nutzerstudie verlangen. Lösung: Kernziele, Mindestartefakt und optionale Erweiterungen werden priorisiert. Ergebnis: Ein realistischer und bewertbarer Projektumfang.
7. Forschungsfragen Seite 57	Thema: Welche Fragen die Arbeit beantwortet Problem: Technische Features allein sind keine Forschungsfragen. Lösung: Fragen werden an beobachtbare Variablen, Konfigurationen und Metriken gebunden. Ergebnis: Ein roter Faden von Literatur über Implementierung bis Evaluation.
8. Untersuchungsumfang und Abgrenzung Seite 59	Thema: Was ausdrücklich in und außerhalb des Scopes liegt Problem: Unklare Grenzen führen zu ständigem Umplanen und nicht abschließbaren Erweiterungen. Lösung: Core, bedingte Erweiterungen und Future Work werden getrennt. Ergebnis: Ein stabiler Scope für die Thesis.
9. Evaluationskorporus: Danphe Seite 61	Thema: Auf welcher Codebasis untersucht wird Problem: Ergebnisse sind ohne eingefrorenen Snapshot und dokumentierte Corpus-Auswahl nicht reproduzierbar. Lösung: Commit, Lizenz, Projektstruktur, Datenbankartefakte und Ausschlüsse werden protokolliert. Ergebnis: Ein prüfbares Evaluationskorporus.
10. Forschungsartefakt Cross-Repository Code Intelligence Seite 63	Thema: Wie das Forschungsartefakt aufgebaut ist Problem: Code-, Mapping- und Datenbankinformationen liegen in verschiedenen Repräsentationen vor. Lösung: Extractor, Normalizer, Code-Database Evidence Graph, Query Layer und Verifier werden als getrennte Module entworfen. Ergebnis: Eine ausführbare und testbare Referenzimplementierung.

Lernnavigation - Fortsetzung

11. Forschungsmethodik Seite 67	Thema: Wie wissenschaftlich vorgegangen wird Problem: Ein funktionierender Prototyp allein beantwortet keine Forschungsfrage. Lösung: Design Science wird mit kontrollierten Experimenten und dokumentierten Iterationen verbunden. Ergebnis: Ein nachvollziehbarer Build-Evaluate-Zyklus.
12. Goldstandard Seite 69	Thema: Wie die richtige Antwort festgelegt wird Problem: Automatische Ausgaben können nicht mit sich selbst validiert werden. Lösung: Manuell geprüfte Evidence Records und Answerability-Labels bilden die Referenz. Ergebnis: Eine unabhängige Grundlage für Precision, Recall und Fehleranalyse.
13. Experimentelles Design Seite 71	Thema: Welche Konfigurationen verglichen werden Problem: Ohne kontrollierte Baselines lässt sich der Nutzen einzelner Komponenten nicht bestimmen. Lösung: Extraktions-, Repräsentations- und Retrievalvarianten werden separat variiert. Ergebnis: Vergleichbare und reproduzierbare Experimente.
14. Metriken und Auswertung Seite 73	Thema: Wie Ergebnisse gemessen werden Problem: Nur Accuracy verdeckt unvollständige, falsche oder unnötig verweigerte Antworten. Lösung: Mehrere Metriken erfassen Extraktion, Retrieval, Pfade, Traceability und Refusal. Ergebnis: Eine differenzierte Bewertung statt einer einzigen Zahl.
15. Fehleranalyse Seite 76	Thema: Warum das System scheitert Problem: Aggregierte Kennzahlen zeigen nicht, welche technischen Muster problematisch sind. Lösung: Fehler werden nach Ursache, Artefakttyp, Frameworkmuster und Evidence Gap kategorisiert. Ergebnis: Konkrete Verbesserungs- und Limitationsaussagen.

Lernnavigation - Fortsetzung

16. Erwartete Beiträge Seite 77	Thema: Welchen wissenschaftlichen und praktischen Wert die Arbeit liefert Problem: Eine Thesis muss mehr beitragen als Quellcode. Lösung: Artefakt, Datenmodell, Goldstandard, Evaluation und Designwissen werden als Beiträge getrennt. Ergebnis: Eine klare Contribution-Argumentation.
17. Erfolgskriterien Seite 78	Thema: Wann das Projekt als erfolgreich gilt Problem: Perfektion ist bei statischer Analyse nicht realistisch, aber unklare Ziele sind nicht prüfbar. Lösung: Mindestfunktion, Messbarkeit, Reproduzierbarkeit und dokumentierte Grenzen werden definiert. Ergebnis: Objektive Abschlusskriterien.
18. Validitätsbedrohungen Seite 79	Thema: Wie die Aussagekraft begrenzt sein kann Problem: Ein einzelnes Open-Source-System und manuelle Annotation können Verzerrungen erzeugen. Lösung: Konstrukt-, interne, externe und Schlussfolgerungsvalidität werden systematisch geprüft. Ergebnis: Transparente Grenzen der Ergebnisse.
19. Ethische und rechtliche Rahmenbedingungen Seite 81	Thema: Welche Verantwortung gilt Problem: Healthcare-Code kann sensible Domänenlogik enthalten, auch wenn keine Patientendaten verarbeitet werden. Lösung: Lizenz, Datenschutz, Datenminimierung und Claim-Grenzen werden dokumentiert. Ergebnis: Ein verantwortbarer Forschungsprozess.
20. Arbeits- und Zeitplan Seite 82	Thema: Dieses Kapitel ordnet den Abschnitt „Arbeits- und Zeitplan“ in den Gesamtprozess ein. Problem: Ohne diesen Abschnitt bleiben Entscheidungen, Umsetzung oder Evaluation unvollständig dokumentiert. Lösung: Der Abschnitt liefert Definitionen, konkrete Regeln, Checklisten oder nachvollziehbare Arbeitsaufträge. Ergebnis: Ein überprüfbarer Zwischen- oder Abschlussoutput.

Lernnavigation - Fortsetzung

21. Vorläufige Gliederung der Bachelorarbeit Seite 83	Thema: Wie die spätere Thesis aufgebaut wird Problem: Ein Lern-Exposé und eine Abgabe-Thesis haben unterschiedliche Funktionen. Lösung: Kapitel werden entlang Problem, Methode, Artefakt, Evaluation und Diskussion angeordnet. Ergebnis: Eine vorbereitete Schreibstruktur.
22. Offene Entscheidungen und Fragen für Prof. Schmid Seite 84	Thema: Dieses Kapitel ordnet den Abschnitt „Offene Entscheidungen und Fragen für Prof. Schmid“ in den Gesamtprozess ein. Problem: Ohne diesen Abschnitt bleiben Entscheidungen, Umsetzung oder Evaluation unvollständig dokumentiert. Lösung: Der Abschnitt liefert Definitionen, konkrete Regeln, Checklisten oder nachvollziehbare Arbeitsaufträge. Ergebnis: Ein überprüfbarer Zwischen- oder Abschlussoutput.
23. Literatur- und Lernressourcen Seite 85	Thema: Dieses Kapitel ordnet den Abschnitt „Literatur- und Lernressourcen“ in den Gesamtprozess ein. Problem: Ohne diesen Abschnitt bleiben Entscheidungen, Umsetzung oder Evaluation unvollständig dokumentiert. Lösung: Der Abschnitt liefert Definitionen, konkrete Regeln, Checklisten oder nachvollziehbare Arbeitsaufträge. Ergebnis: Ein überprüfbarer Zwischen- oder Abschlussoutput.
24. Präsentationskern Seite 86	Thema: Wie das Projekt mündlich erklärt wird Problem: Viele Details können den Hauptbeitrag verdecken. Lösung: Problem, Evidence Gap, Architektur, Evaluation und Beitrag werden auf einen kurzen Kern reduziert. Ergebnis: Ein verständlicher Vortrag.
25. Betreuungshinweise und Bedeutung der Use Cases Seite 87	Thema: Wie Betreuung und Use Cases in Designentscheidungen übersetzt werden Problem: Use Cases dürfen nicht nur Beispiele sein, sondern müssen Anforderungen und Evaluation strukturieren. Lösung: Rollen, Fragen, Evidenzbedarf und Ausgabeformat werden operationalisiert. Ergebnis: Role-specific use cases als verbindlicher Projektbestandteil.

Lernnavigation - Fortsetzung

26. Verbindliche Use Cases für Entwicklung und Evaluation Seite 88	Thema: Welche Aufgaben das System tatsächlich beherrschen soll Problem: Ein universelles Code-QA-System wäre nicht realistisch evaluierbar. Lösung: Eine begrenzte Menge von tabellen-, spalten-, pfad- und rollenbezogenen Fragen wird festgelegt. Ergebnis: Konkrete Tests und Gold-Antworten.
27. Evidenzprinzipien und methodische Disziplin Seite 90	Thema: Welche Regeln jede Systemaussage erfüllen muss Problem: Ohne Evidence Contract können Vermutung, Retrieval-Treffer und Beweis verwechselt werden. Lösung: Jeder Claim benötigt Typ, Provenienz, Source Location und Status. Ergebnis: Reproduzierbare und überprüfbare Antworten.
28. Erweiterte Operationalisierung von READ und WRITE Seite 92	Thema: Wie Datenbankoperationen formal erkannt werden Problem: Framework-Methodennamen allein reichen nicht, um READ oder WRITE sicher zu klassifizieren. Lösung: Operationen werden über Mutationen, Materialisierung, Mapping und Persistenzereignisse definiert. Ergebnis: Explizite Regeln für Tabellen- und Spaltenebene.
29. Erweiterte Evaluationslogik Seite 93	Thema: Wie Pilot, Entwicklung und Test getrennt werden Problem: Wenn dieselben Beispiele zum Entwickeln und Bewerten genutzt werden, entsteht Optimismus-Bias. Lösung: Pilot- und Testmenge, Baselines, Ablationen und Error Taxonomy werden getrennt. Ergebnis: Glaubwürdigere Ergebnisse.
30. Integrierter Arbeits-, Lern- und Sprachplan Seite 95	Thema: Dieses Kapitel ordnet den Abschnitt „Integrierter Arbeits-, Lern- und Sprachplan“ in den Gesamtprozess ein. Problem: Ohne diesen Abschnitt bleiben Entscheidungen, Umsetzung oder Evaluation unvollständig dokumentiert. Lösung: Der Abschnitt liefert Definitionen, konkrete Regeln, Checklisten oder nachvollziehbare Arbeitsaufträge. Ergebnis: Ein überprüfbarer Zwischen- oder Abschlussoutput.

Lernnavigation - Fortsetzung

31. Konsolidierte Abgrenzung und Entscheidungsstand Seite 97	Thema: Welche Entscheidungen aktuell gelten Problem: Viele frühere Varianten können Scope und Terminologie verwirren. Lösung: Aktuelle Core-Entscheidungen werden an einer Stelle zusammengeführt. Ergebnis: Eine verlässliche Arbeitsgrundlage.
32. Prüf- und Gesprächsfragen für die Betreuung Seite 98	Thema: Dieses Kapitel ordnet den Abschnitt „Prüf- und Gesprächsfragen für die Betreuung“ in den Gesamtprozess ein. Problem: Ohne diesen Abschnitt bleiben Entscheidungen, Umsetzung oder Evaluation unvollständig dokumentiert. Lösung: Der Abschnitt liefert Definitionen, konkrete Regeln, Checklisten oder nachvollziehbare Arbeitsaufträge. Ergebnis: Ein überprüfbarer Zwischen- oder Abschlussoutput.
33. Retrieval- und Antwortlogik Seite 100	Thema: Wie aus einer Frage eine evidenzbasierte Antwort wird Problem: Relevante Snippets allein garantieren weder Vollständigkeit noch Richtigkeit. Lösung: Question Contract, Retrieval, Graph Traversal und Verifier werden gestuft ausgeführt. Ergebnis: Traceable answers oder begründete Verweigerung.
34. Architektur- und Forschungsentscheidungen Seite 101	Thema: Warum die Architektur so gewählt wird Problem: Technologieentscheidungen können stillschweigend zu wissenschaftlichen Annahmen werden. Lösung: Jede Entscheidung wird mit Ziel, Alternative, Risiko und Evaluation dokumentiert. Ergebnis: Nachvollziehbares Designwissen.
35. Integrierte Lernstruktur für jeden Fachabschnitt Seite 103	Thema: Wie das Dokument aktiv gelernt wird Problem: Lesen allein führt nicht automatisch zu erklärbarem Wissen. Lösung: Jeder Abschnitt endet mit Recall-Frage, Mini-Erklärung, Beispiel und Transferaufgabe. Ergebnis: Mündliche und schriftliche Abrufbarkeit.

Lernnavigation - Fortsetzung

36. Verbindliche Terminologie Seite 104	Thema: Welche Begriffe konstant verwendet werden Problem: Synonyme und wechselnde Statusnamen erzeugen Missverständnisse. Lösung: Ein kontrolliertes Glossar legt deutsche und englische Begriffe fest. Ergebnis: Konsistente Thesis und Präsentation.
37. Priorisierung von Zielen, Konfigurationen und Erweiterungen Seite 105	Thema: Was zuerst gebaut wird Problem: Deep Learning und Agenten können den Kern verdrängen. Lösung: Core, Baseline, Experiment und Future Work werden strikt getrennt. Ergebnis: Ein umsetzbarer Entwicklungsweg.
38. Implementierungsmanual: vom Corpus zum überprüfbaren Ergebnis Seite 107	Thema: Wie das System schrittweise umgesetzt wird Problem: Ohne klare Modulgrenzen sind Ergebnisse nicht reproduzierbar testbar. Lösung: Jeder Schritt erhält Input, Output, Invariant, Test und Abbruchkriterium. Ergebnis: Ein reproduzierbarer Implementierungspfad.
39. Fachartikelkatalog mit aktuellen Dateinamen und Bezugsquellen Seite 111	Thema: Welche Literatur wofür gebraucht wird Problem: Eine lange Literaturliste zeigt noch nicht, welche Entscheidung eine Quelle stützt. Lösung: Jede Quelle erhält Priorität, Projektbeitrag, Leseauftrag und Bezugslink. Ergebnis: Ein handlungsorientierter Lesekatalog.
40. Technische Lernressourcen und konkrete Übungen Seite 121	Thema: Welche Lernressource zu welchem Output führt Problem: Link-Sammlungen erzeugen leicht passives Lernen ohne Projektergebnis. Lösung: Jede Ressource wird mit genau einer prüfbaren Übung verbunden. Ergebnis: Gespeicherte technische Lernartefakte.

Lernnavigation - Fortsetzung

Anhang C - Dreisprachiges Fachglossar Deutsch - Englisch فارسی - Seite 124	Thema: Wie zentrale Begriffe sicher erklärt werden Problem: Sprachwechsel und Synonyme erschweren Präsentation und Wiederholung. Lösung: Deutsch, Englisch und فارسی werden mit einer einfachen Definition verbunden. Ergebnis: Ein dreisprachiges Wiederholungswerkzeug.
Anhang D - Vollständiges Literaturverzeichnis Seite 128	Thema: Wo alle verwendeten Quellen nachgewiesen sind Problem: Nicht belegte Aussagen gefährden Nachvollziehbarkeit und Eigenständigkeit. Lösung: Alle wissenschaftlichen und technischen Quellen werden vollständig aufgeführt. Ergebnis: Ein prüfbarer Quellenapparat.

Inhaltsverzeichnis mit Seitenzahlen

Die folgende statische Übersicht enthält Haupt- und Unterkapitel. Die Seitenzahlen werden nach der ersten vollständigen Render-Runde eingetragen, damit sie zur gelieferten Datei passen.

1. Ausgangslage und Motivation	18
1.1 Praktischer Problemkontext	18
1.2 Beispiel einer verteilten Evidenzkette	19
1.3 Praktische Relevanz für Wartung und Refactoring	20
1.4 Praktische Relevanz für Fehleranalyse	20
1.5 Praktische Relevanz für Datenschutz und Compliance	20
1.6 Beispielhafte Nutzungsperspektiven	20
1.7 Wissenschaftliche Relevanz	21
1.8 Vorläufige Positionierung des Projekts	22
2. Wissenschaftliche Problemstellung	23
2.1 Extraktion als Gewinnung typisierter Beobachtungen	23
2.2 Repräsentation als provenienzerhaltende Wissensstruktur	24
2.3 Retrieval als Zusammenstellung erforderlicher Evidenz	25
2.4 Antwortbarkeit als kontrollierte Systementscheidung	26
2.5 Zusammenhang der drei Dimensionen	26
2.6 Eingrenzung der wissenschaftlichen Problemstellung	27
3. Begriffe und konzeptionelle Grundlagen	28
3.1 Grundlagen der statischen Programmanalyse	28
3.2 Datenbankzugriffe in C#/NET	31
3.3 Graphbasierte Repräsentation	36
3.4 Evidenz und Provenienz	39
3.5 Evaluationsgrundlagen	41
3.6 Antwortbarkeit und kontrolliertes Antwortverhalten	44
3.7 Verbindliche konzeptionelle Unterscheidungen	46
3.8 Konsolidierte Terminologie	47
3.9 Zusammenführung der Begriffe im Code-Database-Evidence-Graph-Modell	48
3.10 Bedeutung des Kapitels für die folgenden Kapitel	48
3.11 Abschließende didaktische Erklärung	48
4. Forschungsstand	50
4.1 Compilerbasierte statische Analyse mit Roslyn	50
4.2 Codegraphen	50
4.3 Repositoryweites Retrieval	51
4.4 Datenbankbezogene Programmanalyse	51
4.5 Design-Science-Research	51
4.6 Besonders relevante verwandte Systeme	52
5. Forschungslücke	53
6. Zielsetzung und Forschungsziele	55
7. Forschungsfragen	57
7.1 RQ1 - Evidenzextraktion	57
7.2 RQ2 - Evidenzretrieval und Antwortbarkeit	58
7.3 Mögliche analytische Unterfragen	58
8. Untersuchungsumfang und Abgrenzung	59
8.1 Verbindlicher Kern	59
8.2 Erweiterungen nach Abschluss des Minimalumfangs	59
8.3 Außerhalb des Pflichtumfangs	60
9. Evaluationskorporus: Danphe	61
9.1 Begründung der Korpuswahl	61
9.2 Vorläufig bestätigte Bestandsinformationen	61
9.3 Frozen Snapshot und Korpusprotokoll	62
10. Forschungsartefakt Cross-Repository Code Intelligence	63
10.1 Rolle des Artefakts	64
10.2 Geplante Verarbeitungspipeline	64

10.3 Technische Bausteine	65
10.4 Beobachtung, Ableitung und Antwort	65
10.5 Vorgesehenes Evidenzmodell	65
10.6 Umgang mit Unsicherheit	66
11. Forschungsmethodik	67
11.1 Gesamtansatz	67
11.2 Trennung von Extraktion und Retrieval	67
11.3 Reproduzierbarkeit	67
12. Goldstandard	69
12.1 Auswahlstrategie	69
12.2 Annotationseinheit	69
12.3 Qualitätssicherung	69
13. Experimentelles Design	71
13.1 Extraktionskonfigurationen	71
13.2 Retrievalkonfigurationen	71
13.3 Fragenmenge	72
14. Metriken und Auswertung	73
14.1 Extraktionsmetriken	73
14.2 Retrieval- und Antwortmetriken	74
14.3 Antwortbarkeitsmatrix	75
14.4 Statistische Analyse	75
15. Fehleranalyse	76
16. Erwartete Beiträge	77
17. Erfolgskriterien	78
18. Validitätsbedrohungen	79
18.1 Konstruktvalidität	79
18.2 Interne Validität	79
18.3 Externe Validität	79
18.4 Schlussfolgerungsvalidität	79
18.5 Researcher Bias	79
18.6 Ethisch-rechtliche Aspekte	80
19. Ethische und rechtliche Rahmenbedingungen	81
20. Arbeits- und Zeitplan	82
21. Vorläufige Gliederung der Bachelorarbeit	83
22. Offene Entscheidungen und Fragen für Prof. Schmid	84
23. Literatur- und Lernressourcen	85
24. Präsentationskern	86
Anhang A - Relationstypen des Evidenzmodells	86
Anhang B - Prüf-Checkliste für Evidenz und Antwortbarkeit	86
25. Betreuungshinweise und Bedeutung der Use Cases	87
25.1 Umsetzung der Hinweise der Betreuung	87
25.2 Bedeutung der Use Cases	87
26. Verbindliche Use Cases für Entwicklung und Evaluation	88
26.1 UC1 - Methoden finden, die eine Tabelle oder Spalte lesen	88
26.2 UC2 - Methoden und Pfade finden, die Daten verändern	88
26.3 UC3 - Zugriffsmechanismus bestimmen	88
26.4 UC4 - Vollständigen Evidenzpfad erklären	89
26.5 UC5 - Fehlende Evidenz und begründete Verweigerung	89
26.6 UC6 - Cross-Repository-/Cross-Repository/Cross-Project- oder Cross-Database-Pfad untersuchen	15
26.7 UC7 - Negative Kontrolle gegen Namensähnlichkeit	89
27. Evidenzprinzipien und methodische Disziplin	90
27.1 Jede Behauptung wird zu einer überprüfbaren Suchaufgabe	90
27.2 Drei Evidenzebenen	90
27.3 Speicherunabhängiges wissenschaftliches Modell	90
27.4 Kategorische Unsicherheit statt unbegründeter Prozentwerte	91
28. Erweiterte Operationalisierung von READ und WRITE	92
28.1 READ	92
28.2 WRITE	92

28.3 Tabellen- und Spaltenebene	92
28.4 Laufzeitgrenze	92
29. Erweiterte Evaluationslogik	93
30. Integrierter Arbeits-, Lern- und Sprachplan	95
29.1 Entwicklungs-, Pilot- und Testtrennung	93
29.2 Extraktionsvergleich	93
29.3 Retrievalvergleich	93
29.4 Goldstandard und Annotation	93
29.5 Metriken	93
29.6 Fehleranalyse	94
31. Konsolidierte Abgrenzung und Entscheidungsstand	97
32. Prüf- und Gesprächsfragen für die Betreuung	98
33. Retrieval- und Antwortlogik	100
33.1 Retrieval als eigener Untersuchungsgegenstand	100
33.2 Retrievalvarianten	100
33.3 Fragevertrag	100
34. Architektur- und Forschungsentscheidungen	101
35. Integrierte Lernstruktur für jeden Fachabschnitt	103
36. Verbindliche Terminologie	104
36.1 Zweck	104
36.2 Terminologieregeln	104
37. Priorisierung von Zielen, Konfigurationen und Erweiterungen	105
37.1 Verbindlicher Kern	105
37.2 Bedingte Erweiterungen	105
37.3 Nicht verbindliche und zukünftige Erweiterungen	105
38. Implementierungsmanual: vom Corpus zum überprüfbaren Ergebnis	107
39. Fachartikelkatalog mit aktuellen Dateinamen und Bezugsquellen	111
40. Technische Lernressourcen und konkrete Übungen	121
Anhang C - Dreisprachiges Fachglossar Deutsch - Englisch - فارسی	124
Anhang D - Vollständiges Literaturverzeichnis	128

Kurzfassung und aktuelle Forschungsposition

Die Bachelorarbeit entwickelt und evaluiert Cross-Repository Code Intelligence, ein Forschungsartefakt zur statischen, provenienzerhaltenden Analyse von Datenbankoperationen über mehrere C#-Codebasen. Der Artefaktkern extrahiert Code- und Datenbankbeobachtungen, normalisiert sie als Evidence Records, verbindet sie in einem Code-Database Evidence Graph und beantwortet vordefinierte rollenbezogene Fragen nur dann, wenn der erforderliche Evidence Path nachweisbar ist. Die Arbeit untersucht nicht, ob ein allgemeiner Coding Agent „den Code versteht“, sondern welche typisierten Beziehungen zwischen Codeelementen und Datenbankobjekten reproduzierbar rekonstruiert werden können und wie diese Beziehungen für unterschiedliche Informationsbedürfnisse dargestellt werden.

Die Arbeit trennt drei Ebenen. Erstens wird die Extraktionsqualität gemessen: Welche READ-, WRITE- und gegebenenfalls Procedure-Beziehungen werden korrekt und vollständig erkannt? Zweitens wird die Repräsentation geprüft: Bleiben Source Locations, Ableitungsregeln, Evidence Gaps und Pfade erhalten? Drittens werden Role-Specific Use Cases untersucht: Können Developer-, Architect- und QA-orientierte Fragen korrekt, vollständig, nachvollziehbar und angemessen vorsichtig beantwortet werden? Repository-QA-Benchmarks zeigen, dass strukturierte Retrieval-Signale hilfreich sind, aber aktuelle Modelle weiterhin deutliche Grenzen bei Multi-File- und Dependency-Fragen haben (vgl. Peng et al., 2026; Alebachew et al., 2026; Du et al., 2025).

Deep Learning, RAG oder ein LLM-gestützter Query Adapter sind optionale Experimente. Sie dürfen Kandidaten finden, Fragen strukturieren oder Ergebnisse formulieren, ersetzen aber nicht die deterministische Evidence-Verifikation. Der wissenschaftliche Kern bleibt speicher- und modellunabhängig: Ein Claim muss auf überprüfbare Source Locations und gültige Relationsschritte zurückführbar sein.

Frage für Prof. Schmid

Soll die offizielle Einreichungsfassung zwei Hauptforschungsfragen verwenden - Extraktion/Modellierung und Role-Specific Use Cases - oder soll ein dritter Vergleich der Retrieval-Repräsentationen ausdrücklich als eigene Forschungsfrage formuliert werden?

1. Ausgangslage und Motivation

Kapitelorientierung - Schritt für Schritt

Thema	Warum das Thema relevant ist
Problem	Die Verbindung zwischen C#-Methoden und Datenbankobjekten ist über Dateien, Projekte, Frameworks und SQL-Artefakte verteilt.
Lösung	Statische Extraktion und explizite Evidenzpfade verbinden lokale Beobachtungen.
Ergebnis	Ein klar abgegrenztes, praktisch relevantes Forschungsproblem.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Dieses Kapitel beschreibt die praktische Ausgangslage des Projekts und erläutert, warum nachvollziehbare Evidenz zu Datenbankoperationen sowohl für die Softwarepraxis als auch für die Forschung relevant ist. Im Mittelpunkt steht die Beobachtung, dass Datenbankzugriffe in realen Softwaresystemen häufig nicht lokal, sondern nur über mehrere Artefakte und technische Schichten hinweg verstanden werden können (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

1.1 Praktischer Problemkontext

In typischer Unternehmenssoftware sind Geschäftslogik, Persistenzlogik, objektrelationales Mapping, Datenbankschemata und technische Hilfsfunktionen auf zahlreiche Dateien, Projekte und Architekturschichten verteilt. Eine Controller-Methode kann zunächst einen Service aufrufen. Der Service delegiert anschließend an ein Repository oder eine Hilfsmethode. Diese arbeitet möglicherweise mit einem DbContext, einem DbSet, einer LINQ-Abfrage, einem ADO.NET-Befehl oder einem SQL-Fragment. Die konkrete Zuordnung zwischen einer C#-Entity und einer physischen Datenbanktabelle kann wiederum in einer separaten Konfigurationsklasse, in Attributen oder durch Frameworkkonventionen definiert sein (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Die fachlich relevante Information, welche Methode welches Datenbankelement liest oder verändert, entsteht daher häufig erst durch das Zusammenspiel mehrerer Artefakte. Eine einzelne Codezeile bildet den gesamten Zusammenhang nur selten vollständig ab (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Bei Wartung, Refactoring, Fehleranalyse, Architekturarbeit, Datenschutzprüfung oder der Einführung neuer Funktionen entstehen Fragen wie:

- Welche Methoden lesen eine bestimmte Tabelle oder Spalte?
- Welche Methoden können Daten in einer bestimmten Tabelle verändern?
- Welche Aufrufpfade führen zu einer Datenbankoperation?
- Erfolgt der Zugriff über Entity Framework, ADO.NET, eingebettetes SQL oder eine Stored Procedure?
- Welche Aussage ist direkt durch eine konkrete Quellstelle belegt?
- Welche Aussage wurde erst über Mappings, Hilfsmethoden oder Aufrufbeziehungen abgeleitet?
- Wo fehlt ein Mapping oder ein Zwischenschritt, sodass keine sichere Aussage möglich ist?
- Welche Teile einer Antwort beruhen auf statisch sicher rekonstruierbarer Evidenz und welche bleiben unsicher?

Eine einfache Textsuche kann bei der ersten Orientierung helfen. Sie stellt jedoch keine Programmbedeutung dar. Derselbe Name kann in Kommentaren, Tests, DTOs, Aliasen, generiertem

Code oder fachlich nicht zusammenhängenden Klassen vorkommen. Ein echter Datenbankzugriff kann umgekehrt den physischen Tabellen- oder Spaltennamen überhaupt nicht enthalten, wenn die Zuordnung über Entity-Framework-Mappings oder Konventionen erfolgt (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Auch SQL-bezogene Evidenz kann über mehrere Stellen verteilt sein. Ein SQL-String kann teilweise in einer Methode erzeugt, in einer Hilfsfunktion ergänzt und erst später ausgeführt werden. Eine Stored Procedure kann an einer C#-Aufrufstelle nur namentlich referenziert werden, während ihre eigentliche Datenbanklogik in einer separaten SQL-Datei definiert ist. Aus einem einzelnen Treffer lässt sich daher nicht zuverlässig ableiten, welche Datenbankelemente tatsächlich betroffen sind (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Die zentrale praktische Herausforderung besteht deshalb nicht im bloßen Finden eines Wortes, sondern in der Rekonstruktion einer begründeten Evidenzkette über Datei-, Methoden-, Projekt- und Technologiegrenzen hinweg.

Für verantwortbare Wartungs- oder Architekturentscheidungen müssen Entwicklerinnen und Entwickler nachvollziehen können:

- welche Programmelemente beteiligt sind,
- welche Datenbankelemente betroffen sind,
- welche Zugriffsart vorliegt,
- welche Quellstellen die Aussage tragen,
- welche Beziehungen direkt beobachtet wurden,
- welche Beziehungen abgeleitet wurden,
- wo die Evidenz vollständig ist,
- wo die statische Analyse an ihre Grenzen stößt.

Cross-Repository Code Intelligence setzt an dieser praktischen Herausforderung an. Das System soll nicht nur Treffer sammeln, sondern strukturiert dokumentieren, welche Evidenz eine Aussage tatsächlich trägt (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Didaktische Erklärung - einfaches Deutsch

In einem großen Projekt steht die Antwort selten an nur einer Stelle. Eine Methode ruft eine andere Methode auf, diese arbeitet mit Entity Framework, und das Mapping zur echten Tabelle steht vielleicht in einer anderen Datei. Deshalb reicht es nicht, nur nach einem Wort zu suchen. Man muss den ganzen Weg der Evidenz rekonstruieren.

1.2 Beispiel einer verteilten Evidenzkette

Ein typischer, vereinfachter Zusammenhang könnte wie folgt aussehen:

- Eine Controller-Methode ruft eine Service-Methode auf.
- Die Service-Methode ruft eine Repository-Methode auf.
- Die Repository-Methode verwendet ein `DbSet<Patient>`.
- Eine LINQ-Abfrage greift auf die Property `DateOfBirth` zu.
- Eine Entity-Framework-Konfiguration ordnet die Entity `Patient` der Tabelle `Patients` zu.
- Eine weitere Mapping-Regel ordnet die Property `DateOfBirth` der Spalte `DateOfBirth` zu.
- Eine Materialisierungsoperation wie `ToListAsync()` führt die Abfrage aus.

Erst die Kombination dieser Beobachtungen kann eine Aussage wie die folgende tragen:

Die Repository-Methode liest statisch belegbar die Spalte `Patients.DateOfBirth`.

Fehlt dagegen die Property-zu-Spalte-Zuordnung, wäre möglicherweise nur eine tabellenbezogene Aussage unterstützt. Fehlt die Zuordnung zwischen `DbSet` und Entity, könnte die gesamte Antwort nicht ausreichend belegt sein (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Dieses Beispiel zeigt, warum Cross-Repository Code Intelligence zwischen lokalen Beobachtungen und abgeleiteten Relationen unterscheiden muss (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Status: synthetisches Beispiel zur Erklärung; kein bestätigter Danphe-Befund.

1.3 Praktische Relevanz für Wartung und Refactoring

Bei Änderungen an Datenmodellen müssen Entwicklerinnen und Entwickler häufig abschätzen, welche Codebereiche betroffen sind. Wird eine Tabellenspalte umbenannt, entfernt oder fachlich verändert, reicht eine textuelle Suche nach dem Spaltennamen nicht immer aus. Der Name kann im Code durch eine andere Property repräsentiert sein oder ausschließlich über Konfigurationen zugeordnet werden.

Ein evidenzbasiertes System könnte hier unterstützen, indem es nicht nur mögliche Treffer anzeigt, sondern den vollständigen Zusammenhang offenlegt:

- C#-Methode,
- verwendete Entity,
- Mapping,
- Tabelle oder Spalte,
- Zugriffsart,
- Quellstellen,
- offene Unsicherheiten.

Dadurch können Änderungen besser eingegrenzt und potenzielle Auswirkungen nachvollziehbarer beurteilt werden.

1.4 Praktische Relevanz für Fehleranalyse

Bei Fehlern in datenintensiven Systemen stellt sich häufig die Frage, wo ein bestimmter Wert gelesen, verändert oder persistiert wird. Lokale Debugging-Informationen reichen insbesondere dann nicht aus, wenn mehrere Hilfsmethoden, Services oder Datenzugriffsschichten beteiligt sind (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Cross-Repository Code Intelligence soll keine Laufzeitanalyse ersetzen. Es kann jedoch eine statische Evidenzbasis liefern, die mögliche Zugriffs- und Veränderungspfade sichtbar macht. Dadurch können Entwicklerinnen und Entwickler gezielter entscheiden, welche Codebereiche für eine weitere Untersuchung relevant sind (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

1.5 Praktische Relevanz für Datenschutz und Compliance

Für Datenschutz- und Compliance-Fragen ist nicht nur wichtig, dass ein bestimmter Begriff im Quellcode vorkommt. Entscheidend ist, ob ein konkreter Codepfad tatsächlich auf eine sensible Tabelle oder Spalte zugreift und ob dieser Zugriff lesend oder schreibend erfolgt (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Ein statischer Code-Database Evidence Graph kann dabei eine technische Grundlage schaffen, indem er nachvollziehbar dokumentiert:

- wo Datenbankoperationen erkannt wurden,
- welche Tabellen oder Spalten betroffen sind,
- welche Mappings den Zusammenhang herstellen,
- welche Aufrufpfade beteiligt sind,
- welche Teile der Aussage sicher und welche unaufgelöst sind.

Cross-Repository Code Intelligence liefert damit eine technische Evidenzbasis. Es ersetzt jedoch weder eine juristische Bewertung noch eine vollständige Datenschutzprüfung (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Wichtige Abgrenzung

Ein gefundener oder nicht gefundener Codepfad beweist allein keine rechtliche Konformität oder Nichtkonformität.

1.6 Beispielhafte Nutzungsperspektiven

Zur verständlichen Darstellung können drei beispielhafte Rollen verwendet werden.

Lara - Junior-Entwicklerin. Lara möchte vor einer Änderung wissen, welche Methoden eine bestimmte Tabelle oder Spalte lesen. Sie benötigt eine verständliche Liste von Methoden und die zugehörigen Quellstellen.

Sven - Senior-Architekt. Sven interessiert sich für indirekte Datenbankabhängigkeiten. Er möchte verstehen, ob eine Methode über Services, Repositories, Hilfsfunktionen oder Mappings zu einem Datenbankzugriff führt (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Andrea - Compliance-Verantwortliche. Andrea möchte nachvollziehen, über welche Codepfade bestimmte sensible Daten gelesen oder verändert werden können und welche technischen Belege für diese Aussage vorhanden sind.

Diese Rollen dienen zunächst ausschließlich der didaktischen Veranschaulichung. Sie sind nicht automatisch Bestandteil des experimentellen Designs. Falls Personas später in die Evaluation aufgenommen werden, müssen ihre Aufgaben, Informationsbedürfnisse und Bewertungskriterien wissenschaftlich begründet werden (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Status: illustrative Personas; keine empirischen Studienteilnehmerinnen.

1.7 Wissenschaftliche Relevanz

Das beschriebene Problem verbindet mehrere Forschungsbereiche:

- statische Programmanalyse,
- datenbankbezogene Codeanalyse,
- Analyse heterogener und mehrsprachiger Systeme,
- graphbasierte Wissens- und Code-Repräsentation,
- Provenienzmodellierung,
- repositoryweites Retrieval,
- evidenzbasierte Fragebeantwortung,
- Umgang mit Unsicherheit und Nichtbeantwortbarkeit.

Statische Analyse ermöglicht es, Programmstrukturen und potenzielle Beziehungen zu untersuchen, ohne das System auszuführen. Für C# können compilerbasierte Ansätze wie Roslyn Syntaxbäume, Symbole und semantische Informationen bereitstellen. Diese Informationen sind erforderlich, um zwischen einem bloßen Texttreffer und einem tatsächlich aufgelösten Methoden-, Property- oder Typzugriff zu unterscheiden (vgl. Microsoft, 2026a; Shatnawi et al., 2019).

Arbeiten zu Code Property Graphs zeigen, dass unterschiedliche Programmdarstellungen in einem gemeinsamen Graphmodell verbunden und über mehrstufige Traversierungen analysiert werden können (Yamaguchi et al., 2014). Für Cross-Repository Code Intelligence ist insbesondere die Idee relevant, dass komplexe Zusammenhänge nicht als isolierte Treffer, sondern als verbundene Pfade repräsentiert werden.

Forschung zur Analyse mehrsprachiger und heterogener Systeme weist darauf hin, dass realistische Softwareanalyse über eine einzelne Programmiersprache hinausgehen muss. Konfigurationen, Frameworks, String-Literale und unterschiedliche Artefakttypen können wesentliche Abhängigkeiten enthalten (Shatnawi et al., 2019).

Arbeiten zur statischen Lokalisierung von SQL-Ausführungsstellen zeigen außerdem, dass datenbankbezogene Operationen teilweise statisch erkannt werden können, während dynamisch erzeugte oder unvollständig auflösbare Query-Fragmente als Grenzen sichtbar bleiben müssen (Nagy et al., 2015).

Repositoryweite Retrievalansätze zeigen, dass relevante Informationen häufig außerhalb der unmittelbar betrachteten Datei liegen und durch eine systematische Suche über das Repository besser nutzbar gemacht werden können (Zhang et al., 2023). Für Cross-Repository Code Intelligence reicht thematisch relevanter Kontext jedoch nicht aus. Eine Antwort muss durch einen nachvollziehbaren Evidenzpfad getragen werden.

Die wissenschaftliche Relevanz von Cross-Repository Code Intelligence ergibt sich daher nicht aus einem einzelnen technischen Baustein. Sie entsteht aus der kontrollierten Verbindung von: (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

- statischer Extraktion,
- semantischer Auflösung,
- datenbankbezogener Normalisierung,
- provenienzerhaltender Repräsentation,
- graphbasierter Verknüpfung,
- repositoryweitem Retrieval,
- expliziter Antwortbarkeitsprüfung,
- kontrollierter Evaluation.

1.8 Vorläufige Positionierung des Projekts

Cross-Repository Code Intelligence ist kein allgemeines Codeverständnissystem. Es ist auch kein universelles Question-Answering-System und kein vollständiges Werkzeug zur Laufzeitanalyse (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Der Fokus liegt auf einer engeren wissenschaftlichen Aufgabe:

Nachvollziehbare statische Evidenz zu Datenbankoperationen in heterogenen C#- und SQL-Artefakten gewinnen, verknüpfen und unter expliziten Antwortbarkeitsregeln für repositoryweite Fragen nutzen (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Der wissenschaftliche Beitrag liegt damit nicht allein in der Verwendung von Roslyn, Graphtechnologie oder SQL-Analyse. Diese Technologien sind Mittel zur Umsetzung. Der Beitrag liegt in der Methodik, mit der unterschiedliche Beobachtungen in ein kanonisches Evidenzmodell überführt, als überprüfbare Pfade verbunden und getrennt hinsichtlich Extraktion und Retrieval evaluiert werden (vgl. Microsoft, 2026a; Shatnawi et al., 2019).

2. Wissenschaftliche Problemstellung

Kapitelorientierung - Schritt für Schritt

Thema	Welche wissenschaftliche Schwierigkeit untersucht wird
Problem	Treffer sind nicht automatisch Evidenz, und plausible Antworten können falsch oder unvollständig sein.
Lösung	Extraktion, Repräsentation, Retrieval und Antwortbarkeit werden getrennt operationalisiert.
Ergebnis	Messbare Untersuchungsdimensionen und prüfbare Systementscheidungen.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Die wissenschaftliche Problemstellung wird aus der Perspektive eines evidenzbasierten Assistenzsystems formuliert. Cross-Repository Code Intelligence soll relevante Hinweise aus heterogenen C#- und datenbankbezogenen Artefakten erfassen, diese als strukturierte Wissens- und Evidenzbasis organisieren und nur dann eine Antwort ausgeben, wenn die zugrunde liegende Evidenz die Schlussfolgerung ausreichend trägt (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Im Mittelpunkt steht daher nicht nur die technische Erkennung möglicher READ- und WRITE-Beziehungen. Entscheidend ist vielmehr, wie ein System statisch gewonnene Evidenz so verarbeiten kann, dass:

- Beobachtungen korrekt extrahiert werden,
- Beziehungen nachvollziehbar abgeleitet werden,
- die Herkunft jeder Aussage erhalten bleibt,
- Unsicherheit sichtbar bleibt,
- vollständige Evidenzpfade abgerufen werden,
- nicht ausreichend belegte Antworten verhindert werden,
- begründete Verweigerungen als korrektes Verhalten messbar werden.

Die wissenschaftliche Problemstellung umfasst drei eng miteinander verbundene Dimensionen: Extraktion, Repräsentation sowie Retrieval und Antwortbarkeit (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

2.1 Extraktion als Gewinnung typisierter Beobachtungen

Die erste Dimension betrifft die statische Gewinnung relevanter Beobachtungen aus heterogenen Artefakten.

Cross-Repository Code Intelligence muss in C#-Code, Entity-Framework-Konfigurationen, SQL-Dateien und gegebenenfalls Stored-Procedure-Bezügen diejenigen Strukturen erkennen, die für Datenbankoperationen relevant sind. Dazu können insbesondere gehören: (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

- Ableitungen von DbContext,
- DbSet-Properties,
- LINQ-Abfragen,
- Query-Materialisierungen,
- Aufrufe wie Find, First, Any, ToList oder ToListAsync,

- Mutationsoperationen wie Add, Update oder Remove,
- Persistenzoperationen wie SaveChanges,
- ADO.NET-Typen und Befehle,
- eingebettete SQL-Strings,
- SQL-SELECT-, INSERT-, UPDATE- oder DELETE-Anweisungen,
- Aufrufe von Stored Procedures,
- Entity-zu-Tabelle-Mappings,
- Property-zu-Spalte-Mappings,
- Aufrufbeziehungen zwischen Methoden.

Ziel ist nicht das bloße Auffinden dieser Wörter. Das System muss möglichst bestimmen, welches Symbol, welcher Typ, welche Methode oder welche Property an einer konkreten Stelle gemeint ist.

Eine Beobachtung soll daher mindestens dokumentieren:

- den Quellort,
- das erkannte Muster,
- den Detektor,
- die angewendete Regel,
- die beteiligten Programmelemente,
- den vorläufigen Operationstyp,
- den Evidenzstatus,
- gegebenenfalls bestehende Unsicherheiten.

Ein Fehler in der Extraktion kann unterschiedliche Formen annehmen:

- Eine relevante Stelle wird nicht erkannt.
- Eine irrelevante Stelle wird als Evidenz interpretiert.
- Ein Symbol wird falsch aufgelöst.
- Ein Mapping wird übersehen.
- Ein Mutationssignal wird fälschlich als persistierter WRITE bewertet.
- Eine Query-Konstruktion wird mit einer Query-Ausführung verwechselt.
- Ein reiner Texttreffer wird als semantische Beziehung behandelt.

Die Extraktion bildet die Grundlage aller späteren Schritte. Fehler auf dieser Ebene können durch gutes Retrieval nicht vollständig ausgeglichen werden (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Didaktische Erklärung - einfaches Deutsch

Die Extraktion ist die Wahrnehmungsstufe des Systems. Das System sucht nicht nur nach Wörtern, sondern versucht zu erkennen, welches Programmelement gemeint ist und ob die Stelle wirklich etwas mit einem Datenbankzugriff zu tun hat.

2.2 Repräsentation als provenienzerhaltende Wissensstruktur

Die zweite Dimension betrifft die Repräsentation der gewonnenen Beobachtungen.

Beobachtungen aus unterschiedlichen Technologien können nicht sinnvoll genutzt werden, wenn sie nur als isolierte Treffer gespeichert werden. Sie müssen in ein gemeinsames Modell überführt werden, das ihre Unterschiede berücksichtigt, aber eine gemeinsame Auswertung ermöglicht.

Das kanonische Evidenzmodell muss insbesondere unterscheiden zwischen:

- Artefakten,
- Programmelementen,
- Datenbankelementen,
- direkten Beobachtungen,
- abgeleiteten Relationen,
- strukturellen Beziehungen,
- fachlichen READ- und WRITE-Beziehungen,

- Provenienz,
- Unsicherheit,
- Antwortstatus.

Die Repräsentation muss verhindern, dass technologiespezifische Informationen zu früh zu einer flachen Aussage verdichtet werden. Beispielsweise darf ein Zugriff auf eine Entity-Property nicht ohne nachvollziehbare Mapping-Evidenz automatisch als Zugriff auf eine konkrete physische Spalte dargestellt werden (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Cross-Repository Code Intelligence leitet daraus folgende Designanforderung ab:

Direkte Beobachtungen, abgeleitete Relationen und endgültige Antworten müssen getrennt modelliert werden.

Eine direkte Beobachtung könnte beispielsweise dokumentieren, dass eine Methode auf eine Entity-Property zugreift. Eine abgeleitete Relation könnte anschließend über ein Mapping die Verbindung zu einer Datenbankspalte herstellen. Erst wenn der gesamte erforderliche Pfad vollständig ist, darf eine Antwort formuliert werden (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Die Provenienz jeder Beobachtung und jeder abgeleiteten Relation muss erhalten bleiben. Dazu gehören beispielsweise: Datei, Projekt, Methode, Symbol, Position, Detektor, Regel, Mappingquelle und Ableitungsschritte.

Ohne diese Informationen könnte ein System zwar plausible Antworten liefern, aber nicht zeigen, warum diese Antworten gelten. Eine solche Antwort wäre für wissenschaftliche Evaluation, Wartung und Fehleranalyse nur eingeschränkt brauchbar (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Designentscheidung von Cross-Repository Code Intelligence: Provenienz und Unsicherheit werden nicht nachträglich aus der Repräsentation entfernt oder zu einer scheinbar sicheren Beziehung verdichtet (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

2.3 Retrieval als Zusammenstellung erforderlicher Evidenz

Die dritte Dimension betrifft das Retrieval.

Retrieval bedeutet in dieser Arbeit nicht nur, relevante Dateien oder Textstellen zu finden. Für eine konkrete Frage muss das System genau die Beobachtungen und Relationen zusammenstellen, die für eine Antwort erforderlich sind (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Die Frage „Welche Methoden lesen die Spalte X?“ benötigt beispielsweise nicht nur Treffer für den Namen X. Abhängig vom Korpus können folgende Evidenzteile erforderlich sein: (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

- Methode,
- Property-Zugriff,
- Query-Struktur,
- Query-Ausführung,
- Entity-Typ,
- DbSet- oder Kontextbezug,
- Entity-zu-Tabelle-Mapping,
- Property-zu-Spalte-Mapping,
- READ-Klassifikation,
- genaue Quellprovenienz.

Graphbasiertes Retrieval soll solche mehrstufigen Beziehungen als Evidenzpfade rekonstruieren. Die Qualität des Retrievals hängt daher nicht nur davon ab, ob relevante Fragmente gefunden werden, sondern auch davon, ob: (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

- alle erforderlichen Pfadteile vorhanden sind,
- keine ungültigen Kanten verwendet werden,
- keine wesentliche Evidenz ausgelassen wird,
- irrelevante Evidenz begrenzt wird,
- die Herkunft jedes Pfadteils inspizierbar bleibt.

2.4 Antwortbarkeit als kontrollierte Systementscheidung

Ein zentrales Merkmal von Cross-Repository Code Intelligence ist die explizite Prüfung der Antwortbarkeit.

Eine Frage gilt nicht automatisch als beantwortbar, nur weil das System einzelne relevante Treffer gefunden hat. Eine verantwortbare Antwort setzt voraus, dass die für den jeweiligen Fragetyp definierten Muss-Bedingungen erfüllt sind (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Die Arbeit unterscheidet vorläufig zwischen folgenden Zuständen:

- **SUPPORTED** - Die notwendige Evidenz ist vollständig vorhanden und der erforderliche Pfad ist durch direkte Beobachtungen oder dokumentierte Ableitungsregeln gestützt.
- **PARTIALLY_SUPPORTED** - Ein Teil der erforderlichen Evidenz ist vorhanden, aber mindestens ein wesentlicher Schritt fehlt oder bleibt ungelöst.
- **NOT_ANSWERABLE** - Die vorhandene Evidenz reicht innerhalb des definierten statischen Analyseumfangs nicht aus, um die verlangte Aussage zu tragen.

Eine begründete Verweigerung liegt vor, wenn das System eine nicht ausreichend belegte Frage nicht beantwortet und die fehlende Evidenz explizit benennt. Mögliche Gründe sind: (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

- fehlendes Mapping,
- unaufgelöstes Symbol,
- dynamisch erzeugtes SQL,
- fehlende Stored-Procedure-Definition,
- unterbrochene Aufrufkette,
- nicht unterstütztes ORM-Muster,
- fehlender Persistenztrigger,
- unzugängliches Artefakt,
- widersprüchliche Evidenz.

Die Verweigerung ist damit kein technisches Versagen, sondern kann ein gewünschtes Ergebnis sein. Eine nicht gestützte Antwort ist dagegen problematisch, weil sie den Anschein von Sicherheit erzeugt, obwohl die Evidenz unvollständig ist (vgl. Liu et al., 2026; Abedu et al., 2025).

Didaktische Erklärung - einfaches Deutsch

Das System soll nicht immer antworten. Es soll zuerst prüfen, ob alle wichtigen Beweise vorhanden sind. Fehlt ein notwendiger Schritt, soll es sagen: „Diese Frage kann ich mit der vorhandenen statischen Evidenz nicht sicher beantworten.“

2.5 Zusammenhang der drei Dimensionen

Extraktion, Repräsentation und Retrieval dürfen nicht als voneinander unabhängige technische Komponenten betrachtet werden.

Eine fehlende Beobachtung in der Extraktion führt zu einer Lücke im Evidenzmodell. Eine ungeeignete Repräsentation kann Provenienz oder Unsicherheit verlieren. Ein unvollständiges Retrieval kann vorhandene Evidenz übersehen. Eine zu großzügige Antwortlogik kann trotz unvollständiger Evidenz eine nicht gestützte Antwort erzeugen (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Die Gesamtqualität des Systems ergibt sich daher aus dem Zusammenspiel aller Stufen:

Artefakte → Beobachtungen → normalisierte Evidenz → abgeleitete Relationen → Evidenzpfade → Antwortbarkeitsprüfung → Antwort oder begründete Verweigerung (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Diese Verarbeitungskette bildet die wissenschaftliche Grundstruktur von Cross-Repository Code Intelligence.

2.6 Eingrenzung der wissenschaftlichen Problemstellung

Cross-Repository Code Intelligence erhebt nicht den Anspruch, ein vollständiges allgemeines Programmverständnis zu erzeugen. Ebenso wenig soll das System sämtliche Laufzeitverhalten, alle C#-Sprachmerkmale, alle ORMs oder alle SQL-Dialekte vollständig analysieren (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Der verbindliche Kern konzentriert sich auf:

- statisch analysierbare C#-Artefakte,
- die im Korpus tatsächlich vorkommenden Entity-Framework-Muster,
- READ- und WRITE-Beziehungen,
- Tabellenebene,
- Spaltenebene nur bei ausreichender Mapping-Evidenz,
- Quellprovenienz,
- normalisierte Evidenz,
- graphbasierte und nicht graphbasierte Retrievalvarianten,
- Antwortbarkeit und begründete Verweigerung,
- kontrollierte Evaluation auf einem eingefrorenen Korpus.

Dynamische oder schwer auflösbare Konstrukte werden nicht stillschweigend vereinfacht. Sie werden als ungelöste Evidenz dokumentiert und können die Antwortbarkeit einschränken. Dazu gehören insbesondere: Reflection, vollständig dynamisches SQL, Laufzeitkonfigurationen, komplexes Dynamic Dispatch, externe Dienste, nicht verfügbare Datenbankdefinitionen und schwer rekonstruierbare interprozedurale Pfade.

Die präzisierte wissenschaftliche Problemstellung lautet damit:

Wie kann in einem realistischen, eingefrorenen C#/.NET-Korpus eine nachvollziehbare und provenienzerhaltende Evidenzbasis zu Datenbankoperationen statisch gewonnen, strukturiert repräsentiert und für repositoryweite Fragen verantwortbar genutzt werden, wobei Unsicherheit, Analysegrenzen und begründete Verweigerung explizit berücksichtigt werden? (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

3. Begriffe und konzeptionelle Grundlagen

Kapitelorientierung - Schritt für Schritt

Thema	Welches Wissen für das Projekt benötigt wird
Problem	Ohne präzise Begriffe werden Designentscheidungen, Messgrößen und Ergebnisse vermischt.
Lösung	Die zentralen Konzepte aus statischer Analyse, Datenbankzugriff, Graphen, Evidenz und Evaluation werden definiert.
Ergebnis	Ein verbindliches Vokabular für Implementierung und Präsentation.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Dieses Kapitel führt die zentralen Begriffe ein, auf denen die Konzeption, Implementierung und Evaluation von Cross-Repository Code Intelligence basiert. Ziel ist nicht, lediglich einzelne Definitionen aufzulisten. Die Begriffe werden vielmehr als zusammenhängendes konzeptionelles Modell dargestellt, das erklärt, wie aus heterogenen C#- und datenbankbezogenen Artefakten überprüfbare Aussagen über Datenbankoperationen entstehen können (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Das Kapitel unterscheidet dabei durchgängig zwischen drei Ebenen:

- Artefakte und Programmelemente, die im untersuchten Korpus vorhanden sind.
- Evidenz, die durch statische Analyse aus diesen Artefakten gewonnen und miteinander verbunden wird.
- Antworten, die nur dann ausgegeben werden dürfen, wenn die erforderliche Evidenz vollständig und gültig vorliegt.

Diese Trennung ist für die gesamte Arbeit verbindlich. Ein gefundener Text oder ein erkannter Methodenaufruf ist noch keine vollständige Datenbankrelation. Eine einzelne Datenbankrelation ist wiederum nicht automatisch eine ausreichende Antwort auf eine repositoryweite Frage (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Didaktische Erklärung - einfaches Deutsch

Dieses Kapitel erklärt die Fachbegriffe, die später immer wieder verwendet werden. Wichtig ist besonders die Unterscheidung zwischen einem gefundenen Hinweis, einer daraus abgeleiteten Beziehung und einer vollständigen Antwort. Cross-Repository Code Intelligence darf aus einem kleinen Hinweis nicht sofort eine große Behauptung machen.

3.1 Grundlagen der statischen Programmanalyse

3.1.1 Begriff und Ziel der statischen Programmanalyse

Statische Programmanalyse bezeichnet die Untersuchung eines Programms und seiner zugehörigen Artefakte, ohne das Anwendungssystem auszuführen. Analysiert werden beispielsweise Quellcode, Projektdateien, Konfigurationen, Mappingdefinitionen und SQL-Artefakte.

Für Cross-Repository Code Intelligence ist die statische Analyse der methodische Ausgangspunkt. Das System soll nachvollziehbare Aussagen über Datenbankoperationen aus den vorhandenen Artefakten

rekonstruieren, ohne auf konkrete Laufzeitbeobachtungen angewiesen zu sein (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Die statische Analyse kann unter anderem folgende Informationen liefern: syntaktische Struktur eines Programms, deklarierte Klassen, Methoden und Properties, Methodenaufrufe, Typinformationen, Symbolbeziehungen, Vererbungsbeziehungen, Zugriffe auf Properties und Felder, Entity-Framework-Strukturen, Mappingdefinitionen, SQL-Anweisungen sowie mögliche READ- und WRITE-Beziehungen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Die statische Analyse rekonstruiert mögliche oder durch Artefakte belegte Beziehungen. Sie beweist jedoch nicht automatisch, dass ein bestimmter Pfad in einem konkreten Programmlauf tatsächlich ausgeführt wird.

Beispielsweise kann eine Methode statisch einen belegten Pfad zu einer Datenbankabfrage besitzen. Daraus folgt, dass diese Methode unter geeigneten Laufzeitbedingungen die betreffende Tabelle lesen kann. Daraus folgt jedoch nicht, dass die Methode in jedem Programmlauf aufgerufen oder die Abfrage tatsächlich ausgeführt wird (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Cross-Repository Code Intelligence verwendet deshalb bewusst den Ausdruck **statisch unterstützte Beziehung**.

Verbindliche Aussage: Eine statisch unterstützte READ- oder WRITE-Beziehung beschreibt eine durch die analysierten Artefakte belegte Zugriffsmöglichkeit innerhalb des definierten Analyseumfangs. Sie ist kein Nachweis einer konkreten Laufzeitausführung.

3.1.2 Statische und dynamische Analyse

Statische und dynamische Analyse betrachten Programme aus unterschiedlichen Perspektiven.

Die statische Analyse untersucht den Quellcode und weitere Artefakte ohne Ausführung. Sie kann potenzielle Pfade und strukturelle Zusammenhänge erkennen, auch wenn diese in einem konkreten Testlauf nicht auftreten.

Die dynamische Analyse untersucht dagegen das Verhalten eines tatsächlich ausgeführten Systems. Sie kann beispielsweise aufzeichnen, welche Methoden real aufgerufen wurden, welche SQL-Abfragen ausgeführt wurden, welche konkreten Parameter verwendet wurden, welche Daten gelesen oder verändert wurden und welcher Laufzeittyp bei polymorphen Aufrufen vorlag (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Beide Ansätze besitzen unterschiedliche Stärken und Grenzen:

Aspekt	Statische Analyse	Dynamische Analyse
Programmausführung erforderlich	Nein	Ja
Betrachtung nicht ausgeführter Pfade	Möglich	Nicht im beobachteten Lauf
Konkrete Laufzeitwerte	Normalerweise nicht verfügbar	Verfügbar
Abdeckung	Potenziell breiter	Auf beobachtete Ausführungen beschränkt
Dynamisches SQL	Häufig nur teilweise auflösbar	Als ausgeführter Text beobachtbar
Reproduzierbarkeit	Bei festem Snapshot hoch	Abhängig von Eingaben und Umgebung
Primäre Rolle in Cross-Repository Code Intelligence	Verbindlicher Kern	Außerhalb des Pflichtumfangs

Cross-Repository Code Intelligence ist als statisches Forschungsartefakt konzipiert. Dynamische Analyse kann in zukünftigen Arbeiten als ergänzende Evidenzquelle betrachtet werden, gehört jedoch nicht zum verbindlichen Kern dieser Bachelorarbeit (vgl. Hevner et al., 2004).

Didaktische Erklärung - einfaches Deutsch

Statische Analyse liest den Code, ohne das Programm zu starten. Dynamische Analyse beobachtet das laufende Programm. Cross-Repository Code Intelligence arbeitet hauptsächlich statisch. Deshalb kann es zeigen, was laut Code möglich und belegt ist, aber nicht, was in einem bestimmten Moment wirklich ausgeführt wurde.

3.1.3 Quelltext als Zeichenfolge und als Programmstruktur

Ein Quelltext kann auf unterschiedlichen Abstraktionsebenen betrachtet werden.

Auf der einfachsten Ebene ist Quelltext eine Zeichenfolge. Eine Textsuche kann feststellen, ob ein bestimmter Name in einer Datei vorkommt. Diese Information ist für eine erste Orientierung nützlich, reicht jedoch für eine verlässliche Programmanalyse nicht aus (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Der Ausdruck `Patients` könnte beispielsweise vorkommen: als Name eines `DbSet`, als Tabellename in einem SQL-String, in einem Kommentar, als Property eines DTOs, als Testvariable, als Teil eines längeren Namens oder in nicht ausgeführtem oder generiertem Code (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Erst eine strukturierte syntaktische und semantische Analyse kann diese Fälle voneinander unterscheiden.

Cross-Repository Code Intelligence vergleicht deshalb bewusst verschiedene Analyseebenen: text- oder namensbasierte Analyse, syntaxbasierte Analyse, semantische Analyse mit Symbolauflösung sowie semantische Analyse mit datenbankspezifischen Mapping- und Evidenzregeln. Diese Ebenen bilden später die Grundlage der Extraktionskonfigurationen (vgl. Microsoft, 2026a; Shatnawi et al., 2019).

3.1.4 Abstract Syntax Tree

Ein Abstract Syntax Tree, kurz AST, repräsentiert die syntaktische Struktur eines Programms als Baum. Dabei werden relevante Sprachkonstruktionen als Knoten dargestellt (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

Typische Knoten eines C#-Syntaxbaums sind: Namespace-Deklarationen, Klassen, Methoden, Parameter, lokale Variablen, Methodenaufrufe, Memberzugriffe, Zuweisungen, Lambda-Ausdrücke, Rückgabeanweisungen und Bedingungen (vgl. Microsoft, 2026a; Shatnawi et al., 2019).

Betrachtet man beispielsweise den Ausdruck

```
context.Patients.Where(p => p.IsActive).ToList();
```

als reinen Text, sieht man lediglich eine Zeichenfolge. Ein Syntaxbaum kann dagegen unterscheiden zwischen dem Ausdruck `context.Patients`, dem Methodenaufruf `Where`, dem Lambda-Parameter `p`, dem Property-Zugriff `p.IsActive` und dem abschließenden Methodenaufruf `ToList` (vgl. Microsoft, 2026a; Shatnawi et al., 2019).

Diese Struktur ermöglicht präzisere Regeln. Ein Detektor kann beispielsweise gezielt nach einem Methodenaufruf suchen, dessen Empfänger eine LINQ-Abfrage ist, statt lediglich nach dem Wort `ToList` zu suchen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Ein AST beschreibt jedoch hauptsächlich die syntaktische Form. Er beantwortet nicht automatisch, welchen Typ `context` besitzt, ob `Patients` tatsächlich ein `DbSet<Patient>` ist, welche konkrete `Where`-Methode aufgerufen wird oder ob `IsActive` zu einer gemappten Datenbankspalte gehört. Dafür ist eine semantische Analyse erforderlich (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Merksatz: Der Syntaxbaum zeigt, wie der Code aufgebaut ist. Das semantische Modell zeigt, was die Elemente im konkreten Programm bedeuten.

3.1.5 Semantisches Modell

Ein semantisches Modell verbindet syntaktische Stellen mit ihrer Bedeutung im konkreten Programmkontext. In der .NET-Compilerplattform Roslyn kann über ein semantisches Modell beispielsweise ermittelt werden, welches Symbol an einer Syntaxstelle referenziert wird, welchen Typ ein Ausdruck besitzt, welche Überladung einer Methode aufgerufen wird, ob eine Konvertierung stattfindet und zu welchem Namespace oder Assembly ein Typ gehört (vgl. Microsoft, 2026a; Shatnawi et al., 2019).

Für Cross-Repository Code Intelligence ist dies besonders wichtig, weil identische Namen unterschiedliche Bedeutungen besitzen können. Der Ausdruck (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

```
repository.Update(patient);
```


liefert als Text lediglich den Namen Update. Eine semantische Analyse kann dagegen bestimmen, zu welchem Typ repository gehört, welche konkrete Update-Methode aufgerufen wird, ob diese Methode zu Entity Framework, zu einem projektspezifischen Repository oder zu einer anderen Bibliothek gehört und welchen Typ das Argument patient besitzt (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Dadurch können False Positives reduziert werden. Nicht jede Methode mit dem Namen Add, Remove, Update, Find oder Save stellt eine Datenbankoperation dar. Das semantische Modell bildet daher eine zentrale Grundlage für die präzise Erkennung von Datenbankevidenz in C#.

3.1.6 Symbol und Symbolauflösung

Ein Symbol repräsentiert ein eindeutig identifizierbares Programmelement. Dazu gehören beispielsweise: Namespace, Typ, Klasse, Interface, Methode, Property, Feld, Parameter und lokale Variable.

Die Symbolauflösung ordnet einer Syntaxstelle das tatsächlich gemeinte Symbol zu. Diese Zuordnung ist notwendig, weil Namen allein nicht eindeutig sind. Zwei Klassen können eine Methode namens Save enthalten. Mehrere Namespaces können einen Typ mit demselben Namen definieren. Eine Methode kann mehrfach überladen sein (vgl. Microsoft, 2026a; Shatnawi et al., 2019).

Cross-Repository Code Intelligence soll deshalb stabile Symbolinformationen verwenden, um Beziehungen nicht nur über Textnamen, sondern möglichst über eindeutig aufgelöste Programmelemente zu modellieren (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Ein Symbol kann im Evidenzmodell unter anderem durch folgende Merkmale beschrieben werden: vollständig qualifizierter Name, Symboltyp, enthaltender Namespace, enthaltender Typ, Methodensignatur, Projektzugehörigkeit, Quellort und stabile projektinterne ID.

Typischer Fehler: Zwei gleich geschriebene Namen werden automatisch als dasselbe Programmelement behandelt. Korrekte Vorgehensweise: Namen werden durch Projekt-, Typ-, Signatur- und Symbolinformationen disambiguiert (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.1.7 Syntax-, semantik- und domänenspezifische Regeln

Cross-Repository Code Intelligence unterscheidet drei Regelklassen.

Syntaxbasierte Regeln verwenden die Form des Codes. Beispiel: Ein Invocation-Expression-Knoten mit dem Methodennamen ToList wurde erkannt. Eine solche Beobachtung ist struktureller als ein Texttreffer, sagt aber noch nicht sicher, welche ToList-Methode gemeint ist (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Semantikbasierte Regeln verwenden aufgelöste Typen und Symbole. Beispiel: Der erkannte Aufruf ist eine LINQ-Materialisierungsoperation auf einem IQueryable<Patient>. Diese Aussage ist stärker als die reine Syntaxbeobachtung (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Domänenspezifische Evidenzregeln übersetzen syntaktische und semantische Informationen in datenbankbezogene Evidenz. Beispiel: Eine materialisierte LINQ-Abfrage auf einem DbSet<Patient> erzeugt eine READ-Beobachtung für die Entity Patient. Erst weitere Mappingevidenz kann daraus eine tabellen- oder spaltenbezogene Relation ableiten (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Diese Trennung ist für die Evaluation wichtig. Sie ermöglicht, die Wirkung unterschiedlicher Analyseebenen kontrolliert zu vergleichen.

3.2 Datenbankzugriffe in C#/.NET

3.2.1 Heterogene Formen des Datenbankzugriffs

Datenbankzugriffe in C#/.NET-Systemen können über verschiedene Technologien und Abstraktionsebenen erfolgen. In realen Systemen können mehrere Mechanismen gleichzeitig auftreten. Dazu gehören insbesondere: Entity Framework oder Entity Framework Core, LINQ-basierte Abfragen, Repository- und Unit-of-Work-Muster, ADO.NET, eingebettete SQL-Strings, externe SQL-Dateien, Stored Procedures, projektspezifische Datenzugriffsabstraktionen sowie Hilfsmethoden und Wrapper (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Diese Heterogenität ist ein zentrales Merkmal des Forschungsproblems. Eine Analyse, die nur einen Zugriffsmechanismus berücksichtigt, kann relevante Evidenz aus anderen Technologien übersehen (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

In Cross-Repository Code Intelligence bezeichnet *heterogen* daher nicht nur unterschiedliche Dateiformate. Der Begriff umfasst mehrere Dimensionen: unterschiedliche Programmiersprachen oder Artefaktsprachen, unterschiedliche Datenzugriffstechnologien, unterschiedliche Architekturschichten, unterschiedliche Formen direkter und indirekter Evidenz sowie unterschiedliche Projekte oder Datenbankkontexte (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.2.2 Entity Framework als objektrelationaler Mapper

Entity Framework, kurz EF, ist eine Datenzugriffstechnologie beziehungsweise ein objektrelationaler Mapper. Ein ORM bildet Objekte und Klassen des Anwendungsprogramms auf relationale Datenbankstrukturen ab. Dabei werden typischerweise folgende Zuordnungen hergestellt: Entity-Klasse → Tabelle, Property → Spalte, Objektrelation → Fremdschlüsselbeziehung, C#-Abfrage → SQL-Abfrage, Objektänderung → Datenbankänderung (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Ein ORM reduziert die Notwendigkeit, SQL für jeden Zugriff manuell zu formulieren. Gleichzeitig erschwert es eine rein textbasierte Datenbankanalyse, weil der physische Tabellen- oder Spaltenname an der Zugriffsstelle möglicherweise nicht sichtbar ist (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

```
var activePatients = context.Patients
.Where(patient => patient.IsActive)
.ToList();
```

Der Code enthält möglicherweise weder den exakten Datenbankschemanamen noch den physischen Tabellennamen. Die Verbindung zur Datenbank kann sich erst aus Entity-Framework-Konfigurationen oder Konventionen ergeben. Für Cross-Repository Code Intelligence ist Entity Framework deshalb sowohl eine wichtige Evidenzquelle als auch eine Ursache mehrstufiger Ableitungen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.2.3 DbContext

Ein DbContext ist das zentrale Kontextobjekt in Entity-Framework-basierten Anwendungen. Er repräsentiert eine Sitzung beziehungsweise eine Arbeitseinheit mit dem Datenmodell und der Datenbank. Ein DbContext übernimmt typischerweise Aufgaben wie die Bereitstellung von DbSet-Properties, die Konstruktion und Ausführung von Queries, die Materialisierung von Datenbankergebnissen, Change Tracking, die Verwaltung von Entity-Zuständen, die Persistierung von Änderungen und die Konfiguration von Mappings (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Eine Klasse, die von DbContext ableitet, ist daher ein zentraler Einstiegspunkt für die Analyse.

```
public class HospitalContext : DbContext
{
    public DbSet<Patient> Patients { get; set; }
    public DbSet<Visit> Visits { get; set; }
}
```

Aus dieser Struktur können zunächst folgende Beobachtungen gewonnen werden: HospitalContext ist ein Datenbankkontext, Patients ist ein DbSet<Patient> und Visits ist ein DbSet<Visit>. Diese Beobachtungen stellen noch keine konkreten READ- oder WRITE-Operationen dar. Sie bilden jedoch wichtige Struktur- und Mappingevidenz (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.2.4 DbSet

Ein DbSet repräsentiert in Entity Framework eine Menge von Entitäten eines bestimmten Typs, die im Kontext abgefragt oder verändert werden können. Ein DbSet erfüllt mehrere Rollen: Ausgangspunkt für

LINQ-Abfragen, Zugriffspunkt für Entity-Operationen, Verbindung zwischen DbContext und Entity-Typ sowie möglicher Anker für Entity-zu-Tabelle-Mappings (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Der bloße Zugriff auf ein DbSet ist jedoch nicht automatisch eine ausgeführte Datenbankabfrage.

```
var query = context.Patients.Where(p => p.IsActive);
```

Hier wird zunächst eine Abfrage konstruiert. Abhängig von Provider und weiterer Verwendung kann die tatsächliche Ausführung erst später erfolgen: (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

```
var result = query.ToList();
```

Cross-Repository Code Intelligence muss deshalb zwischen folgenden Konzepten unterscheiden: Zugriff auf ein DbSet, Konstruktion einer Query, Materialisierung oder Ausführung der Query, Zugriff auf Properties innerhalb der Query sowie Mapping der Entity und ihrer Properties (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.2.5 LINQ

Language Integrated Query, kurz LINQ, ist eine in C# integrierte Abfragesyntax beziehungsweise API zur Formulierung von Abfragen über Datenquellen. LINQ-Ausdrücke können gegen unterschiedliche Datenquellen ausgeführt werden: In-Memory-Sammlungen, Datenbankprovider, XML-Strukturen und projektspezifische Queryprovider (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Ein Aufruf von Where, Select, Any oder First ist daher nicht allein aufgrund seines Namens automatisch eine Datenbankoperation. Entscheidend ist der Typ der Datenquelle und die verwendete Implementierung.

Cross-Repository Code Intelligence muss insbesondere unterscheiden zwischen IEnumerable<T> und In-Memory-Verarbeitung, IQueryable<T> und providerbasierter Query, Entity-Framework-DbSet sowie projektspezifischen Queryabstraktionen. Diese Unterscheidung erfordert semantische Typinformationen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.2.6 Query-Konstruktion und Query-Ausführung

Viele LINQ-Abfragen verwenden Deferred Execution. Das bedeutet, dass die Definition einer Query nicht notwendigerweise ihre sofortige Ausführung bewirkt (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

```
var query = context.Patients.Where(p => p.IsActive);
```

Dieser Ausdruck kann zunächst nur eine Queryrepräsentation erzeugen. Eine Ausführung oder Materialisierung erfolgt möglicherweise erst durch query.ToList();, query.Any(); oder eine foreach-Iteration über die Query.

Für Cross-Repository Code Intelligence ist diese Trennung konzeptionell wichtig. Eine Query-Konstruktion kann eine potenzielle READ-Evidenz darstellen. Eine erkannte Materialisierungs- oder Terminaloperation liefert stärkere Evidenz dafür, dass der Datenzugriff an dieser Stelle ausgelöst wird (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Das Evidenzmodell sollte diese Stufen getrennt repräsentieren, beispielsweise durch QUERY_CONSTRUCTION, QUERY_EXECUTION, READS_ENTITY, READS_TABLE und READS_COLUMN. Die endgültigen Relationstypen werden im Evidenzschema präzisiert. Die konzeptionelle Trennung bleibt jedoch verbindlich.

Typischer Fehler: Jede Verwendung von Where wird sofort als ausgeführter Datenbank-READ klassifiziert. Korrekte Interpretation: Zuerst wird geprüft, ob die Quelle datenbankgebunden ist und ob beziehungsweise wo die Query ausgeführt oder materialisiert wird.

3.2.7 Change Tracking und Persistierung

Entity Framework verfolgt Änderungen an geladenen oder hinzugefügten Entities. Diese Änderungen können zunächst nur im Speicher des DbContext vorhanden sein (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

```
patient.Name = "New Name";
```

Diese Zuweisung verändert zunächst ein C#-Objekt. Sie beweist allein noch nicht, dass eine Datenbankspalte geschrieben wird. Erst ein Persistenztrigger wie `context.SaveChanges();` oder `await context.SaveChangesAsync();` kann die im Kontext verfolgten Änderungen in Datenbankoperationen überführen.

Cross-Repository Code Intelligence muss daher unterscheiden zwischen Mutationssignal, Entity-Zustandsänderung, Kontextzugehörigkeit, Mapping zum Datenbankziel und Persistenztrigger. Eine belastbare WRITE-Relation kann je nach verwendetem Muster erst aus der Kombination dieser Evidenz entstehen (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

3.2.8 Add, Update und Remove

Methoden wie Add, Update und Remove liefern Hinweise auf beabsichtigte Veränderungen.

```
context.Patients.Add(patient);
context.Patients.Update(patient);
context.Patients.Remove(patient);
```

Diese Aufrufe können als Mutationsbeobachtungen klassifiziert werden. Sie stellen jedoch nicht notwendigerweise bereits eine vollständig persistierte Datenbankoperation dar.

Eine vollständige statische WRITE-Ableitung kann folgende Evidenz benötigen: aufgelöster Mutationsaufruf, betroffener Entity-Typ, Zugehörigkeit zu einem DbContext oder DbSet, Mapping zur Tabelle oder Spalte, erreichbarer oder zugeordneter Persistenztrigger und dokumentierter Ableitungspfad. Die genaue Operationalisierung hängt vom Korpus und dem gewählten Analyseumfang ab (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.2.9 ADO.NET

ADO.NET ermöglicht direktere Datenbankzugriffe über Klassen wie `SqlConnection`, `SqlCommand`, `DbConnection`, `DbCommand`, `SqlDataReader`, `ExecuteReader`, `ExecuteScalar` und `ExecuteNonQuery` (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

```
using var command = new SqlCommand(
    "SELECT DateOfBirth FROM Patients WHERE Id = @id",
    connection);
using var reader = command.ExecuteReader();
```

Hier ist der SQL-Text direkter sichtbar als bei einem ORM-basierten Zugriff. Dennoch bleiben Analyseprobleme bestehen: SQL kann dynamisch zusammengesetzt werden, der Command-Text kann in einer anderen Methode erzeugt werden, Tabellen- oder Spaltennamen können über Variablen eingebunden werden, Wrapper können die tatsächlichen API-Aufrufe verbergen und die Ausführung kann weit von der Konstruktion entfernt liegen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

ADO.NET gehört deshalb zu den möglichen heterogenen Evidenzquellen. Ob und in welchem Umfang es verbindlich implementiert wird, hängt vom Corpus Census und der Pilotstudie ab (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.2.10 Eingebettetes SQL

Eingebettetes SQL bezeichnet SQL-Code, der als String innerhalb von C#-Code enthalten ist.

```
var sql = "SELECT Id, Name FROM Patients WHERE IsActive = 1";
```

Statisch konstante SQL-Strings können vergleichsweise gut analysiert werden. Schwierigkeiten entstehen bei String-Konkatenation, Interpolation, `StringBuilder`, Hilfsmethoden, bedingter Query-Konstruktion, externen Parametern und dynamischen Tabellen- oder Spaltennamen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Cross-Repository Code Intelligence soll nicht auflösbare Teile als Unsicherheit dokumentieren. Ein teilweise rekonstruierter SQL-String darf nicht ohne Kennzeichnung wie ein vollständig bekannter SQL-Befehl behandelt werden (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.2.11 Stored Procedures

Eine Stored Procedure ist eine in der Datenbank definierte ausführbare Routine. Im C#-Code kann lediglich der Name der Prozedur oder ein entsprechender Aufruf sichtbar sein (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Für eine vollständige Analyse können mehrere Artefakte erforderlich sein: C#-Aufrufstelle, Command-Konfiguration, Name der Stored Procedure, SQL-Definition der Prozedur und die darin enthaltenen Tabellen- und Spaltenzugriffe (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Ist die Prozedurdefinition nicht verfügbar, kann Cross-Repository Code Intelligence möglicherweise nur feststellen: „Methode M ruft Stored Procedure P auf.“ Eine weitergehende Aussage wie „Methode M schreibt Tabelle T“ wäre ohne verfügbare und analysierte Prozedurdefinition nicht ausreichend gestützt. Stored Procedures verdeutlichen damit besonders klar den Unterschied zwischen lokaler Beobachtung und vollständigem Evidenzpfad (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.2.12 Mapping

Ein Mapping bezeichnet die Zuordnung zwischen Elementen des objektorientierten Programmmodells und Elementen der relationalen Datenbank. Wichtige Mappingtypen sind: Entity → Tabelle, Property → Spalte, Beziehung → Fremdschlüssel, DbSet → Entity und DbContext → Datenbank oder Datenbankkontext (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Mappings können unterschiedlich definiert sein: Attribute beziehungsweise Data Annotations, Fluent API, OnModelCreating, separate Entity-Konfigurationsklassen, Konventionen und projektspezifische Konfigurationen (vgl. Peng et al., 2026; Chen et al., 2025; Du et al., 2025).

Beispiel einer expliziten Tabellenzuordnung:

```
modelBuilder.Entity<Patient>()
    .ToTable("Patients");
```

Beispiel einer expliziten Spaltenzuordnung:

```
modelBuilder.Entity<Patient>()
    .Property(p => p.DateOfBirth)
    .HasColumnName("DateOfBirth");
```

Explizite Mappings liefern in der Regel stärkere und leichter inspizierbare Evidenz als konventionsbasierte Zuordnungen.

3.2.13 Explizite und konventionsbasierte Mappings

Ein explizites Mapping ist unmittelbar in einem Artefakt definiert, beispielsweise durch ein Attribut oder einen Fluent-API-Aufruf.

Ein konventionsbasiertes Mapping wird aus Frameworkkonventionen abgeleitet. Dabei kann beispielsweise der Klassenname zur Bestimmung des Tabellennamens oder der Propertyname zur Bestimmung des Spaltennamens verwendet werden.

Konventionsbasierte Ableitungen können methodisch zulässig sein, müssen jedoch sichtbar bleiben. Cross-Repository Code Intelligence darf sie nicht so darstellen, als wären sie direkt beobachtete Mappings. Daher wird unterschieden zwischen direkt beobachtetem Mapping, regelbasiert abgeleitetem Mapping und unaufgelöstem Mapping (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Designentscheidung: Jede konventionsbasierte Zuordnung erhält einen abgeleiteten Status und referenziert die angewendete Konventionsregel.

3.3 Graphbasierte Repräsentation

3.3.1 Motivation für eine Graphrepräsentation

Eine flache Liste einzelner Treffer reicht für repositoryweite Datenbankfragen nicht aus. Relevante Aussagen entstehen häufig aus Beziehungen zwischen mehreren Elementen: Methode, Invocation, aufgerufene Methode, DbContext, DbSet, Entity, Property, Tabelle, Spalte und SQL-Artefakt (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Eine graphbasierte Repräsentation ist geeignet, solche mehrstufigen Beziehungen explizit darzustellen. In einem Graphen werden Elemente als Knoten und Beziehungen als Kanten modelliert. Dadurch kann eine Aussage als Pfad rekonstruiert werden (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

Method

→ INVOKES

RepositoryMethod

→ ACCESSES

DbSet

→ REPRESENTS

Entity

→ MAPS_TO

Table

→ READ

Die tatsächlichen Relationstypen des endgültigen Evidenzmodells können davon abweichen. Das Beispiel zeigt jedoch die Grundidee: Eine Antwort basiert nicht auf einem einzelnen Treffer, sondern auf einem verbundenen Pfad.

3.3.2 Graph

Ein Graph besteht formal aus einer Menge von Knoten und einer Menge von Kanten. Knoten repräsentieren Entitäten oder Elemente. Kanten repräsentieren Beziehungen zwischen ihnen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Für Cross-Repository Code Intelligence können Knoten beispielsweise repräsentieren: Repository, Solution, Projekt, Datei, Namespace, Klasse, Methode, Property, Invocation, Entity, DbContext, DbSet, Datenbank, Schema, Tabelle, Spalte, Stored Procedure und Beobachtung (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Kanten können beispielsweise ausdrücken: CONTAINS, DECLARES, INVOKES, MAPS_TO, READS, WRITES, MUTATES, PERSISTS und SUPPORTED_BY. Die genaue Ontologie wird in einem späteren Kapitel als kanonisches Evidenzmodell definiert (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

3.3.3 Property Graph

Ein Property Graph ist ein Graphmodell, in dem Knoten und Kanten zusätzliche Eigenschaften besitzen können.

Ein Methodenknoten kann beispielsweise folgende Eigenschaften tragen: vollständiger Name, Signatur, Projekt, Datei sowie Start- und Endposition. Eine Evidenzkante kann Eigenschaften enthalten wie Detektor, Regel, Evidenzstatus, direkte oder abgeleitete Natur, Quellposition und Unsicherheitskategorie (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

Diese Eigenschaftsstruktur ist für Cross-Repository Code Intelligence besonders nützlich, weil die Arbeit nicht nur wissen muss, dass zwei Elemente verbunden sind, sondern auch, warum sie verbunden sind, woher die Beziehung stammt, durch welche Regel sie erzeugt wurde und wie sicher beziehungsweise vollständig sie ist (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

3.3.4 Code Property Graph

Der Code Property Graph, kurz CPG, verbindet unterschiedliche Programmdarstellungen in einer gemeinsamen Graphstruktur. In der ursprünglichen Konzeption werden insbesondere Informationen aus folgenden Strukturen kombiniert: Abstract Syntax Tree, Control Flow Graph und Program Dependence Graph (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Dadurch lassen sich komplexe Programmmuster als Graphtraversierungen formulieren. Code Property Graphs wurden insbesondere für großskalige Programmanalyse und Sicherheitsanalyse bekannt (Yamaguchi et al., 2014).

Cross-Repository Code Intelligence übernimmt nicht notwendigerweise ein vollständiges klassisches CPG. Der CPG dient vielmehr als konzeptioneller Bezugspunkt für folgende Idee: Unterschiedliche strukturelle und semantische Informationen können in einer gemeinsamen, abfragbaren Graphrepräsentation verbunden werden (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Cross-Repository Code Intelligence erweitert beziehungsweise spezialisiert diese Idee für datenbankbezogene Evidenz. Im Mittelpunkt stehen Datenbankoperationen, Mappingbeziehungen, Provenienz, Unsicherheit, Antwortbarkeit und begründete Verweigerung (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Abgrenzung: Cross-Repository Code Intelligence ist kein vollständiger allgemeiner Code Property Graph. Es ist ein evidenzzentrierter Graph für nachvollziehbare Datenbankoperationen (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

3.3.5 Code-Database Evidence Graph

Der Code-Database Evidence Graph ist die zentrale wissenschaftliche Repräsentation von Cross-Repository Code Intelligence. Er verbindet Artefakte, Programmelemente, Datenbankelemente, direkte Beobachtungen, abgeleitete Relationen, Provenienzinformationen und Unsicherheitszustände (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Der Code-Database Evidence Graph verfolgt nicht das Ziel, alle Eigenschaften des gesamten Programms abzubilden. Er enthält gezielt diejenigen Elemente und Beziehungen, die für die untersuchten Datenbankfragen erforderlich sind (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Seine Kernfunktionen sind: heterogene Evidenz in einem gemeinsamen Modell darstellen, direkte Beobachtungen und Ableitungen trennen, Quellprovenienz erhalten, mehrstufige Beziehungen sichtbar machen, strukturierte Retrievalpfade ermöglichen, fehlende oder unterbrochene Pfade identifizieren und Antwortbarkeit prüfen (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

3.3.6 Knoten

Ein Knoten repräsentiert ein identifizierbares Element des Korpus oder Evidenzmodells. Cross-Repository Code Intelligence unterscheidet konzeptionell mehrere Knotenklassen (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

- Artefaktknoten: Repository, Solution, Projekt, C#-Datei, SQL-Datei, Konfigurationsdatei.
- Programmknöten: Namespace, Klasse, Methode, Property, Symbol, Invocation.
- Datenbankknöten: Datenbank, Schema, Tabelle, Spalte, Stored Procedure.
- Evidenzknöten: Je nach endgültigem Modell können Beobachtungen und Ableitungen entweder als eigene Knoten oder als strukturierte Kanten mit Provenienzeigenschaften repräsentiert werden.

Die Entscheidung, Evidenz als Knoten, Kante oder separates Datenobjekt zu modellieren, ist eine Architekturentscheidung. Wissenschaftlich verbindlich ist lediglich, dass Evidenz, Provenienz und Ableitungsstatus explizit erhalten bleiben (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

3.3.7 Kanten und Relationstypen

Eine Kante beschreibt eine Beziehung zwischen zwei Knoten.

Strukturelle Relationen können beispielsweise ausdrücken: Datei enthält Klasse, Klasse deklariert Methode, Methode enthält Invocation, Projekt enthält Datei.

Semantische Relationen können ausdrücken: Invocation referenziert Methode, Methode ruft Methode auf, DbSet repräsentiert Entity, Property gehört zu Entity (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Datenbankbezogene Relationen können ausdrücken: Entity ist auf Tabelle gemappt, Property ist auf Spalte gemappt, Methode liest Tabelle, Methode schreibt Spalte.

Evidenzrelationen können ausdrücken: Relation wird durch Beobachtung unterstützt, Antwort wird durch Evidenzpfad getragen, Ableitung verwendet Regel (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Die technischen Relationstypen sollen konsistent in Großbuchstaben geschrieben werden, beispielsweise READ, WRITE, INVOKES, MAPS_TO, MUTATES, PERSISTS und SUPPORTED_BY.

3.3.8 Graphtraversierung

Eine Graphtraversierung ist das systematische Durchlaufen von Knoten und Kanten nach definierten Regeln.

Für Cross-Repository Code Intelligence kann eine Traversierung beispielsweise bei einer Methode beginnen und prüfen: Welche Invocations enthält die Methode? Welche Symbole werden aufgerufen? Welche DbSet oder Queries sind beteiligt? Welche Entity-Typen werden verwendet? Welche Tabellen- oder Spaltenmappings bestehen? Welche READ- oder WRITE-Beobachtungen tragen den Pfad? Sind alle Kanten gültig und durch Evidenz gestützt? (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Eine andere Traversierung kann bei einer Datenbankspalte beginnen und rückwärts nach allen Methoden suchen, die einen gültigen READ-Pfad zu dieser Spalte besitzen. Graphtraversierungen sind damit die Grundlage für das graphbasierte Retrieval (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

3.3.9 Mehrstufige Beziehungen

Eine mehrstufige Beziehung ist eine Beziehung, die nicht durch eine einzelne Beobachtung, sondern durch eine Kette von Zwischenschritten entsteht.

ControllerMethod → INVOKES

ServiceMethod → INVOKES

RepositoryMethod → USES

DbSet<Patient> → MAPS_TO

Patients → READ

Die Schlussfolgerung „Die Controller-Methode kann über diesen Pfad zu einem READ auf Patients führen“ ist nur dann unterstützt, wenn jede relevante Kante des Pfads gültig ist (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

Mehrstufige Beziehungen erhöhen die Ausdruckskraft des Systems, erhöhen aber auch das Fehlerrisiko. Eine einzige ungültige oder fehlende Kante kann die gesamte Schlussfolgerung schwächen. Cross-Repository Code Intelligence bewertet deshalb nicht nur Endpunkte, sondern auch die Pfadgültigkeit (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

3.3.10 Speicherunabhängigkeit

Der wissenschaftliche Code-Database Evidence Graph ist von einer konkreten Speichertechnologie zu unterscheiden. Mögliche technische Umsetzungen sind: In-Memory-Graph, Neo4j, relationale Speicherung, JSON oder JSONL, SQLite oder eine Kombination mehrerer Speicherformen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Die Wahl einer Graphdatenbank ist keine Voraussetzung dafür, dass das wissenschaftliche Modell graphbasiert ist.

Designentscheidung: Das kanonische Evidenzmodell bleibt speicherunabhängig. Die Speichertechnologie ist eine austauschbare Implementierungsentscheidung (vgl. Ullrich et al., 2025).

Dies erhöht die Reproduzierbarkeit und verhindert, dass der wissenschaftliche Beitrag mit einem einzelnen Softwareprodukt verwechselt wird.

3.4 Evidenz und Provenienz

3.4.1 Begriff der Evidenz

Der Begriff Evidenz wird in Cross-Repository Code Intelligence enger verwendet als ein allgemeiner Hinweis oder eine plausible Vermutung. Evidenz bezeichnet strukturierte, typisierte und überprüfbare Information, die eine spätere Aussage über eine Datenbankoperation unterstützen kann (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Eine Evidenzinformation sollte mindestens beantworten können: Was wurde erkannt? Wo wurde es erkannt? Welches Programmelement ist beteiligt? Welches Datenbankelement ist möglicherweise beteiligt? Welcher Detektor hat die Information erzeugt? Welche Regel wurde angewendet? Ist die Information direkt beobachtet oder abgeleitet? Welche Unsicherheit besteht? (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Nicht jeder Treffer ist automatisch Evidenz gleicher Stärke. Ein Textfund besitzt einen anderen Status als ein semantisch aufgelöster Aufruf mit explizitem Mapping.

3.4.2 Beobachtung

Eine Beobachtung ist ein direkt aus einem konkreten Artefakt extrahierter, typisierter Befund. Beispiele: Eine Klasse leitet von DbContext ab. Eine Property besitzt den Typ DbSet<Patient>. Eine Methode ruft SaveChangesAsync auf. Ein Fluent-API-Ausdruck ordnet Patient der Tabelle Patients zu. Ein SQL-Statement enthält ein SELECT auf Patients. Eine Methode greift in einer Query auf die Property DateOfBirth zu (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Eine Beobachtung ist lokal, quellgebunden, detektorbasiert, regelbasiert und überprüfbar. Eine Beobachtung ist noch keine vollständige Antwort. Beispielsweise ist der erkannte Property-Zugriff auf DateOfBirth noch kein Beweis dafür, dass die physische Spalte Patients.DateOfBirth gelesen wird. Dazu können zusätzliche Mapping- und Queryevidenz erforderlich sein.

3.4.3 Direkte Evidenz

Direkte Evidenz stammt unmittelbar aus einer konkreten Quellstelle. Beispiele: expliziter Aufruf von SaveChanges, explizites ToTable("Patients"), explizites HasColumnName("DateOfBirth"), SQL-Statement SELECT DateOfBirth FROM Patients sowie ein semantisch aufgelöster Aufruf auf einem DbSet (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Direkte Evidenz bedeutet nicht automatisch, dass die daraus gezogene Gesamtaussage direkt ist. Eine vollständige Antwort kann mehrere direkte Beobachtungen kombinieren.

3.4.4 Abgeleitete Relation

Eine abgeleitete Relation entsteht, wenn eine oder mehrere Beobachtungen durch eine dokumentierte Regel verbunden werden.

Beispiel:

- Beobachtung: Methode GetPatients materialisiert eine Query auf DbSet<Patient>.
- Beobachtung: DbSet<Patient> repräsentiert die Entity Patient.
- Beobachtung: Patient ist auf Tabelle Patients gemappt.
- Ableitungsregel: Eine materialisierte Entity-Query auf einem gemappten DbSet trägt eine READ-Relation zur Zieltabelle.

Daraus entsteht die abgeleitete Relation: GetPatients READS Patients.

Die Relation muss auf ihre tragenden Beobachtungen zurückgeführt werden können. Eine abgeleitete Relation enthält daher mindestens: Ausgangselement, Zielelement, Relationstyp, angewendete Ableitungsregel, unterstützende Beobachtungen, Evidenzstatus und offene Einschränkungen (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

3.4.5 Provenienz

Provenienz bezeichnet den nachvollziehbaren Herkunftsnachweis einer Beobachtung, Relation oder Antwort. Zur Provenienz können gehören: Repository und Snapshot, Projekt, Datei, Zeilen- und Spaltenposition, Syntaxknoten, Symbol, Detektor, Regel, Konfiguration, Toolversion und Ableitungsschritte (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

Provenienz ist kein optionales Zusatzfeld. Sie ist ein Kernbestandteil des wissenschaftlichen Evidenzbegriffs. Ohne Provenienz wäre nicht überprüfbar, ob die Quelle tatsächlich existiert, ob der Detektor korrekt gearbeitet hat, welche Regel die Relation erzeugt hat, ob eine Antwort reproduziert werden kann und wo ein Fehler entstanden ist.

Cross-Repository Code Intelligence soll daher für jede unterstützte Aussage einen inspizierbaren Rückweg zu den tragenden Quellstellen bereitstellen (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

3.4.6 Provenienzerhaltung

Provenienzerhaltung bedeutet, dass die Herkunft einer Information bei Normalisierung, Verknüpfung, Graphkonstruktion und Retrieval nicht verloren geht (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Wenn eine lokale Beobachtung in eine normalisierte Struktur überführt wird, müssen ihre Quellinformationen erhalten bleiben. Wenn mehrere Beobachtungen eine Relation tragen, muss die Relation auf diese Beobachtungen verweisen. Wenn eine Antwort mehrere Relationen verwendet, muss sichtbar bleiben, welche Pfade die einzelnen Aussagen unterstützen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Eine problematische Verdichtung wäre beispielsweise Method A WRITES Table T ohne Informationen darüber, welche Quellstellen verwendet wurden, ob ein Mutationssignal vorhanden war, ob SaveChanges erkannt wurde, welches Mapping verwendet wurde oder ob Teile des Pfads nur konventionsbasiert waren. Eine provenienzerhaltende Darstellung würde dagegen den vollständigen Ableitungsweg dokumentieren (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.4.7 Evidenzpfad

Ein Evidenzpfad ist eine geordnete Folge von Beobachtungen, Elementen und Relationen, die eine Schlussfolgerung trägt.

Method: UpdatePatient

- mutiert Entity: Patient
- Entity gehört zu DbContext
- SaveChanges wird aufgerufen
- Patient MAPS_TO Patients
- Ergebnis: statisch unterstützter WRITE auf Patients

Ein Evidenzpfad muss bei einem überprüfbaren Programmelement beginnen, über gültige Relationen verlaufen, das relevante Datenbankziel erreichen, auf konkrete Quellbeobachtungen zurückgeführt werden können und seinen Unsicherheitsstatus offenlegen. Evidenzpfade bilden die zentrale Einheit des graphbasierten Retrievals (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

3.4.8 Evidenzlücke

Eine Evidenzlücke liegt vor, wenn ein für die Schlussfolgerung notwendiger Schritt fehlt oder nicht ausreichend belegt ist. Typische Evidenzlücken sind: Entity-zu-Tabelle-Mapping fehlt, Property-zu-Spalte-Mapping ist unbekannt, Symbol kann nicht aufgelöst werden, Hilfsmethode wird nicht interprozedural verfolgt, SQL-String ist nur teilweise rekonstruierbar, Stored-Procedure-Definition fehlt, Persistenztrigger ist nicht zuordenbar oder Projekt- beziehungsweise Datenbankgrenze ist nicht auflösbar (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Evidenzlücken werden nicht stillschweigend geschlossen. Sie müssen im Ergebnis sichtbar sein und können zu einer teilweisen Unterstützung oder Verweigerung führen (vgl. Liu et al., 2026; Abedu et al., 2025).

3.4.9 Evidenzstatus

Cross-Repository Code Intelligence verwendet kategorische Evidenzzustände. Die verbindlichen Grundzustände lauten SUPPORTED, PARTIALLY_SUPPORTED und NOT_ANSWERABLE (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Für interne Evidenzelemente können zusätzliche Statuswerte erforderlich sein, beispielsweise DIRECT, DERIVED, UNRESOLVED, CONVENTION_BASED und CONFLICTING. Die genaue technische Ausgestaltung wird im Evidenzmodell festgelegt.

Numerische Konfidenzwerte werden nicht ohne gesonderte Kalibrierung verwendet. Ein Wert wie „85 % sicher“ wäre wissenschaftlich nicht begründet, solange keine geeignete Kalibrierungsstudie mit gelabelten Daten durchgeführt wurde.

Merksatz: Eine erklärbare Kategorie ist besser als eine scheinpräzise Prozentzahl ohne empirische Grundlage.

3.4.10 Evidenz und Wahrheit

Cross-Repository Code Intelligence modelliert Evidenz innerhalb eines festgelegten Korpus und Analyseumfangs. Eine unterstützte Aussage bedeutet daher: Die Aussage wird durch die im Frozen Snapshot vorhandenen und vom System analysierten Artefakte nach den definierten Regeln getragen (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Sie bedeutet nicht, dass die Aussage für alle zukünftigen Versionen des Systems gilt, dass der Code zur Laufzeit tatsächlich ausgeführt wird, dass keine externen Datenbankzugriffe existieren, dass das System vollständig fehlerfrei ist oder dass eine rechtliche oder fachliche Bewertung abgeschlossen ist. Diese epistemische Begrenzung muss in der Interpretation der Ergebnisse erhalten bleiben (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.5 Evaluationsgrundlagen

3.5.1 Korpus

Ein Korpus ist die festgelegte Sammlung von Artefakten, auf der Entwicklung und Evaluation durchgeführt werden. Der Korpus kann C#-Dateien, Projektdateien, Solution-Dateien, Konfigurationen, SQL-Dateien, Stored-Procedure-Definitionen und weitere datenbankbezogene Artefakte enthalten.

Alle Aussagen über Extraktionsleistung, Evidenzvollständigkeit oder Antwortqualität gelten relativ zu diesem Korpus. Der Korpus ist damit nicht nur eine technische Eingabe. Er definiert den empirischen Untersuchungsraum der Arbeit.

3.5.2 Corpus Census

Der Corpus Census ist eine systematische Bestandsaufnahme des Korpus. Er dokumentiert beispielsweise die Anzahl der Projekte, die Anzahl der C#-Dateien, verwendete Frameworkversionen, vorhandene DbContexts, DbSet-Strukturen, Mappingformen, SQL-Artefakte, Stored Procedures, ADO.NET-Nutzung, generierten Code, ausgeschlossene Ordner sowie Test- und Drittanbietercode (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Der Corpus Census erfüllt mehrere Funktionen: Er beschreibt den Untersuchungsgegenstand, unterstützt die Scope-Entscheidung, hilft bei der Auswahl des Goldstandard-Subsets, macht sichtbar, welche Evidenzarten tatsächlich vorkommen, und verhindert, dass optionale Features ohne Korpusgrundlage entwickelt werden (vgl. Peng et al., 2026; Chen et al., 2025; Du et al., 2025).

Didaktische Erklärung - einfaches Deutsch

Der Corpus Census ist eine vollständige Inventur des Projekts. Bevor Cross-Repository Code Intelligence gebaut wird, muss klar sein, welche Dateien, Technologien und Zugriffsmuster im Korpus wirklich vorhanden sind.

3.5.3 Frozen Snapshot

Ein Frozen Snapshot ist eine unveränderlich referenzierte Version des Korpus. Er kann beispielsweise durch folgende Informationen festgelegt werden: Repository-URL, Commit-Hash, Abrufdatum, Archivdatei, Prüfsumme und dokumentierte Ausschlüsse (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Ein eingefrorener Snapshot ist notwendig, weil sich ein aktives Repository verändern kann. Würden unterschiedliche Experimente auf unterschiedlichen Versionen laufen, wären die Ergebnisse nicht fair vergleichbar. Jede Änderung des Korpus erzeugt deshalb eine neue Korpusversion. Änderungen dürfen nicht stillschweigend in die finale Evaluation eingehen.

3.5.4 Goldstandard

Ein Goldstandard ist eine manuell geprüfte Referenzmenge, gegen die Systemausgaben verglichen werden. Ein Gold-Item kann beispielsweise enthalten: Subjektmethode, Operationstyp, Zieltabellen- oder Zielspaltelement, Zugriffsmechanismus, erforderlicher Evidenzpfad, genaue Quellorte, direkter oder abgeleiteter Status, Antwortbarkeitsstatus und dokumentierte ungelöste Stellen (vgl. Peng et al., 2026; Chen et al., 2025; Du et al., 2025).

Der Goldstandard ist erforderlich, weil eine plausibel wirkende Ausgabe nicht automatisch korrekt ist. Ohne Referenzmenge könnte nicht zuverlässig bestimmt werden, ob ein Treffer richtig ist, ob relevante Evidenz fehlt, ob eine Relation falsch abgeleitet wurde oder ob eine Verweigerung berechtigt ist. Der Goldstandard muss versioniert, dokumentiert und nach klaren Annotationsregeln erstellt werden (vgl. Peng et al., 2026; Chen et al., 2025; Du et al., 2025).

3.5.5 Positives Beispiel

Ein positives Beispiel ist ein Fall, in dem die gesuchte Relation laut Goldstandard tatsächlich besteht. Beispiel: Methode M liest Tabelle T über einen vollständig belegten Entity-Framework-Pfad. Positive Beispiele sind notwendig, um Recall und korrekte Erkennung zu bewerten (vgl. Peng et al., 2026; Chen et al., 2025; Du et al., 2025).

3.5.6 Negatives Beispiel

Ein negatives Beispiel ist ein Fall, der einem Datenbankzugriff ähnlich sieht, aber die gesuchte Relation nicht erfüllt. Beispiele: Tabellename nur in einem Kommentar, gleichnamige DTO-Property ohne Mapping, In-Memory-LINQ-Abfrage, Objektänderung ohne Persistenzpfad, Testdaten ohne produktive Datenbankoperation, [NotMapped]-Property. Negative Beispiele sind besonders wichtig für die Bewertung von False Positives (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.5.7 Hard Negative

Ein Hard Negative ist ein negatives Beispiel, das einem positiven Beispiel stark ähnelt und deshalb für einfache Detektoren besonders schwierig ist.

```
var patients = inMemoryList.Where(p => p.IsActive).ToList();
```

Dieser Code sieht einer Entity-Framework-Abfrage ähnlich, arbeitet jedoch möglicherweise nur auf einer In-Memory-Liste. Hard Negatives helfen zu prüfen, ob semantische Analyse tatsächlich einen Vorteil gegenüber Text- und Syntaxbaselines bietet (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

3.5.8 True Positive, False Positive und False Negative

Ein True Positive (TP) liegt vor, wenn das System ein Evidenzelement erkennt, das im Goldstandard als relevant definiert ist. Ein False Positive (FP) liegt vor, wenn das System ein Evidenzelement ausgibt, das laut Goldstandard nicht relevant oder falsch ist. Ein False Negative (FN) liegt vor, wenn ein relevantes Gold-Evidenzelement vom System nicht erkannt wird (vgl. Peng et al., 2026; Chen et al., 2025; Du et al., 2025).

Optional kann ein True Negative definiert werden, wenn eine klar abgegrenzte Menge negativer Kandidaten vorliegt. Bei offenen Extraktionsaufgaben ist die Menge aller möglichen Nichttreffer jedoch häufig nicht eindeutig definierbar. Deshalb stehen für RQ1 primär TP, FP und FN im Vordergrund.

3.5.9 Precision

Precision misst, welcher Anteil der vom System ausgegebenen Evidenzelemente korrekt ist.

$$\text{Precision} = (\text{TP}) / (\text{TP} + \text{FP})$$

Eine hohe Precision bedeutet, dass das System nur wenige falsche Evidenzelemente ausgibt. Beispiel: Das System gibt zehn READ-Relationen aus. Acht davon sind laut Goldstandard korrekt. Damit beträgt $\text{Precision} = (8) / (10) = 0,8$, also 80 Prozent (vgl. Peng et al., 2026; Chen et al., 2025; Du et al., 2025).

3.5.10 Recall

Recall misst, welcher Anteil der relevanten Gold-Evidenz vom System gefunden wurde.

$$\text{Recall} = (\text{TP}) / (\text{TP} + \text{FN})$$

Ein hoher Recall bedeutet, dass das System nur wenige relevante Evidenzelemente übersieht. Beispiel: Der Goldstandard enthält zwölf READ-Relationen. Das System findet acht davon korrekt. Damit beträgt $\text{Recall} = (8) / (12) \approx 0,67$, also ungefähr 67 Prozent (vgl. Peng et al., 2026; Chen et al., 2025; Du et al., 2025).

3.5.11 F1-Score

Der F1-Score ist das harmonische Mittel aus Precision und Recall.

$$\text{F1} = 2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall})$$

Der F1-Score ist nützlich, wenn Precision und Recall gemeinsam betrachtet werden sollen. Er ersetzt jedoch nicht die getrennte Berichterstattung. Ein einzelner F1-Wert kann Unterschiede zwischen Evidenzkategorien verdecken. Deshalb werden Precision, Recall, F1 und Rohzahlen getrennt nach Kategorien und Konfigurationen berichtet (vgl. Peng et al., 2026; Chen et al., 2025; Du et al., 2025).

3.5.12 Auswertungseinheit

Die Auswertungseinheit legt fest, was genau gezählt und verglichen wird. Mögliche Einheiten sind: einzelne Beobachtung, abgeleitete Relation, Methode-Operation-Ziel-Tupel, Evidenzpfad, Frage, Antwort und einzelne Antwortaussage (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Ohne klar definierte Einheit wären Metriken mehrdeutig. Beispielsweise kann ein System dieselbe Tabellenrelation aus mehreren Quellstellen erkennen. Es muss vorab festgelegt werden, ob jede Quellstelle als separates Evidenzelement oder die normalisierte Relation nur einmal gezählt wird (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.5.13 Mikro- und Makroaggregation

Bei einer Mikroaggregation werden alle Einzelentscheidungen gemeinsam gezählt, bevor eine Metrik berechnet wird. Häufige Kategorien beeinflussen das Ergebnis dadurch stärker. Bei einer Makroaggregation wird die Metrik zunächst pro Kategorie berechnet und anschließend über die Kategorien gemittelt. Seltene Kategorien erhalten dadurch ein vergleichbares Gewicht.

Cross-Repository Code Intelligence soll Rohzahlen und Kategorieergebnisse berichten. Die endgültige Aggregationsregel wird vor dem finalen Experiment festgelegt (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

3.5.14 Baseline

Eine Baseline ist ein einfacher Vergleichsansatz, gegen den die vorgeschlagene Methode bewertet wird. Mögliche Baselines sind: Textsuche, namensbasierte Suche, reine Syntaxanalyse, symbolbewusstes Retrieval ohne Graph und flache Suche über Evidenzdatensätze (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Eine Baseline zeigt, ob die zusätzliche Komplexität der vorgeschlagenen Methode tatsächlich einen messbaren Vorteil bietet. Ohne Baseline könnte nur gezeigt werden, dass Cross-Repository Code Intelligence funktioniert, nicht aber, ob Graphmodell, semantische Analyse oder Antwortbarkeitslogik einen zusätzlichen Nutzen liefern (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

3.5.15 Konfiguration

Eine Konfiguration ist eine klar definierte Kombination von Analysebausteinen, Regeln und Parametern.

Eine Extraktionskonfiguration kann beispielsweise festlegen: Textanalyse aktiv, Syntaxanalyse aktiv, Semantik aktiv, EF-Mappings aktiv, SQL-Analyse aktiv. Eine Retrievalkonfiguration kann festlegen: Textretrieval, flaches Evidenzretrieval, Graphtraversierung, Antwortbarkeitsprüfung. Konfigurationen müssen versioniert und vor der finalen Messung eingefroren werden (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

3.5.16 Reproduzierbarkeit

Reproduzierbarkeit bedeutet, dass die Untersuchung mit dokumentierten Artefakten, Regeln, Versionen und Konfigurationen nachvollzogen und erneut ausgeführt werden kann. Dazu gehören: Frozen Snapshot, Analyzer-Version, Toolversionen, Konfigurationsdateien, Goldstandard, Fragenmenge, Metrikdefinitionen, Rohoutputs, Auswertungsskripte und dokumentierte Laufparameter. Reproduzierbarkeit ist ein zentraler Bestandteil des wissenschaftlichen Beitrags von Cross-Repository Code Intelligence (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

3.6 Antwortbarkeit und kontrolliertes Antwortverhalten

3.6.1 Begriff der Antwortbarkeit

Antwortbarkeit bezeichnet die Feststellung, ob die für eine konkrete Frage erforderliche Evidenz innerhalb des definierten Analyseumfangs vollständig und gültig vorhanden ist. Antwortbarkeit ist nicht dasselbe wie thematische Relevanz. Ein System kann relevante Dateien oder Begriffe finden und trotzdem nicht genügend Evidenz für eine verantwortbare Antwort besitzen (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Beispiel: Auf die Frage „Welche Methoden schreiben die Spalte Patients.DateOfBirth?“ könnte gefundene Evidenz zeigen, dass eine Methode Patient.DateOfBirth verändert, dass ein DbContext vorhanden ist und dass ein Mapping von Patient auf Patients bekannt ist. Fehlende Evidenz wären dagegen ein belegter Persistenztrigger und eine verifizierte Property-zu-Spalte-Zuordnung. Das System besitzt relevante Informationen, aber möglicherweise keine vollständige Grundlage für eine spaltenbezogene WRITE-Antwort (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.6.2 Fragevertrag

Ein Fragevertrag definiert vorab, welche Evidenz für einen bestimmten Fragetyp erforderlich ist. Ein Fragevertrag kann enthalten: Zieltyp der Frage, erlaubter Analyseumfang, notwendige Relationstypen, minimale Provenienz, akzeptierte Unsicherheitszustände, Regeln für Teilantworten und Regeln für Verweigerung (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Beispiel für die Frage „Welche Methoden schreiben Tabelle T?“ - mögliche Muss-Bedingungen: identifizierte Methode, Mutationssignal, Entity- oder SQL-Ziel, Mapping zu Tabelle T, Persistenztrigger (sofern frameworkbedingt erforderlich) und gültige Provenienz für alle wesentlichen Schritte. Frageverträge verhindern, dass Antwortqualität erst nach Betrachtung der Resultate beliebig definiert wird (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

3.6.3 SUPPORTED

Der Status SUPPORTED bedeutet, dass die erforderliche Evidenz vollständig vorhanden ist. Eine unterstützte Antwort muss alle Muss-Bedingungen des Fragevertrags erfüllen, auf gültigen Relationen beruhen, eine nachvollziehbare Provenienz besitzen, innerhalb des definierten Scopes liegen und keine kritische ungelöste Lücke enthalten (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

SUPPORTED bedeutet nicht, dass die Aussage außerhalb des statischen Analyseumfangs uneingeschränkt wahr ist. Der Status ist immer relativ zu Korpus, Regeln und Scope.

3.6.4 PARTIALLY_SUPPORTED

Der Status PARTIALLY_SUPPORTED bedeutet, dass ein Teil der erforderlichen Evidenz vorhanden ist, aber mindestens ein wesentlicher Schritt fehlt. Beispiele: Tabelle ist bekannt, Spaltenmapping fehlt; Mutationssignal ist vorhanden, Persistenztrigger ist ungelöst; Stored-Procedure-Aufruf ist bekannt, Definition fehlt; Aufrufkette ist teilweise rekonstruiert; Entity-Mapping ist nur konventionsbasiert und für die konkrete Frage nicht ausreichend (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Eine teilweise unterstützte Antwort darf nur ausgegeben werden, wenn der Fragevertrag Teilantworten erlaubt. Die fehlende Evidenz muss ausdrücklich genannt werden (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

3.6.5 NOT_ANSWERABLE

Der Status NOT_ANSWERABLE bedeutet, dass die verfügbare Evidenz nicht ausreicht, um die verlangte Schlussfolgerung zu tragen. Dieser Status kann auftreten, obwohl einzelne relevante Treffer vorhanden sind. Beispiele: dynamisch zusammengesetzter Tabellename, nicht verfügbare Stored Procedure, unaufgelöste Reflection, fehlendes Mapping, unbekannte externe Bibliothek, nicht analysierter Projektteil (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

NOT_ANSWERABLE ist eine Aussage über die Evidenzlage im definierten Scope, nicht über die prinzipielle Existenz einer Antwort.

3.6.6 Begründete Verweigerung

Eine begründete Verweigerung liegt vor, wenn das System eine nicht ausreichend belegte Frage bewusst nicht beantwortet und die konkrete Evidenzlücke dokumentiert (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Beispiel: Eine spaltenbezogene Antwort ist nicht möglich, weil für die Property DateOfBirth im untersuchten Snapshot kein ausreichend verifiziertes Spaltenmapping rekonstruiert werden konnte. Eine tabellenbezogene READ-Relation ist jedoch unterstützt.

Eine Verweigerung ist nur dann wissenschaftlich nützlich, wenn sie begründet und nachvollziehbar ist. Ein bloßes „Keine Antwort“ reicht nicht aus. Die Begründung soll möglichst enthalten, welche Evidenz vorhanden ist, welche Evidenz fehlt, warum die Lücke relevant ist und welche eingeschränkte Aussage gegebenenfalls möglich bleibt (vgl. Liu et al., 2026; Abedu et al., 2025).

3.6.7 Korrekte Verweigerung

Eine korrekte Verweigerung liegt vor, wenn die Frage laut Goldstandard beziehungsweise Fragevertrag nicht beantwortbar ist und das System die Antwort berechtigt verweigert. Die Rate korrekter Verweigerungen ist eine zentrale Metrik für RQ2. Sie misst, ob die Antwortbarkeitslogik verhindert, dass unzureichend belegte Aussagen ausgegeben werden (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

3.6.8 Falsche Verweigerung

Eine falsche Verweigerung liegt vor, wenn die erforderliche Evidenz laut Goldstandard vorhanden ist, das System aber trotzdem keine Antwort ausgibt. Eine zu vorsichtige Antwortlogik kann daher ebenfalls problematisch sein. Das Ziel ist nicht maximale Verweigerung, sondern angemessenes Antwortverhalten (vgl. Peng et al., 2026; Chen et al., 2025; Du et al., 2025).

3.6.9 Nicht gestützte Antwort

Eine nicht gestützte Antwort liegt vor, wenn das System eine Schlussfolgerung ausgibt, obwohl die notwendige Evidenz unvollständig oder ungültig ist. Beispiele: Spaltenzugriff wird ohne Spaltenmapping behauptet; WRITE wird ohne Persistenzpfad behauptet; Stored-Procedure-Aufruf wird ohne Definition zu konkreten Tabellenzugriffen erweitert; Namensähnlichkeit wird als semantische Beziehung behandelt (vgl. Liu et al., 2026; Abedu et al., 2025).

Nicht gestützte Antworten sind besonders kritisch, weil sie plausibel und präzise erscheinen können, obwohl ihre Evidenzbasis unzureichend ist (vgl. Liu et al., 2026; Abedu et al., 2025).

3.6.10 Antwortkorrektheit

Antwortkorrektheit bewertet, ob die ausgegebene Schlussfolgerung mit der Goldantwort übereinstimmt. Eine Antwort kann sprachlich überzeugend sein und trotzdem inhaltlich falsch sein. Deshalb wird nicht die Formulierung, sondern die strukturierte Schlussfolgerung bewertet. Mögliche Bewertungseinheiten sind: aufgelistete Methoden, Operationstyp, Zieltabellen oder Zielspalten, Antwortstatus und enthaltene Einschränkungen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.6.11 Evidenzvollständigkeit

Evidenzvollständigkeit misst, welcher Anteil der für eine Antwort erforderlichen Gold-Evidenz tatsächlich abgerufen wurde. Eine inhaltlich korrekte Antwort kann trotzdem eine unvollständige Evidenzbasis besitzen. Beispiel: Das System nennt korrekt eine lesende Methode, zeigt jedoch nur die Methodenzeile und lässt das relevante Entity-zu-Tabelle-Mapping aus. Die Antwort kann korrekt sein, aber ihre Evidenzvollständigkeit ist reduziert.

3.6.12 Evidenzpräzision

Evidenzpräzision misst, welcher Anteil der zurückgegebenen Evidenz tatsächlich relevant und korrekt ist. Ein System kann alle notwendigen Belege finden, aber zusätzlich viele irrelevante Dateien und Pfade zurückgeben. Dies erschwert die Nachvollziehbarkeit und kann Nutzerinnen und Nutzer verwirren.

3.6.13 Nachvollziehbarkeit

Nachvollziehbarkeit bezeichnet die Fähigkeit, eine Aussage auf konkrete Quellen und Ableitungsschritte zurückzuführen. Eine nachvollziehbare Antwort zeigt beispielsweise die beteiligte Methode, die relevante Quellposition, die erkannte Operation, das Mapping, das Zieldatenbankelement, die verwendete Regel und den offenen Unsicherheitsstatus. Nachvollziehbarkeit ist mehr als die Ausgabe eines Dateinamens. Sie erfordert einen strukturierten, inspizierbaren Zusammenhang zwischen Aussage und Evidenz.

3.6.14 Pfadgültigkeit

Pfadgültigkeit prüft, ob jede wesentliche Kante eines Evidenzpfads durch eine direkte Beobachtung oder eine dokumentierte Ableitungsregel gestützt ist. Ein Pfad ist nicht vollständig gültig, wenn beispielsweise ein Methodenaufruf nur aufgrund gleicher Namen angenommen wird, ein Mapping ohne Regel ergänzt wird, ein Persistenztrigger fehlt oder ein dynamischer SQL-Teil als statisch bekannt behandelt wird. Die Pfadgültigkeit ist daher eine zentrale Metrik des graphbasierten Ansatzes (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

3.6.15 Coverage

Coverage bezeichnet den Anteil der Fragen oder Fälle, für die das System ein bestimmtes Verhalten liefern kann. Je nach Definition kann Coverage messen: Anteil der Fragen mit unterstützter Antwort, Anteil der Fragen mit Antwort oder begründeter Verweigerung, Anteil der Evidenzkategorien, die technisch unterstützt werden, oder Anteil des Korpus, der analysiert wurde. Die konkrete Definition muss für jede Verwendung angegeben werden. Der Begriff darf nicht ohne Zähler und Nenner verwendet werden (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

3.7 Verbindliche konzeptionelle Unterscheidungen

Die folgenden Unterscheidungen sind für die gesamte Arbeit verbindlich.

- 3.7.1 Texttreffer ist nicht gleich Symbol. Ein Textname kann in verschiedenen Kontexten auftreten. Erst die Symbolauflösung bestimmt, welches Programmelement tatsächlich gemeint ist.
- 3.7.2 Syntax ist nicht gleich Semantik. Ein Syntaxknoten zeigt die Form eines Ausdrucks. Die semantische Analyse bestimmt Typ, Symbol und konkrete Bedeutung.
- 3.7.3 Beobachtung ist nicht gleich Relation. Eine lokale Beobachtung kann eine Relation unterstützen, ist aber nicht automatisch selbst eine vollständige fachliche Beziehung.

- 3.7.4 Relation ist nicht gleich Antwort. Eine einzelne READ- oder WRITE-Relation kann Teil einer Antwort sein. Eine repositoryweite Frage kann jedoch mehrere Relationen, Vollständigkeitsbedingungen und Evidenzpfade benötigen.
- 3.7.5 Mutation ist nicht gleich Persistierung. Eine Objektänderung oder ein Add-Aufruf ist nicht automatisch ein ausgeführter Datenbank-WRITE. Der Persistenzpfad muss entsprechend dem Frameworkmodell berücksichtigt werden.
- 3.7.6 Query-Konstruktion ist nicht gleich Query-Ausführung. Ein IQueryable kann erstellt werden, ohne an derselben Stelle ausgeführt zu werden. Konstruktion und Materialisierung werden getrennt behandelt.
- 3.7.7 Entity ist nicht gleich Tabelle. Eine Entity-Klasse repräsentiert ein Programmkonzept. Die Zuordnung zu einer physischen Tabelle muss durch Mapping oder eine dokumentierte Konventionsregel hergestellt werden.
- 3.7.8 Property ist nicht gleich Spalte. Eine C#-Property kann gemappt, nicht gemappt, umbenannt, berechnet oder rein transient sein. Eine Spaltenaussage erfordert ausreichende Mappingevidenz.
- 3.7.9 Relevanz ist nicht gleich Beweis. Ein relevantes Textfragment kann bei der Suche helfen. Eine verantwortbare Antwort benötigt jedoch gültige Evidenz.
- 3.7.10 Unsicherheit ist nicht gleich Fehler. Unsicherheit kann eine reale Eigenschaft des statisch untersuchten Problems sein. Ein System handelt korrekt, wenn es diese Unsicherheit sichtbar macht.
- 3.7.11 Verweigerung ist nicht automatisch Versagen. Eine begründete Verweigerung ist korrekt, wenn die notwendige Evidenz fehlt. Eine unbegründete oder falsche Verweigerung bleibt dagegen ein Fehler.
- 3.7.12 Statisch unterstützt ist nicht gleich zur Laufzeit ausgeführt. Cross-Repository Code Intelligence beschreibt statisch belegbare Beziehungen. Konkrete Laufzeitausführung liegt außerhalb des verbindlichen Nachweises.

3.8 Konsolidierte Terminologie

Die folgenden Begriffe werden in der gesamten Arbeit bevorzugt verwendet.

Englischer Begriff	Verbindliche deutsche Bezeichnung
Evidence	Evidenz
Observation	Beobachtung
Derived Relation	abgeleitete Relation
Provenance	Provenienz
Evidence Path	Evidenzpfad
Answerability	Antwortbarkeit
Traceability	Nachvollziehbarkeit
Correct Refusal	begründete beziehungsweise korrekte Verweigerung
Unsupported Answer	nicht gestützte Antwort
Corpus	Korpus
Corpus Census	Corpus Census / Korpusbestandsaufnahme
Frozen Snapshot	Frozen Snapshot / eingefrorener Snapshot
Gold Standard	Goldstandard
Research Artifact	Forschungsartefakt
Evidence Completeness	Evidenzvollständigkeit
Evidence Precision	Evidenzpräzision
Path Validity	Pfadgültigkeit
Query Construction	Query-Konstruktion
Query Execution	Query-Ausführung
Mapping	Mapping / Zuordnung
Static Analysis	statische Analyse
Dynamic Analysis	dynamische Analyse

Technische Relationstypen werden konsistent in Großbuchstaben geschrieben: READ, WRITE, INVOKES, MAPS_TO, MUTATES, PERSISTS, SUPPORTED_BY. Antwortzustände werden ebenfalls konsistent in Großbuchstaben geschrieben: SUPPORTED, PARTIALLY_SUPPORTED, NOT_ANSWERABLE.

3.9 Zusammenführung der Begriffe im Code-Database-Evidence-Graph-Modell

Die in diesem Kapitel eingeführten Begriffe bilden gemeinsam die konzeptionelle Verarbeitungskette von Cross-Repository Code Intelligence.

- Stufe 1: Korpus. Ein dokumentierter und eingefrorener Korpus stellt die analysierbaren Artefakte bereit.
- Stufe 2: Parsing und semantische Analyse. C#- und SQL-Artefakte werden strukturiert untersucht. Syntax, Typen und Symbole werden soweit möglich aufgelöst.
- Stufe 3: Beobachtungen. Detektoren erzeugen lokale, typisierte und provenienzerhaltende Beobachtungen.
- Stufe 4: Normalisierung. Technologiespezifische Beobachtungen werden in ein gemeinsames Evidenzmodell überführt.
- Stufe 5: Ableitung. Dokumentierte Regeln verbinden Beobachtungen zu fachlichen Relationen wie READ, WRITE oder MAPS_TO.
- Stufe 6: Code-Database Evidence Graph. Artefakte, Programmelemente, Datenbankelemente, Beobachtungen und Relationen werden als abfragbare Struktur miteinander verbunden.
- Stufe 7: Retrieval. Für eine vordefinierte Frage werden die erforderlichen Evidenzpfade gesucht und zusammengestellt.
- Stufe 8: Antwortbarkeitsprüfung. Das System prüft, ob alle Muss-Bedingungen des Fragevertrags erfüllt sind.
- Stufe 9: Antwort oder Verweigerung. Das System gibt entweder eine unterstützte Antwort, eine klar begrenzte teilweise Antwort oder eine begründete Verweigerung aus.

Die gesamte Kette lässt sich folgendermaßen zusammenfassen:

Frozen Corpus → Syntax und Semantik → Beobachtungen → normalisierte Evidenz → abgeleitete Relationen → Code-Database Evidence Graph → Retrieval → Antwortbarkeitsprüfung → SUPPORTED / PARTIALLY_SUPPORTED / NOT_ANSWERABLE (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

3.10 Bedeutung des Kapitels für die folgenden Kapitel

Dieses Kapitel bildet das begriffliche Fundament der weiteren Arbeit. Der Forschungsstand kann nun untersuchen, welche bestehenden Ansätze einzelne Teile dieser Verarbeitungskette unterstützen. Die Forschungslücke kann präzise beschreiben, welche Kombination aus Extraktion, Provenienz, Graphrepräsentation, Retrieval und Antwortbarkeit noch nicht ausreichend untersucht ist (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Die Forschungsfragen können auf klar definierte Konstrukte zurückgreifen: RQ1 untersucht die Qualität der Evidenzextraktion, RQ2 untersucht die Qualität des Retrievals und des Antwortverhaltens. Das Architekturkapitel kann Beobachtungen, Relationen und Evidenzpfade technisch operationalisieren. Die Evaluation kann Precision, Recall, Evidenzvollständigkeit, Nachvollziehbarkeit, Pfadgültigkeit und Verweigerungsverhalten auf eindeutig definierte Einheiten beziehen. Damit verhindert Kapitel 3, dass zentrale Begriffe erst während der Implementierung oder nach Betrachtung der Ergebnisse beliebig interpretiert werden (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

3.11 Abschließende didaktische Erklärung

Cross-Repository Code Intelligence beginnt nicht mit einer fertigen Antwort. Das System arbeitet schrittweise. Zuerst liest es die vorhandenen Dateien. Danach erkennt es konkrete Strukturen im Code. Aus diesen Strukturen erzeugt es kleine, überprüfbare Beobachtungen. Anschließend verbindet es die Beobachtungen durch dokumentierte Regeln. Diese Beziehungen werden in einem Code-Database Evidence Graphs gespeichert (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Wenn eine Frage gestellt wird, sucht das System nicht nur nach ähnlichen Wörtern. Es prüft, welcher vollständige Evidenzpfad für die Frage notwendig ist. Ist der Pfad vollständig, kann es eine unterstützte Antwort geben. Fehlt ein wichtiger Schritt, muss es die Einschränkung nennen oder die Antwort begründet verweigern (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Zentraler Merksatz: Cross-Repository Code Intelligence soll nicht die wahrscheinlichste Antwort erzeugen, sondern die stärkste Antwort, die durch die vorhandene statische Evidenz tatsächlich getragen wird (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Mündliche Kurzfassung: Kapitel 3 definiert die Grundbegriffe meiner Arbeit. Cross-Repository Code Intelligence analysiert C#- und Datenbankartefakte statisch. Es erzeugt zuerst lokale Beobachtungen, zum Beispiel einen DbSet-Zugriff, einen SaveChanges-Aufruf oder ein Mapping. Mehrere Beobachtungen können durch dokumentierte Regeln zu READ- oder WRITE-Relationen verbunden werden. Alle Quellen und Ableitungsschritte bleiben als Provenienz erhalten. Für eine Antwort wird ein vollständiger Evidenzpfad benötigt. Wenn ein wesentlicher Schritt fehlt, gibt das System nur eine teilweise Antwort oder verweigert begründet (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Begriffsüberblick (Kurzreferenz):

Begriff	Bedeutung im Projekt
Korpus	Die festgelegte Sammlung von Quellcode- und Datenbankartefakten, auf der Entwicklung und Evaluation durchgeführt werden.
Corpus Census	Eine vollständige Bestandsaufnahme der für die Untersuchung relevanten Dateien und Artefakttypen.
Frozen Snapshot	Eine unveränderlich referenzierte Version des Korpus, beispielsweise über den Commit-Hash und ein zusätzliches Archiv.
Statische Analyse	Analyse von Programm- und Datenbankartefakten ohne Ausführung des Anwendungssystems.
Beobachtung	Ein direkt aus einer konkreten Quellstelle extrahierter, typisierter Hinweis.
Abgeleitete Relation	Eine Beziehung, die aus mehreren Beobachtungen zusammengesetzt ist und durch eine dokumentierte Regel beschrieben wird.
Provenienz	Nachweis, woher eine Beobachtung oder Aussage stammt: Datei, Position, Symbol, Detektor und Regel.
Goldstandard	Manuell geprüfte Referenzmenge, gegen die die Systemausgaben verglichen werden.
Antwortbarkeit	Feststellung, ob die erforderliche Evidenz innerhalb des definierten Analyseumfangs vollständig vorliegt.
Begründete Verweigerung	Bewusste Nichtbeantwortung einer Frage, weil ein notwendiger Evidenzschritt fehlt oder nicht statisch auflösbar ist.

4. Forschungsstand

Kapitelorientierung - Schritt für Schritt

Thema	Welche Ansätze bereits existieren
Problem	Einzelne Arbeiten behandeln Codegraphen, Retrieval, SQL-Analyse oder Repository-QA, aber nicht exakt dieselbe Kombination.
Lösung	Verwandte Arbeiten werden nach Problem, Methode, Ergebnis und Übertragbarkeit verglichen.
Ergebnis	Eine belastbare Positionierung der Arbeit.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Didaktische Erklärung - einfaches Deutsch

Der Forschungsstand zeigt, welche Bausteine bereits existieren und wo ihre Grenzen liegen. Er bildet die Brücke zwischen vorhandener Forschung und der eigenen Forschungslücke von Cross-Repository Code Intelligence. Ziel ist nicht nur eine Literaturliste, sondern eine strukturierte Einordnung: Welche bestehenden Ansätze liefern Grundlagen für Extraktion, Repräsentation, Retrieval und Methodik, und was bleibt trotz dieser Ansätze offen?

4.1 Compilerbasierte statische Analyse mit Roslyn

Die .NET-Compilerplattform Roslyn stellt Syntaxbäume, Symbolinformationen, semantische Modelle sowie Workspace-Zugriff auf Lösungen, Projekte und Dokumente bereit. Damit lässt sich ein bloßer Texttreffer von einem semantisch aufgelösten Methodenaufruf, einem Typ- oder Property-Zugriff unterscheiden und die in Kapitel 3 eingeführten Begriffe wie AST, Semantic Model und Symbolauflösung praktisch anwenden. Roslyn ist deshalb das vorgesehene Analyseframework für die C#-Seite des Projekts (vgl. Microsoft, 2026a; Shatnawi et al., 2019).

Roslyn liefert jedoch nicht automatisch eine datenbankfachliche Interpretation. Ein aufgelöster Aufruf wie `ToList`, `Find` oder `SaveChanges` muss durch domänenspezifische Regeln in eine mögliche READ- oder WRITE-Beobachtung übersetzt werden. Ebenso muss eine C#-Property erst über ein Mapping mit einer physischen Datenbankspalte verknüpft werden. Cross-Repository Code Intelligence nutzt Roslyn daher als Grundlage für syntaktische und semantische Präzision, ergänzt diese aber um eigene, projektspezifische Regeln zur Erkennung und Typisierung von Datenbankevidenz (vgl. Microsoft, 2026a; Shatnawi et al., 2019).

4.2 Codegraphen

Code Property Graphs kombinieren Abstract Syntax Trees, Control-Flow- und Program-Dependence-Informationen in einer gemeinsamen Graphstruktur und ermöglichen, wiederkehrende Muster als Graphtraversierungen auszudrücken (Yamaguchi et al., 2014). Der Nutzen für Cross-Repository Code Intelligence liegt in der Komposition: Methoden, Aufrufe, Mappings, Tabellen, Spalten und Evidenzquellen lassen sich als verbundener Pfad darstellen, sodass die in Kapitel 3 beschriebenen Evidenzpfade tatsächlich als Graphpfade modelliert und abgefragt werden können.

Neuere Arbeiten wie LogicLens erweitern diese Idee auf semantische Multi-Repository-Graphen und zeigen, dass explizite, repositoryübergreifende Strukturen systemweite Fragen unterstützen können (Usai et al., 2026). Sie illustrieren, wie ein einzelner Codegraph für mehrere Projekte und Artefakttypen gemeinsame Abhängigkeitsstrukturen sichtbar machen kann. Ein allgemeiner Codegraph ist für die

vorliegende Fragestellung jedoch entweder zu breit oder enthält die erforderliche datenbankspezifische Provenienz und Antwortbarkeitslogik nicht automatisch. Cross-Repository Code Intelligence positioniert sich deshalb als evidenzzentrierte Spezialisierung: Es übernimmt die graphbasierte Grundidee, fokussiert aber ausdrücklich auf READ/WRITE-Relationen, Provenienz, Unsicherheit und Antwortbarkeitsregeln.

4.3 Repositoryweites Retrieval

Repository-Level-Retrieval zeigt, dass relevante Informationen häufig außerhalb der unmittelbar betrachteten Datei liegen. Ansätze wie RepoCoder nutzen iteratives Retrieval und Generierung, um verstreuten Repositorykontext für Codevervollständigung nutzbar zu machen, und weisen deutliche Verbesserungen gegenüber In-File- und einfachen Retrieval-Baselines nach (Zhang et al., 2023). Sie demonstrieren, dass Systeme, die beim Suchen von Lösungen gesamte Repositories berücksichtigen, komplexere Aufgaben besser bewältigen können als rein lokale Ansätze.

Ähnlichkeit oder thematische Relevanz ist allerdings kein Beweis dafür, dass eine bestimmte Methode eine bestimmte Spalte liest oder schreibt. Für Cross-Repository Code Intelligence stehen nicht nur „relevante Textstellen“, sondern korrekt rekonstruierte Datenbankoperationen mit nachvollziehbarer Evidenz im Mittelpunkt. Das System bewertet Retrieval daher nicht nur nach Kontextrelevanz, sondern anhand von Antwortkorrektheit, Evidenzvollständigkeit, Nachvollziehbarkeit, Pfadgültigkeit und korrekter Verweigerung. Damit verschiebt sich der Fokus von „passenden Stellen“ zu „tragfähigen Evidenzpfaden“, die eine Antwort unter den in Kapitel 3 definierten Antwortbarkeitskategorien wirklich stützen (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

4.4 Datenbankbezogene Programmanalyse

Verwandte Arbeiten behandeln unter anderem SQL-Injection, ORM-Verhalten, Schemaevolution, Query-Extraktion, Program Slicing und Datenfluss. Nagy et al. (2015) zeigen beispielsweise, wie Ausführungsstellen von SQL-Abfragen aus JDBC- und Hibernate-basierten Systemen statisch lokalisiert und teilweise unaufgelöste Query-Fragmente explizit behandelt werden können. Der Ansatz macht deutlich, dass dynamische oder fragmentierte SQL-Konstruktion nicht ignoriert, sondern als Unsicherheitsquelle sichtbar gemacht werden sollte (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Der Survey von Alshemaimri et al. (2021) verdeutlicht darüber hinaus die Vielfalt datenbankbezogener Zugriffstechniken und Problemklassen über eingebettetes SQL, Call-Level-Interfaces, ORMs und Stored Procedures hinweg. Viele Ansätze optimieren jedoch für eine bestimmte Fehlerklasse (zum Beispiel SQL-Injection) oder einen bestimmten Zugriffsmechanismus (etwa ein spezielles ORM). Sie liefern spezialisierte Detektoren, aber keine einheitliche, provenienzerhaltende Evidenzbasis für unterschiedliche Technologien in einem durchgängig analysierten Repository (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Cross-Repository Code Intelligence untersucht hingegen die Integration heterogener, provenienzerhaltender Evidenz für repositoryweite Datenbankfragen. Es nimmt ernst, dass Enterprise-Systeme zugleich Entity Framework, ADO.NET, eingebettetes SQL und Stored Procedures verwenden können, und betrachtet diese Vielfalt nicht als Randproblem, sondern als Untersuchungsgegenstand. Die Arbeit knüpft an bestehende Analysen an, geht aber darüber hinaus, indem sie eine gemeinsame Evidenzrepräsentation und eine explizit evidenzbasierte Antwortlogik für mehrere Zugriffstechniken entwirft (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

4.5 Design-Science-Research

Design-Science-Research ist geeignet, weil ein technisches Artefakt konstruiert und zugleich wissenschaftlich evaluiert wird. Hevner et al. (2004) beschreiben Design Science als problemorientiertes Forschungsparadigma, in dem Erkenntnis durch Konstruktion, Anwendung und Bewertung innovativer Artefakte entsteht. Ein funktionierender Prototyp allein reicht daher nicht aus. Die Gestaltung des Evidenzmodells, des Korpus, der Vergleichsbedingungen, Messregeln, Grenzen und Rohresultate muss so dokumentiert werden, dass Nutzen, wissenschaftlicher Beitrag und Reproduzierbarkeit überprüfbar sind (vgl. Hevner et al., 2004).

Cross-Repository Code Intelligence folgt dieser Logik: Das System ist nicht nur ein Tool zur Entwicklerunterstützung, sondern ein Forschungsartefakt, das auf klar definierten Anforderungen, Entwurfsentscheidungen und Evaluationen basiert. Design-Science-Research liefert den methodischen Rahmen, in dem Problemdefinition, Artefaktentwurf, Implementierung, Evaluation und Reflexion verbunden werden. Damit ist der Forschungsstand nicht nur technische Literatur, sondern auch methodische Grundlage, um das Projekt zwischen Softwaretechnik, Informationssystemforschung und empirischer Systemanalyse zu verorten (vgl. Hevner et al., 2004).

4.6 Besonders relevante verwandte Systeme

LogicLens konstruiert einen semantischen Multi-Repository-Graphen aus syntaktischer Analyse und LLM-basierter semantischer Anreicherung und nutzt diesen Graphen für natürlichsprachliche, systemweite Fragen (Usai et al., 2026). Das System zeigt, wie eine graphbasierte Struktur mit zusätzlichen, aus Sprachmodellen gewonnenen semantischen Informationen kombiniert werden kann, um Fragen in freier Sprache zu beantworten und mehrere Repositories gemeinsam zu betrachten.

Die Abgrenzung von Cross-Repository Code Intelligence liegt im engeren Fokus auf statisch nachvollziehbarer Evidenz zu Datenbankoperationen, expliziter Quellprovenienz, kontrollierten Extraktionsregeln sowie formalen Antwortbarkeits- und Verweigerungsregeln. LLM-basierte semantische Anreicherung und offene natürlichsprachliche Abfrage gehören nicht zum verbindlichen Kern von Cross-Repository Code Intelligence, sondern bleiben bewusst außerhalb des Pflichtumfangs. Cross-Repository Code Intelligence setzt statische Analyse, explizite Regeln und einen evidenzzentrierten Graphen ein, um klar prüfbare READ- und WRITE-Beziehungen samt Unsicherheiten und Grenzen zu rekonstruieren (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Gerade durch die Kombination dieser Bausteine - Roslyn für statische C#-Analyse, Codegraphen als inspirierende Struktur, repositoryweites Retrieval als Kontextbegriff, datenbankbezogene Analysen als Detektoren und Design-Science-Research als methodischer Rahmen - wird die Forschungslücke sichtbar: Ein System, das heterogene, statisch gewonnene Evidenz zu Datenbankoperationen provenienzerhaltend integriert, graphbasiert repräsentiert und unter expliziten Antwortbarkeitsregeln empirisch vergleicht, ist im aktuellen Forschungsstand noch nicht hinreichend beschrieben. Cross-Repository Code Intelligence setzt genau hier an (vgl. Hevner et al., 2004).

5. Forschungslücke

Kapitelorientierung - Schritt für Schritt

Thema	Was trotz vorhandener Werkzeuge offen bleibt
Problem	Moderne Agents können Code durchsuchen, liefern aber nicht automatisch reproduzierbare, typisierte Code-Datenbank-Evidenz mit Source Locations.
Lösung	Die Arbeit fokussiert einen engeren, empirisch prüfbaren Evidence-Contract.
Ergebnis	Eine begründete wissenschaftliche Nische statt bloßer Tool-Konkurrenz.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Die Forschungslücke beschreibt keine triviale Aussage im Sinne von „Niemand hat das je gemacht“, sondern eine spezifische Kombination vorhandener Bausteine, die bisher nicht ausreichend als durchgängiges, provenienzerhaltendes und vorsichtiges System untersucht wurde. Es geht um eine gezielte Lücke zwischen bestehenden Komponenten - statische Analyse, Codegraphen, datenbankbezogene Detektoren, repositoryweites Retrieval und wissensgestützte QA - und einem integrierten, empirisch evaluierten Gesamtsystem für Datenbankoperationsevidenz in C#/.NET-Repositories (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Bestehende Forschung liefert wichtige Grundlagen in mehreren Bereichen: Code Property Graphs für kombinierte Struktur-, Kontrollfluss- und Abhängigkeitsrepräsentationen (Yamaguchi et al., 2014), statische Analyse heterogener und mehrsprachiger Systeme (Shatnawi et al., 2019), Query-Lokalisierung in datenintensiven Anwendungen (Nagy et al., 2015), repositoryweites Retrieval (Zhang et al., 2023), semantische Multi-Repository-Graphen (Usai et al., 2026) sowie Knowledge-Graph-gestütztes Repository-QA (Abedu et al., 2025). Einzelne Arbeiten adressieren jedoch jeweils bestimmte Fehlerklassen, Zugriffsmechanismen, Repräsentationsformen oder generative Interaktionsmodelle und fokussieren meist entweder auf allgemeine Codeverständnisaufgaben, spezifische Sicherheitsprobleme oder auf natürlichsprachliche Frage-Antwort-Systeme.

Für die hier untersuchte Fragestellung bleibt jedoch eine Lücke bestehen. Diese Lücke betrifft eine reproduzierbare End-to-End-Methodik, die Datenbankoperationsevidenz aus mehreren heterogenen C#- und SQL-Repräsentationen systematisch extrahiert, technologiespezifische Beobachtungen in ein gemeinsames Evidenzmodell normalisiert, für direkte und abgeleitete Beziehungen eine präzise Quellprovenienz nachhält, Evidenz über Dateien, Projekte, Mappings, Methodenaufrufe und Datenbankartefakte hinweg zu inspizierbaren Pfaden verbindet, Extraktionsqualität und Retrievalqualität getrennt evaluiert, für repositoryweite Fragen explizite Bedingungen der Antwortbarkeit festlegt und begründete Verweigerung als messbares positives Systemverhalten behandelt. Weder klassische Codegraphen noch vorhandene datenbankbezogene Analysen kombinieren all diese Aspekte in einem einzigen, durchgängig dokumentierten System (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Die Arbeit behauptet nicht, dass es keinerlei verwandte Frameworks oder Werkzeuge gibt. Vielmehr wird eine spezifische, bewusst eingegrenzte Kombination betrachtet, die im derzeit geprüften Forschungsstand nicht ausreichend als provenienzerhaltendes und vorsichtiges Gesamtsystem für heterogene C#- und SQL-Artefakte dokumentiert ist. Cross-Repository Code Intelligence versteht sich als Integration und Weiterentwicklung dieser Bausteine unter dem Fokus auf statisch gewonnene Datenbankoperationsevidenz, Provenienz und kontrolliertes Antwortverhalten (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

Insbesondere liegt der Fokus auf einem Evidenzmodell, das die Trennung von Beobachtung, abgeleiteter Relation und Antwort strikt einhält, Unsicherheit und ungelöste Stellen nicht nachträglich aus den Pfaden entfernt und für jede ausgegebene Antwort explizit angibt, welche Evidenz sie trägt und welche Lücken zu einer Verweigerung geführt hätten. Damit wird Antwortbarkeit selbst zu einem expliziten Forschungsgegenstand und nicht nur zu einer impliziten Eigenschaft des Systems (vgl. Liu et al., 2026; Abedu et al., 2025).

Damit ergibt sich eine klar umrissene Forschungslücke: Ein System wie Cross-Repository Code Intelligence, das heterogene statische Evidenz zu Datenbankoperationen nicht nur integriert, sondern unter kontrollierten Bedingungen hinsichtlich Extraktion, graphbasierter Repräsentation, Antwortbarkeit und begründeter Verweigerung empirisch vergleicht und dokumentiert, wird im aktuellen Stand der Literatur nicht hinreichend beschrieben. Genau diese Kombination - evidenzzentriertes Graphmodell, getrennte Evaluation von Extraktion und Retrieval, explizite Unsicherheits- und Verweigerungsregeln und ein definierter C#/.NET-Korpus - bildet den wissenschaftlichen Beitrag der Arbeit (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

6. Zielsetzung und Forschungsziele

Kapitelorientierung - Schritt für Schritt

Thema	Was gebaut und untersucht wird
Problem	Ein zu breites Ziel würde gleichzeitig Parser, GraphRAG, Agent, UI und Nutzerstudie verlangen.
Lösung	Kernziele, Mindestartefakt und optionale Erweiterungen werden priorisiert.
Ergebnis	Ein realistischer und bewertbarer Projektumfang.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Didaktische Erklärung - einfaches Deutsch

Die Ziele zerlegen das Gesamtvorhaben in konkrete wissenschaftliche Leistungen: Modell, Extraktoren, Graph, Fragen, Vergleiche und Fehleranalyse. Jedes Ziel muss später durch ein überprüfbares Ergebnis belegt werden. So entsteht aus der allgemeinen Motivation eine klare Liste von Aufgaben, die im Rahmen der Bachelorarbeit tatsächlich erreicht und gemessen werden können.

Übergeordnetes Ziel ist die Entwicklung und empirische Evaluation einer statischen Analysemethodik zur Gewinnung und Abfrage nachvollziehbarer Datenbankoperationsevidenz aus heterogenen C#- und Datenbankartefakten. Die Methodik soll nicht nur ein lauffähiges Tool liefern, sondern ein transparent dokumentiertes Evidenzmodell, eine reproduzierbare Verarbeitungspipeline und eine kontrollierte Evaluation auf einem definierten Korpus (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

- O1 - Evidenzmodell. Ein kanonisches Evidenzmodell für Programm-, Datenbank- und Operationselemente, Mappings, Aufrufe, Provenienz und Unsicherheit definieren. Dieses Modell bildet die konzeptionelle Grundlage von Cross-Repository Code Intelligence und legt fest, welche Artefaktarten, Programmelemente, Datenbankelemente und Relationstypen unterschieden werden, wie Beobachtungen und abgeleitete Relationen beschrieben werden und wie Provenienz und Unsicherheit im System sichtbar bleiben.
- O2 - Extraktoren. Reproduzierbare Extraktoren für ausgewählte C#- und SQL-bezogene Evidenzquellen implementieren. Die Extraktoren setzen das Evidenzmodell praktisch um, indem sie aus Roslyn-basierter C#-Analyse, Entity-Framework-Mappings, SQL-Dateien und gegebenenfalls Stored-Procedure-Definitionen typisierte Beobachtungen erzeugen. Reproduzierbarkeit bedeutet, dass dieselben Artefakte unter denselben Regeln zu denselben Evidenzdaten führen.
- O3 - Code-Database Evidence Graph. Einen Code-Database Evidence Graphs konstruieren, der Fakten über Artefaktgrenzen hinweg verbindet und dabei ihre Herkunft bewahrt. Der Code-Database Evidence Graph verknüpft Beobachtungen aus unterschiedlichen Quellen zu inspizierbaren Pfaden und stellt sicher, dass Beziehungen zwischen Methoden, Hilfsfunktionen, Mappings, Tabellen und Spalten mit ihrer Provenienz, ihrem Status und ihrer Ableitungslogik dokumentiert werden.
- O4 - Fragen und Antwortbarkeit. Realistische repositoryweite Fragen sowie Regeln für Antwortbarkeit, Unvollständigkeit und Verweigerung definieren. Die Fragen orientieren sich an praktischen Wartungs- und Compliance-Szenarien, etwa „Welche Methoden lesen Spalte X?“ oder „Über welche Pfade kann Tabelle Y geändert werden?“. Die Antwortbarkeitsregeln legen fest, unter welchen Bedingungen eine Frage als beantwortbar, teilweise beantwortbar oder nicht beantwortbar gilt und wann eine Verweigerung als korrekt zu bewerten ist.
- O5 - Vergleich der Extraktionskonfigurationen. Extraktionskonfigurationen hinsichtlich Korrektheit und Vollständigkeit vergleichen. Unterschiedliche Konfigurationen - etwa nur Texttreffer, reine

Syntaxanalyse, vollständige Roslyn-Semantik oder zusätzliche SQL- und Mapping-Evidenz - werden gegen einen Goldstandard ausgewertet, um zu messen, wie sich Korrektheit und Vollständigkeit in Abhängigkeit von den verwendeten Quellen und Regeln verändern.

- O6 - Vergleich der Retrievalkonfigurationen. Retrievalkonfigurationen hinsichtlich Antwortkorrektheit, Evidenzvollständigkeit, Nachvollziehbarkeit und korrekter Verweigerung vergleichen. Hier werden mindestens eine textbasierte Baseline, ein flaches Evidenz-Retrieval und ein graphbasiertes Retrieval über Evidenzpfade gegenübergestellt. Bewertet wird nicht nur, ob Antworten inhaltlich korrekt sind, sondern auch, ob die zugehörige Evidenz vollständig, nachvollziehbar und über gültige Pfade verknüpft ist und ob das System bei unzureichender Evidenz begründet verweigert.
- O7 - Fehler- und Grenzanalyse. Fehler und Grenzen analysieren und daraus Hinweise für zukünftige datenbankbezogene Codeanalyse ableiten. Neben quantitativen Metriken sollen typische Fehlermuster, Grenzen der statischen Analyse, problematische Artefaktkonstellationen und Auswirkungen dynamischer Konstrukte qualitativ untersucht werden.

Mit dieser Zielstruktur wird die Brücke von der Ausgangslage (Kapitel 1), der Problemstellung (Kapitel 2) und den Grundlagen (Kapitel 3) zur konkreten Forschungsarbeit geschlagen. Jedes Ziel kann später klar einem Teil der Architektur, des Experiments oder der Auswertung zugeordnet und mit überprüfbaren Ergebnissen belegt werden.

7. Forschungsfragen

Kapitelorientierung - Schritt für Schritt

Thema	Welche Fragen die Arbeit beantwortet
Problem	Technische Features allein sind keine Forschungsfragen.
Lösung	Fragen werden an beobachtbare Variablen, Konfigurationen und Metriken gebunden.
Ergebnis	Ein roter Faden von Literatur über Implementierung bis Evaluation.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Aktuelle Formulierung der Forschungsfragen

RQ1	Wie korrekt und vollständig können statische Analysekonfigurationen Datenbankoperationsevidenz über Codebase- und Projektgrenzen hinweg extrahieren und als gültige Evidence Paths modellieren?
RQ2	In welchem Maß unterstützt der Code-Database Evidence Graph korrekte, vollständige, nachvollziehbare und rollenangemessene Antworten auf definierte Use Cases?
Optionale RQ3	Welchen zusätzlichen Nutzen liefern lexikalische, neuronale oder graphbasierte Retrieval-Varianten gegenüber deterministischen Query Contracts?

Didaktische Erklärung - einfaches Deutsch

RQ1 misst, wie zuverlässig relevante Evidenz extrahiert wird. RQ2 untersucht, wie gut unterschiedliche Repräsentationen und Retrievalverfahren diese Evidenz für korrekte, vollständige und nachvollziehbare Antworten nutzbar machen. Zusammen beschreiben die Forschungsfragen, was Cross-Repository Code Intelligence wissenschaftlich leisten soll: nicht nur Evidenz finden, sondern sie auch in verantwortbare Antworten übersetzen.

7.1 RQ1 - Evidenzextraktion

Wie genau und vollständig können ausgewählte statische Analysestrategien nachvollziehbare Evidenz zu Datenbankoperationen aus heterogenen C#- und SQL-bezogenen Artefakten im eingefrorenen Evaluationskorpus gewinnen? (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

RQ1 wird durch den Vergleich der extrahierten Evidenz mit einem manuell geprüften Goldstandard beantwortet. Ergebnisse werden für definierte Evidenzkategorien und Extraktionskonfigurationen berichtet, nicht nur als ein einziger Gesamtwert. Dadurch wird sichtbar, welche Strategien bestimmte Evidenzarten zuverlässig erkennen und wo sie zu fehlenden oder falschen Hinweisen führen. RQ1 ist direkt mit Ziel O2 (Extraktoren) und O5 (Vergleich der Extraktionskonfigurationen) verknüpft (vgl. Peng et al., 2026; Chen et al., 2025; Du et al., 2025).

7.2 RQ2 - Evidenzretrieval und Antwortbarkeit

Wie schneidet das vorgeschlagene graphbasierte Evidenzmodell im Vergleich zu nicht graphbasierten Repräsentationen hinsichtlich Antwortkorrektheit, Evidenzvollständigkeit, Nachvollziehbarkeit, Coverage und begründeter Verweigerung ab? (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

RQ2 vergleicht mindestens eine Text- oder Keyword-Baseline mit strukturierter Evidenzsuche und graphbasiertem Retrieval. Bewertet werden Aussagen gemeinsam mit der sie tragenden Evidenz, nicht nur sprachlich formulierte Antworten. Damit steht nicht allein im Mittelpunkt, ob eine Antwort „richtig klingt“, sondern ob sie durch vollständige, nachvollziehbare und gültige Evidenzpfade gestützt wird und ob das System in Fällen unzureichender Evidenz korrekt verweigert. RQ2 ist direkt mit Ziel O3 (Code-Database Evidence Graph), O4 (Antwortbarkeitsregeln) und O6 (Vergleich der Retrievalkonfigurationen) verknüpft (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

7.3 Mögliche analytische Unterfragen

- RQ1a: Welche Evidenzkategorien werden zuverlässig erkannt und welche führen zu den meisten False Positives und False Negatives? Diese Unterfrage vertieft RQ1, indem sie Unterschiede zwischen Kategorien sichtbar macht, etwa direkte EF-READs, EF-WRITEs, Hilfsmethoden oder Stored-Procedure-Evidenz.
- RQ1b: Wie viel zusätzliche verifizierte Evidenz entsteht durch Mappings, Hilfsmethoden, SQL-Artefakte und Stored Procedures? Hier wird untersucht, welchen Mehrwert heterogene Evidenzquellen im Vergleich zu rein C#-basierten Strategien liefern.
- RQ2a: Wie verändert graphbasiertes Retrieval die Evidenzvollständigkeit, Nachvollziehbarkeit, Coverage und die richtige Verweigerung? Diese Unterfrage fragt nicht nur, ob graphbasierte Verfahren bessere Antworten liefern, sondern ob sie mehr Evidenz einbeziehen, nachvollziehbare Pfade bereitstellen, eine größere Abdeckung erreichen und häufiger korrekt verweigern.
- RQ2b: Unter welchen Bedingungen verweigert das System korrekt und welche ungelöste Evidenz verhindert die Antwortbarkeit? Hier stehen die Verweigerungsfälle im Mittelpunkt, etwa fehlende Mappings, unaufgelöste Symbole oder dynamische Query-Fragmente.

Die Unterfragen bleiben optional, bis Corpus-Census und Pilotstudie zeigen, dass sie im Umfang einer Bachelorarbeit sinnvoll beantwortbar sind. Sie dienen als Vertiefungspunkte für die Auswertung und können ansonsten als Ausblick für weiterführende Arbeiten genutzt werden (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

8. Untersuchungsumfang und Abgrenzung

Kapitelorientierung - Schritt für Schritt

Thema	Was ausdrücklich in und außerhalb des Scopes liegt
Problem	Unklare Grenzen führen zu ständigem Umplanen und nicht abschließbaren Erweiterungen.
Lösung	Core, bedingte Erweiterungen und Future Work werden getrennt.
Ergebnis	Ein stabiler Scope für die Thesis.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Didaktische Erklärung - einfaches Deutsch

Scope bedeutet, was die Arbeit verbindlich untersucht und was bewusst ausgeschlossen wird. Eine klare Abgrenzung schützt die wissenschaftliche Qualität, weil nur tatsächlich implementierte und evaluierte Fähigkeiten behauptet werden dürfen.

8.1 Verbindlicher Kern

Der verbindliche Kern beschreibt die Fähigkeiten, die im Rahmen der Arbeit tatsächlich implementiert, genutzt und empirisch evaluiert werden sollen:

- C#-Quellcode und Roslyn-basierte Syntax- und Semantikanalyse;
- Entity Framework beziehungsweise die im Korpus tatsächlich eingesetzten EF-Muster;
- DbContext-, DbSet-, LINQ- und SaveChanges-bezogene Evidenz;
- explizite und konventionsbasierte Entity-, Tabellen-, Property- und Spaltenzuordnungen, soweit rekonstruierbar;
- Methoden-, Klassen-, Datei-, Projekt-, Tabellen- und Spaltenprovenienz;
- READ- und WRITE-Operationen auf Tabellen- und, soweit statisch belegbar, auf Spaltenebene;
- Code-Database Evidence Graph und vorab definierte repositoryweite Fragen;
- quantitative Messung und qualitative Fehleranalyse.

Der Kern deckt damit die vollständige Verarbeitungskette von C#-Analyse über EF-basiertes Mapping bis zur Konstruktion und Auswertung eines Code-Database Evidence Graphs ab. Nur für diese Aspekte werden konkrete Leistungsbehauptungen (zum Beispiel zu Precision, Recall, Antwortkorrektheit oder korrekter Verweigerung) erhoben (vgl. Peng et al., 2026; Chen et al., 2025; Du et al., 2025).

8.2 Erweiterungen nach Abschluss des Minimalumfangs

Erweiterungen sind Fähigkeiten, die nach erfolgreicher Implementierung und Evaluation des Minimalumfangs ergänzt werden können, sofern Zeit und Ressourcen dies erlauben:

- ausgewählte ADO.NET-Command-Muster;
- statisch rekonstruierbare eingebettete SQL-Strings;
- SQL-Dateien und ausgewählte Stored-Procedure-Definitionen beziehungsweise Aufrufe;
- begrenzte interprozedurale Verbindung über Hilfsmethoden;
- Cross-Repository-/Cross-Repository/Cross-Project- oder Cross-Database-Beziehungen, sofern sie im eingefrorenen Korpus tatsächlich vorkommen.

Diese Erweiterungen gelten erst dann als Teil des verbindlichen Umfangs, wenn sie wirklich implementiert, dokumentiert und mindestens in einer Pilotstudie geprüft wurden. Bis dahin bleiben sie als „nice to have“ markiert und werden nicht als gesicherte Leistungsmerkmale des Systems dargestellt.

8.3 Außerhalb des Pflichtumfangs

Außerhalb des Pflichtumfangs liegen Fähigkeiten, die für die wissenschaftliche Kernfrage nicht zwingend nötig sind oder den Rahmen einer Bachelorarbeit deutlich sprengen würden: (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

- vollständige Unterstützung aller C#-Features, ORMs und SQL-Dialekte;
- präzise Laufzeitauflösung für Reflection, beliebiges Dynamic Dispatch oder vollständig dynamisch erzeugtes SQL;
- formale Programmverifikation oder vollständige Datenflussanalyse des Gesamtsystems;
- unbegrenztes natürlichsprachliches Question-Answering;
- automatische Code-Reparatur oder Codegenerierung;
- universelle Aussagen über alle Unternehmensrepositories;
- Ersatz für Laufzeittests, Security-Reviews, Rechtsberatung oder menschliche Expertenprüfungen.

Nicht statisch auflösbare Konstrukte werden als ungelöste Evidenz (unresolved evidence) dokumentiert. Sie dürfen nicht automatisch in eine sichere Beziehung umgewandelt werden. Dies ist eine bewusste Designentscheidung: Cross-Repository Code Intelligence soll seine Grenzen offenlegen und Unsicherheit sichtbar machen, statt aus unvollständiger Evidenz scheinbar vollständige Antworten zu konstruieren (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

9. Evaluationskorpus: Danphe

Kapitelorientierung - Schritt für Schritt

Thema	Auf welcher Codebasis untersucht wird
Problem	Ergebnisse sind ohne eingefrorenen Snapshot und dokumentierte Corpus-Auswahl nicht reproduzierbar.
Lösung	Commit, Lizenz, Projektstruktur, Datenbankartefakte und Ausschlüsse werden protokolliert.
Ergebnis	Ein prüfbares Evaluationskorpus.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Noch zu prüfen

Danphe ist der geplante Primärkorpus. Ein Corpus Census muss erst bestätigen, welche EF-, SQL-, Stored-Procedure- und Cross-Project-Muster in ausreichender Zahl vorhanden sind.

Didaktische Erklärung - einfaches Deutsch

Danphe ist die konkrete Fallstudie, an der die Methode entwickelt und geprüft wird. Der Frozen Snapshot sorgt dafür, dass alle Experimente dieselbe Codebasis verwenden und später reproduziert werden können.

9.1 Begründung der Korpuswahl

Danphe ist das vorgesehene Open-Source-Krankenhausinformationssystem für die Fallstudie. Das Projekt eignet sich für die Forschungsfrage, weil es eine reale, modularisierte C#.NET-Codebasis mit SQL-Server-bezogenen Artefakten sowie vielfältigen fachlichen Modulen bereitstellt. Die medizinische Domäne macht Nachvollziehbarkeit besonders anschaulich; daraus darf jedoch nicht ohne separate Prüfung abgeleitet werden, dass jede Beispaltenspalte oder jede angenommene Datenschutzerfordernung im konkreten Snapshot genauso implementiert ist. Die Wahl von Danphe als Evaluationskorpus ist damit unmittelbar mit der späteren Goldstandard-Erstellung verbunden: Nur aus einem klar dokumentierten, eingefrorenen Snapshot kann eine verlässliche Referenzmenge für die Evaluation von Cross-Repository Code Intelligence gewonnen werden (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

9.2 Vorläufig bestätigte Bestandsinformationen

Merkmal	Vorläufiger Stand	Status für die finale Arbeit
C#-Dateien	1.156	Aus der Terminalausgabe der bisherigen lokalen Bestandsaufnahme; auf dem Frozen Snapshot reproduzieren.
C#-Codezeilen	ca. 203.440	Vorläufige einfache Zeilenzählung; Zählregel und Ausschlüsse dokumentieren.
C#-Projekte	10 .csproj-Dateien	Im finalen Snapshot erneut automatisch erfassen.
DbContext-Dateien	49 Dateien nach Namensmuster *DbContext.cs	Dateien öffnen und die tatsächlichen Rollen verifizieren; der Dateiname allein ist keine semantische Garantie.
Komponentenordner	7 unter Code/Components	Snapshot, Pfade und Einschlusskriterien

Merkmal	Vorläufiger Stand	Status für die finale Arbeit
		dokumentieren.
Datenbankartefakte	SQL-Dateien und Datenbankarchive vorhanden	Für die Evaluation nur legal und technisch zugängliche, klar versionierte Artefakte verwenden.

Wichtig: Die Zahl 1.156 ist weder eine Evaluationsleistung noch ein Qualitätswert von Cross-Repository Code Intelligence. Sie ist lediglich eine vorläufige Korpuszählung. Die wissenschaftliche Evaluation beginnt erst, wenn aus einem dokumentierten Snapshot ein überprüfter Goldstandard erstellt und die Systemausgabe mit diesem verglichen wird (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

9.3 Frozen Snapshot und Korpusprotokoll

Vor dem finalen Experiment wird ein Snapshot anhand eines Commit-Hashes oder eines archivierten Zustands eingefroren. Das Korpusprotokoll enthält mindestens Quelle, Abrufdatum, Lizenz, Commit-Hash, Verzeichnis- und Projektumfang, Dateitypen, Ausschlüsse, generierten oder Drittanbieter-Code, Encoding, Toolversionen sowie reproduzierbare Zählskripte. Jede spätere Änderung erzeugt eine neue Korpusversion und darf nicht stillschweigend in die finalen Testergebnisse einfließen. Dieses Protokoll bildet die Grundlage für die Reproduzierbarkeit der Experimente und stellt sicher, dass alle Konfigurationen, Messläufe und Fehleranalysen eindeutig einem bestimmten Korpuszustand zugeordnet werden können (vgl. Lekssays, 2026; Liu et al., 2025).

10. Forschungsartefakt Cross-Repository Code Intelligence

Kapitelorientierung - Schritt für Schritt

Thema	Wie das Forschungsartefakt aufgebaut ist
Problem	Code-, Mapping- und Datenbankinformationen liegen in verschiedenen Repräsentationen vor.
Lösung	Extractor, Normalizer, Code-Database Evidence Graph, Query Layer und Verifier werden als getrennte Module entworfen.
Ergebnis	Eine ausführbare und testbare Referenzimplementierung.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Aktuelle Systemarchitektur

Input	C# Solutions/Projects, EF mappings, SQL files, schema metadata, stored procedures.
Extraction	Roslyn/EF rules plus optional SQL parser and capability-specific adapters.
Normalization	EvidenceRecord with source, rule, entity, operation, target and status.
Representation	Code-Database Evidence Graph
Query	Role-specific Question Contracts and deterministic graph traversals.
Verification	Evidence completeness, path validity and answerability.
Output	SUPPORTED / INCOMPLETE / UNRESOLVED with Source Locations.

Cross-Repository Code Intelligence ist das ausführbare Forschungsartefakt, mit dem die in dieser Arbeit formulierten Forschungsfragen praktisch untersucht werden. Diese Einordnung folgt dem Design-Science-Verständnis von Hevner et al. (2004), nach dem Konstrukte, Modelle, Methoden und Instanziierungen als Artefakte entwickelt und hinsichtlich ihres Nutzens evaluiert werden. Der wissenschaftliche Kern liegt nicht in einem bestimmten Produkt oder einer bestimmten Graphdatenbank, sondern in einem speicherunabhängigen Code-Database Evidence Graphs und einer nachvollziehbaren Verarbeitungspipeline, die Extraktion, Normalisierung, Verknüpfung, Retrieval und Evaluation reproduzierbar miteinander verbindet (vgl. Hevner et al., 2004).

Cross-Repository Code Intelligence wird daher nicht primär nach Produktkriterien (Benutzeroberfläche, Featureumfang) bewertet, sondern nach seiner Fähigkeit, die in den Kapiteln 6 und 7 definierten Ziele und Forschungsfragen nachvollziehbar zu adressieren. Im Zentrum stehen die Qualität der Evidenzextraktion, die Güte und Transparenz der graphbasierten Repräsentation sowie das kontrollierte

Antwortverhalten bei repositoryweiten Fragen zu Datenbankoperationen (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

10.1 Rolle des Artefakts

Cross-Repository Code Intelligence ist ein Forschungsprototyp, der die vorgeschlagene Methodik operationalisiert. Er ist ausdrücklich kein fertiges kommerzielles Produkt, sondern ein Instrument zur kontrollierten Untersuchung von Extraktionsstrategien, Evidenzmodellen, Graphrepräsentationen und Retrievalverfahren. Die Architektur ist so ausgelegt, dass Konfigurationen, Detektoren und Graphmodelle gezielt variiert und ihre Auswirkungen auf Extraktions- und Retrievalqualität gemessen werden können (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

Seine zentrale Aufgabe besteht darin, definierte Korpora und Konfigurationen einzulesen, Beobachtungen nach dokumentierten Regeln zu erzeugen, diese in einem kanonischen Evidenzmodell zu normalisieren, abgeleitete Relationen und Evidenzpfade zu konstruieren, repositoryweite Fragen über diese Pfade zu beantworten oder begründet zu verweigern und alle Zwischenschritte so zu protokollieren, dass sie im Rahmen der Evaluation und Fehleranalyse nachvollzogen werden können. Cross-Repository Code Intelligence ist damit Träger der in Kapitel 3 eingeführten Begriffe und der in Kapitel 6 und 7 formulierten Ziele und Forschungsfragen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

10.2 Geplante Verarbeitungspipeline

Die Verarbeitungspipeline von Cross-Repository Code Intelligence besteht aus klar getrennten Stufen. Jede Stufe hat eine dokumentierte Aufgabe und Schnittstellen zu den anderen Stufen (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

- **Corpus-Intake:** Lösungen, Projekte, C#-Dateien, SQL-Dateien und relevante Konfigurationen werden inventarisiert und erhalten stabile Quellidentifikatoren. Einschluss- und Ausschlussregeln werden dokumentiert, damit nachvollziehbar bleibt, welche Artefakte einbezogen und welche bewusst ausgeschlossen wurden.
- **Parsing und semantische Analyse:** C#-Code wird mit Roslyn syntaktisch und semantisch analysiert. Die semantische Analyse liefert Symbolinformationen und Typbeziehungen, die zur Erkennung von Methodenaufrufen, Property-Zugriffen und DbContext-/DbSet-Strukturen benötigt werden. SQL-Artefakte werden mit einem geeigneten Parser oder nach eng begrenzten, dokumentierten Regeln verarbeitet.
- **Evidenzextraktion:** Detektoren erzeugen typisierte Beobachtungsdatensätze mit Quellort, beteiligtem Programm- und Datenbankelement, Operationstyp, verwendeter Regel und Evidenzstatus. Jeder Detektor arbeitet nach einer klar dokumentierten Regel, etwa für EF-READs, EF-WRITEs, LINQ-Materialisierung, ADO.NET-Kommandos oder SQL-Statements.
- **Normalisierung:** Technologiespezifische Beobachtungen werden in ein kanonisches Evidenzmodell überführt, ohne ihre unterschiedlichen Ursprünge zu verschleiern. Provenienz, Unsicherheit sowie direkte und abgeleitete Natur bleiben explizit erhalten.
- **Evidenzverknüpfung:** Direkte Beobachtungen werden anhand expliziter Regeln mit Symbolen, Methodenaufrufen, Mappings und Datenbankartefakten verknüpft. Daraus entstehen abgeleitete Relationen und Evidenzpfade, die vom ursprünglichen Quellort bis zu Tabellen- oder Spaltenelementen reichen.
- **Graphkonstruktion:** Knoten und Kanten repräsentieren Artefakte, Programmelemente, Datenbankelemente und Evidenzbeziehungen. Der resultierende Code-Database Evidence Graph ist speicherunabhängig modelliert, sodass er prinzipiell auf unterschiedlichen Graphspeichern persistiert oder in verschiedenen Abfrageumgebungen genutzt werden kann.
- **Retrieval:** Vorab definierte Fragen werden in strukturierte Retrievalpläne übersetzt. Diese Pläne legen fest, welche Pfadtypen und Evidenzkategorien für die jeweilige Frage relevant sind und wie Pfade im Graphen zusammengesetzt werden müssen.
- **Antwortbarkeitsprüfung:** Ein Ergebnis wird nur dann als unterstützte Antwort ausgegeben, wenn alle für die Frage erforderlichen Pfadteile vorhanden, gültig und im definierten Scope verankert sind. Fehlt eine wesentliche Relation oder bleibt ein kritischer Aufruf ungelöst, verweigert das System die Antwort begründet und dokumentiert die zugrunde liegende Evidenzlücke.

- Bericht und Export: Antworten enthalten inspizierbare Quellorte, verwendete Regeln, offene Lücken und den Antwortstatus. Rohoutputs und Konfigurationsdaten werden gespeichert, sodass Auswertungen, Fehleranalysen und Wiederholungen der Experimente möglich sind.

10.3 Technische Bausteine

- Roslyn dient der Syntax-, Symbol- und Semantikanalyse für C# und ist als Kernframework für die statische Programmanalyse bestätigt. Es stellt die in Kapitel 3 eingeführten Konzepte wie AST, Semantic Model und Symbolauflösung praktisch bereit.
- Der speicherunabhängige Code-Database Evidence Graph bildet den wissenschaftlichen Kern zur Verknüpfung heterogener Beobachtungen und abgeleiteter Relationen. Die grundsätzliche Eignung graphbasierter Repräsentationen für kombinierte Struktur- und Abhängigkeitsanalysen wird durch Code Property Graphs gestützt; die Übertragbarkeit auf repositoryübergreifende Strukturen wird durch semantische Multi-Repository-Graphen illustriert.
- Ein Graphspeicher dient der Persistenz und Abfrage des Code-Database Evidence Graphs. Die konkrete Technologie, beispielsweise Neo4j oder eine In-Memory-Struktur, wird erst nach einem dokumentierten Prototypvergleich ausgewählt.
- Ein SQL-Parser (etwa ScriptDom für T-SQL oder ein vergleichbares Werkzeug) kann für ausgewählte SQL-Artefakte eingesetzt werden, sofern Korpusanalyse und Machbarkeitsprüfung dies rechtfertigen.
- Eine Benutzeroberfläche oder natürlichsprachliche Abfrage ist sekundär und darf die wissenschaftliche Evaluation nicht ersetzen. Sie kann eingesetzt werden, um Ergebnisse zu inspizieren oder Pfade zu visualisieren, ist aber nicht Teil des verbindlichen Kerns.

10.4 Beobachtung, Ableitung und Antwort

Cross-Repository Code Intelligence unterscheidet strikt zwischen Beobachtungen, abgeleiteten Relationen und Antworten.

Eine Beobachtung liegt vor, wenn eine konkrete Quelltextstelle ein Muster enthält, das ein Detektor gemäß einer dokumentierten Regel erkannt hat. Jede Beobachtung ist typisiert, referenziert ihren Quellort und benennt die Regel, nach der sie entstanden ist. Beobachtungen sind lokal und tragen zunächst nur begrenzte Aussagekraft (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Eine abgeleitete Relation entsteht, wenn mehrere Beobachtungen gemeinsam eine typisierte Beziehung tragen. Die Ableitungsregel und der zugrunde liegende Pfad werden dokumentiert, sodass nachvollziehbar bleibt, welche lokalen Befunde zu welcher übergeordneten Beziehung geführt haben (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Eine Antwort wird ausgegeben, wenn für eine vordefinierte Frage alle erforderlichen Relationen innerhalb des erklärten Analyseumfangs vollständig und gültig sind. Ist dies nicht der Fall, wird die Frage begründet als nicht beantwortbar klassifiziert. Die klare Trennung stellt sicher, dass Extraktions- und Retrievalfehler in der Auswertung voneinander unterschieden und gezielt adressiert werden können (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

10.5 Vorgesehenes Evidenzmodell

Das Evidenzmodell von Cross-Repository Code Intelligence definiert die zentralen Artefakt-, Programm-, Datenbank- und Relationstypen sowie die Struktur von Evidenzdatensätzen und Frageergebnissen (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

- Artefakte umfassen Repository, Solution, Projekt, C#-Datei, SQL-Datei und Konfigurationsdatei.
- Programmelemente umfassen Namespace, Klasse, Methode, Property, Invocation und Symbol.
- Datenbankelemente umfassen Datenbank, Schema, Tabelle, Spalte und Stored Procedure.
- Relationstypen umfassen READ und WRITE für fachliche Datenbankbeziehungen, CALL und INVOKES für Aufrufbeziehungen, MAPS_TO für Mappings, CONTAINS und DECLARES für Strukturbeziehungen sowie weitere Relationstypen nur bei belegtem Bedarf.

Ein Evidenzdatensatz enthält eine stabile Quell-ID, Datei und Position, Detektor, Regel, direkten oder abgeleiteten Status, Gewissheitskategorie und eine kurze Erklärung. Ein Frageergebnis enthält Ziel,

abgerufene Evidenz, Antwortbarkeitszustand, unterstützte Schlussfolgerung und dokumentierte offene Lücken (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

10.6 Umgang mit Unsicherheit

Unsicherheit ist in Cross-Repository Code Intelligence kein Randproblem, sondern ein bewusst modellierter Bestandteil des Evidenz- und Antwortverhaltens. Das System verwendet abgestufte Kategorien, um den Unterstützungsgrad einer Beziehung oder Antwort zu kennzeichnen (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

- `'SUPPORTED'` bedeutet, dass die erforderliche Relation durch verifizierte direkte Beobachtungen oder eine vollständig dokumentierte Ableitungskette gestützt ist. Alle notwendigen Pfadteile sind vorhanden und innerhalb des definierten Scopes gültig.
- `'PARTIALLY_SUPPORTED'` bedeutet, dass ein Teil des Pfades vorhanden ist, aber mindestens ein erforderliches Mapping, ein Symbol oder ein Aufruf ungelöst bleibt.
- `'NOT_ANSWERABLE'` bedeutet, dass die vorhandene Evidenz für die geforderte Schlussfolgerung nicht ausreicht. Das System verweigert die Antwort und benennt die konkrete Lücke.

Unsicherheit wirkt damit direkt auf das Antwortverhalten. Eine vorsichtige Verweigerung ist ein positives, messbares Systemverhalten, wenn sie auf dokumentierten Evidenzlücken beruht. Ein richtig begründetes Schweigen ist ebenso ein Evaluationsziel wie eine korrekt gestützte Antwort (vgl. Liu et al., 2026; Abedu et al., 2025).

11. Forschungsmethodik

Kapitelorientierung - Schritt für Schritt

Thema	Wie wissenschaftlich vorgegangen wird
Problem	Ein funktionierender Prototyp allein beantwortet keine Forschungsfrage.
Lösung	Design Science wird mit kontrollierten Experimenten und dokumentierten Iterationen verbunden.
Ergebnis	Ein nachvollziehbarer Build-Evaluate-Zyklus.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

11.1 Gesamtansatz

Die Arbeit folgt dem Design-Science-Ansatz und verbindet Artefaktkonstruktion mit kontrollierter empirischer Evaluation. Hevner et al. (2004) betonen den iterativen Build-and-Evaluate-Zyklus sowie die Notwendigkeit, Nutzen und wissenschaftlichen Beitrag des Artefakts nachvollziehbar zu bewerten. Die Entwicklung erfolgt daher iterativ, während die finale Messung strikt auf eingefrorenen Regeln, Konfigurationen und Testdaten basiert (vgl. Hevner et al., 2004).

Der Gesamtansatz lässt sich in fünf Zyklen gliedern:

- Problem- und Anforderungszyklus: Fragen, Evidenzkategorien, Scope und Antwortbarkeitsregeln definieren.
- Designzyklus: Evidenzmodell, Detektorschnittstellen, Graphschema, Retrievalstrategien und Ausgabeformat entwerfen.
- Implementierungszyklus: ausgewählte Komponenten als lauffähigen Prototyp realisieren.
- Evaluationszyklus: Konfigurationen mit dem Frozen Corpus und dem Goldstandard vergleichen.
- Reflexionszyklus: Fehler, Validitätsbedrohungen und Designimplikationen analysieren.

11.2 Trennung von Extraktion und Retrieval

Die Extraktionsstufe beantwortet die Frage: Welche Evidenz wurde aus den Artefakten gewonnen? Sie liefert typisierte Beobachtungen und abgeleitete Relationen, unabhängig davon, ob konkrete Fragen bereits gestellt wurden (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Die Retrievalstufe beantwortet die Frage: Welche Evidenz wird für die konkrete Frage benötigt und wurde sie vollständig ausgewählt? Sie stellt Evidenzpfade für definierte Fragen zusammen und entscheidet anhand der Antwortbarkeitsregeln, ob eine Antwort unterstützt, teilweise unterstützt oder nicht verantwortbar ist (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Diese Trennung verhindert, dass gutes Retrieval fehlende Extraktion verdeckt oder dass eine Extraktionslücke fälschlich als reines Antwortproblem interpretiert wird. Sie erlaubt außerdem, Extraktionsqualität (RQ1) und Retrieval-/Antwortqualität (RQ2) getrennt zu messen (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

11.3 Reproduzierbarkeit

Reproduzierbarkeit ist ein zentraler methodischer Anspruch der Arbeit. Damit Ergebnisse nachvollziehbar und wiederholbar sind, werden folgende Elemente explizit dokumentiert:

- eingefrorener Korpus und dokumentierte Einschluss- und Ausschlussregeln;

- versionierter Analyzer-Code und feste Konfigurationen;
- dokumentiertes Graphschema und stabile IDs für Artefakte, Programmelemente und Datenbankelemente;
- versionierter Goldstandard und ein klarer Annotationsguide;
- vorab definierte Fragen und Bewertungsregeln;
- protokollierte Toolversionen, Parameter und Laufzeiten;
- gespeicherte Rohoutputs, Auswertungsskripte und Fehlerlisten.

Didaktische Erklärung - einfaches Deutsch

Der Goldstandard ist die manuell geprüfte Referenz, gegen die das System bewertet wird. Ohne ihn kann man nicht wissen, ob ein gefundener Evidenzpfad wirklich richtig, vollständig oder nur plausibel aussieht. Deshalb müssen Auswahl, Annotation, Versionierung und Nutzung des Goldstandards besonders sorgfältig dokumentiert werden.

Durch diese Maßnahmen wird sichergestellt, dass Experimente nicht nur einmalig funktionieren, sondern von Dritten nachvollzogen, wiederholt und kritisch geprüft werden können.

12. Goldstandard

Kapitelorientierung - Schritt für Schritt

Thema	Wie die richtige Antwort festgelegt wird
Problem	Automatische Ausgaben können nicht mit sich selbst validiert werden.
Lösung	Manuell geprüfte Evidence Records und Answerability-Labels bilden die Referenz.
Ergebnis	Eine unabhängige Grundlage für Precision, Recall und Fehleranalyse.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

12.1 Auswahlstrategie

Eine vollständige manuelle Annotation aller 1.156 C#-Dateien ist für eine Bachelorarbeit voraussichtlich nicht realistisch. Deshalb wird ein geschichtetes, repräsentatives Evaluationssubset gebildet. Es soll direkte Entity-Framework-Reads und -Writes, explizite und konventionsbasierte Mappings, Helper-vermittelte Zugriffe, negative Fälle sowie absichtlich unbeantwortbare Fragen enthalten. Die endgültige Stichprobengröße wird erst nach dem Corpus-Census und der Pilotannotation festgelegt, damit Umfang und Zusammensetzung sowohl machbar als auch für die Evaluationsfragen aussagekräftig sind (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

12.2 Annotationseinheit

Ein Gold-Item enthält mindestens: Subjektmethode, Operationstyp, Ziel-Datenbankelement, Zugriffsmechanismus, erforderlichen Evidenzpfad, exakte Quellorte, Direktheitsstatus und gegebenenfalls eine dokumentierte ungelöste Stelle.

Negative Beispiele werden ebenso explizit erfasst, damit bloße Namenstreffer nicht als positive Evidenz zählen. So wird verhindert, dass allein das Auftauchen eines Tabellennamens oder einer Spalte ohne tatsächlichen Zugriff als Evidenz gewertet wird. Jedes Gold-Item ist damit eine klar definierte Einheit, gegen die Systemausgaben geprüft und kategorisiert werden können.

12.3 Qualitätssicherung

Zur Sicherung der Qualität des Goldstandards werden mehrere Maßnahmen kombiniert:

- schriftlichen Annotation Guide mit positiven, negativen und ambigen Beispielen erstellen;
- Pilotmenge annotieren und Regeln überarbeiten;
- Regeln vor der finalen Testannotation einfrieren;
- jede Gold-Aussage mit Quellort versehen;
- verzögerte zweite Prüfung der Annotation durchführen;
- ambige Fälle gezielt durch eine zweite fachkundige Person prüfen lassen, soweit möglich;
- Änderungen versionieren und Gründe dokumentieren.

Didaktische Erklärung - einfaches Deutsch

Der Annotation Guide und die Pilotannotation sorgen dafür, dass alle Beteiligten ähnliche Entscheidungen treffen, wenn sie Evidenzpfade markieren. Die zweite Prüfung und die Dokumentation von Änderungen stellen sicher, dass der Goldstandard nicht zufällig, sondern bewusst und nachvollziehbar entsteht.

Cohens Kappa ist nur sinnvoll, wenn tatsächlich zwei unabhängig annotierte Mengen mit geeigneten Kategorien vorliegen. Ohne unabhängige Doppelannotation darf kein Inter-Rater-Wert lediglich der Form halber berichtet werden. Wenn eine Doppelannotation im Umfang der Bachelorarbeit nicht realistisch ist, wird dies transparent gemacht und stattdessen auf klare Regeln, Pilotannotation, stichprobenartige Zweitprüfung und detaillierte Dokumentation zurückgegriffen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Damit bleibt der Goldstandard als Referenzmenge nicht nur inhaltlich, sondern auch methodisch nachvollziehbar: Er ist klein genug, um im Rahmen der Arbeit erstellt und geprüft zu werden, und zugleich reichhaltig genug, um die zentralen Evidenzarten und Fehlerklassen von Cross-Repository Code Intelligence abzudecken (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

13. Experimentelles Design

Kapitelorientierung - Schritt für Schritt

Thema	Welche Konfigurationen verglichen werden
Problem	Ohne kontrollierte Baselines lässt sich der Nutzen einzelner Komponenten nicht bestimmen.
Lösung	Extraktions-, Repräsentations- und Retrievalvarianten werden separat variiert.
Ergebnis	Vergleichbare und reproduzierbare Experimente.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Didaktische Erklärung - einfaches Deutsch

Das experimentelle Design legt fest, welche Varianten fair miteinander verglichen werden. Baselines zeigen, ob die zusätzlich eingeführte Komplexität der Methoden tatsächlich einen messbaren Vorteil bringt. Ziel ist, dass jede Konfiguration und jede Fragegruppe transparent definiert ist, bevor gemessen wird.

13.1 Extraktionskonfigurationen

- E0 - Text- oder namensbasierte Baseline. Eine einfache Konfiguration, die vor allem auf Texttreffern und Namen basiert und keine vollständige syntaktische oder semantische Auflösung vornimmt. Sie dient als Referenz, um zu zeigen, welchen Mehrwert strukturierte Analyse überhaupt bringt.
- E1 - Roslyn-Syntaxanalyse ohne vollständige semantische Auflösung. Diese Konfiguration nutzt Syntaxbäume, um Sprachkonstrukte strukturiert zu erkennen, verzichtet aber auf vollständige Typ- und Symbolauflösung.
- E2 - Syntax plus Roslyn-Semantik und Symbolauflösung. Hier werden Syntax und semantische Modelle kombiniert, inklusive Symbolauflösung. Die Konfiguration repräsentiert einen realistischen, compilerbasierten Analyseansatz ohne zusätzliche Datenbank- oder Mappinglogik.
- E3 - Semantische C#-Analyse plus Entity-Framework-Mappings. Diese Konfiguration ergänzt E2 um EF-Mappings und konventionsbasierte Zuordnungen. Sie erlaubt, READ- und WRITE-Beziehungen bis zu Tabellen- und Spaltenebene zu verfolgen, soweit Mappings rekonstruiert werden können.
- E4 - Ausgewählte heterogene Gesamtkonfiguration mit SQL- oder Stored-Procedure-Evidenz, sofern möglich. In E4 werden zusätzlich statisch rekonstruierbare SQL-Dateien, eingebettete SQL-Strings und ausgewählte Stored-Procedure-Definitionen oder -Aufrufe integriert. Die Konfiguration repräsentiert den maximalen, im Projekt realistisch erreichbaren Umfang heterogener Evidenzquellen.

Die genaue Zahl der Konfigurationen kann nach der Pilotstudie reduziert werden. Verbindlich bleiben mindestens eine sinnvolle Baseline (zum Beispiel E0) und eine integrierte vorgeschlagene Konfiguration (zum Beispiel E3 oder E4), damit sowohl Minimal- als auch Maximalumfang empirisch vergleichbar sind.

13.2 Retrievalkonfigurationen

- R0 - Exakte Textsuche als Baseline. Eine einfache, textbasierte Suche über Dateien und Pfade, die als untere Referenzlinie dient.
- R1 - Symbolbewusstes, nicht graphbasiertes Retrieval. Diese Konfiguration nutzt Symbolinformationen, um Treffer auf Methoden, Properties und Typen zu beschränken, bleibt aber ohne explizite Graphstruktur.

- R2 - Flaches Retrieval über normalisierte Evidenzdatensätze. Hier werden normalisierte Evidenzdatensätze direkt abgefragt, ohne Pfade im Graphen zu traversieren.
- R3 - Graphbasiertes Retrieval über verbundene Evidenzpfade. Diese Konfiguration nutzt den Code-Database Evidence Graphs aktiv: Fragen werden als Pfad- und Mustersuchen formuliert, die über Artefakt-, Programm- und Datenbankknoten verlaufen.
- R4 - Graphbasiertes Retrieval mit expliziten Antwort- und Verweigerungsregeln. R4 erweitert R3 um die in Kapitel 3 und 6 definierten Antwortbarkeitsregeln. Das System gibt eine Antwort nur aus, wenn alle für die Frage notwendigen Pfadteile vorhanden und gültig sind; ansonsten verweigert es begründet. Diese Konfiguration repräsentiert den vollständigen, vorsichtigen Ansatz von Cross-Repository Code Intelligence.

Auch hier kann die Zahl der Konfigurationen nach der Pilotstudie angepasst werden, solange mindestens eine Baseline (R0) und eine voll ausgestattete graphbasierte Konfiguration (R4) erhalten bleiben.

13.3 Fragenmenge

Die Fragen werden vor dem finalen Experiment festgelegt und umfassen unterschiedliche Antwortkategorien. Vorgesehen sind Fragen zu:

- lesenden oder schreibenden Methoden für bestimmte Tabellen oder Spalten,
- eingesetzten Zugriffsmechanismen (zum Beispiel EF, ADO.NET, eingebettetes SQL, Stored Procedures),
- Beweispfaden für bestimmte Datenbankoperationen,
- fehlenden oder unklaren Mappings,
- und zur ausreichenden Beleglage für eine bestimmte Aussage.

Die Menge enthält beantwortbare, teilweise beantwortbare und bewusst unbeantwortbare Fälle. Beantwortbare Fragen sind solche, für die der Goldstandard eine klar gestützte Antwort definiert; teilweise beantwortbare Fragen besitzen nur teilweise Evidenzpfade; bewusst unbeantwortbare Fragen sind so gewählt, dass statische Evidenz für eine vollständige Antwort fehlt. So kann das Systemverhalten in allen Antwortbarkeitskategorien beobachtet werden (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Die endgültige Anzahl steht erst nach dem Corpus-Census und der Pilotstudie fest. Zuerst wird geprüft, welche Fragen im ausgewählten Evaluationssubset sinnvoll und realistisch sind, danach wird eine Anzahl gewählt, die für die statistische Auswertung ausreichend, aber im Rahmen einer Bachelorarbeit handhabbar bleibt (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

14. Metriken und Auswertung

Kapitelorientierung - Schritt für Schritt

Thema	Wie Ergebnisse gemessen werden
Problem	Nur Accuracy verdeckt unvollständige, falsche oder unnötig verweigerte Antworten.
Lösung	Mehrere Metriken erfassen Extraktion, Retrieval, Pfade, Traceability und Refusal.
Ergebnis	Eine differenzierte Bewertung statt einer einzigen Zahl.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Didaktische Erklärung - einfaches Deutsch

Metriken übersetzen Begriffe wie korrekt, vollständig und nachvollziehbar in messbare Größen. Da ein einziger Gesamtwert wichtige Fehler verdecken kann, werden die Ergebnisse getrennt nach Kategorien und gemeinsam mit den Rohzahlen berichtet.

Dieses Kapitel beschreibt, wie die Leistungen von Cross-Repository Code Intelligence im Hinblick auf Evidenzextraktion, Retrieval und Antwortverhalten messbar gemacht werden. Ziel ist es, Begriffe wie korrekt, vollständig, nachvollziehbar und vorsichtig in nachvollziehbare Kennzahlen zu übersetzen, statt sie nur qualitativ zu diskutieren. Jede Metrik wird daher mit Zähler, Nenner, Auswertungseinheit und Aggregationsregel definiert (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

14.1 Extraktionsmetriken

Für die Extraktionsstufe werden klassische Metriken aus der Informationsrückgewinnung eingesetzt, ergänzt durch Rohzahlen und Kategorien.

Precision misst den Anteil korrekter Evidenzelemente unter allen vom System extrahierten Evidenzelementen.

$$\text{Precision} = (\text{TP}) / (\text{TP} + \text{FP})$$

- Zähler: Anzahl korrekt extrahierter Evidenzelemente (im Vergleich zum Goldstandard).
- Nenner: Anzahl aller vom System extrahierten Evidenzelemente.
- Auswertungseinheit: Evidenzdatensatz (zum Beispiel einzelne EF-READ, EF-WRITE, Stored-Procedure-Hinweis).
- Aggregationsregel: Berechnung pro Evidenzkategorie und pro Extraktionskonfiguration; anschließende Aggregation über Kategorien nur mit gleichzeitiger Berichterung der Rohzahlen.

Recall misst den Anteil der Goldstandard-Evidenz, den das System gefunden hat.

$$\text{Recall} = (\text{TP}) / (\text{TP} + \text{FN})$$

- Zähler: Anzahl korrekt vom System gefundener Gold-Evidenzelemente.
- Nenner: Anzahl aller im Goldstandard definierten relevanten Evidenzelemente.
- Auswertungseinheit: Evidenzdatensatz.
- Aggregationsregel: Berechnung pro Evidenzkategorie und pro Extraktionskonfiguration; keine alleinige Nutzung eines Gesamtwerts ohne Kategorien.

F1-Score ist das harmonische Mittel aus Precision und Recall und berücksichtigt sowohl falsch positive als auch falsch negative Evidenz.

$$F1 = 2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall})$$

- Auswertungseinheit: Evidenzdatensatz in einer Kategorie und Konfiguration.
- Aggregationsregel: Berechnung pro Kategorie und Konfiguration; Gesamt-F1 nur als Zusatz, nicht als einzige Kennzahl.

Diese Metriken werden nicht nur aggregiert über alle Evidenzdatensätze berichtet, sondern zusätzlich pro Evidenzkategorie, beispielsweise direkte EF-READs, EF-WRITEs, Hilfsmethoden und Stored-Procedure-Evidenz, sowie pro Extraktionskonfiguration E0 bis E4. Ein einzelner Gesamtwert könnte Fehler in kleineren oder besonders schwierigen Kategorien verdecken. Daher werden Rohzahlen (Anzahl gefundener, korrekter, falscher und verpasster Evidenzelemente) sowie Kategorieergebnisse explizit dargestellt. Die genaue formale Spezifikation der Zähler, Nenner und Aggregationsregeln wird in diesem Kapitel tabellarisch festgehalten (siehe Abschnitt 14.4) (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

14.2 Retrieval- und Antwortmetriken

Für die Retrievalstufe und das Antwortverhalten werden Metriken verwendet, die nicht nur die sprachlich formulierten Antworten, sondern ihre Evidenzbasis bewerten (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Antwortkorrektheit prüft, ob die ausgegebene Schlussfolgerung mit der Goldantwort übereinstimmt. Zähler: Anzahl korrekt beantworteter Fragen. Nenner: Anzahl aller Fragen, zu denen das System eine Antwort ausgibt. Auswertungseinheit: Frage/Antwort-Paar. Aggregationsregel: getrennte Auswertung pro Retrievalkonfiguration und Fragetyp (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Evidenzvollständigkeit misst, ob alle für die Antwort erforderlichen Gold-Evidenzelemente abgerufen wurden. Zähler: Anzahl korrekt abgerufener Gold-Evidenzelemente pro Frage. Nenner: Anzahl aller im Goldstandard für diese Frage definierten erforderlichen Evidenzelemente. Auswertungseinheit: Frage (mit zugehöriger Evidenzmenge). Aggregationsregel: Mittelwerte und Verteilungen pro Konfiguration und Fragetyp (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Evidenzpräzision bewertet, ob die zurückgegebene Evidenz relevant und gültig ist. Zähler: Anzahl relevanter und korrekter Evidenzelemente in der zurückgegebenen Evidenzmenge. Nenner: Anzahl aller zurückgegebenen Evidenzelemente. Auswertungseinheit: fragebezogene Evidenzmenge. Aggregationsregel: analog zu Precision in der Extraktion, aber auf Frageebene (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Vollständigkeit der Nachvollziehbarkeit misst, welcher Anteil der Antwortaussagen mit einer inspizierbaren Quelle verknüpft ist. Zähler: Anzahl Antwortaussagen mit klar dokumentiertem Evidenzpfad. Nenner: Anzahl aller in der Antwort formulierten Aussagen. Auswertungseinheit: Antworttext mit zugehörigen Pfaden. Aggregationsregel: Auswertung pro Konfiguration und Fragetyp (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Pfadgültigkeit prüft, ob jede Kante eines Evidenzpfades durch eine direkte Beobachtung oder eine definierte Ableitungsregel begründet ist. Zähler: Anzahl vollständig gültiger Pfade. Nenner: Anzahl aller ausgegebenen Pfade für eine Frage. Auswertungseinheit: Evidenzpfad. Aggregationsregel: Anteil gültiger Pfade pro Konfiguration, ergänzt durch Rohzahlen (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

Zusätzlich werden die Rate korrekter Verweigerungen, die Rate falscher Verweigerungen, die Rate nicht gestützter Antworten und die Abdeckung berichtet. Zähler/Nenner für diese Raten sind jeweils „Anzahl Fälle mit bestimmtem Verhalten“ dividiert durch „Anzahl aller Fälle in der relevanten Kategorie“ (beispielsweise alle laut Goldstandard nicht beantwortbaren Fragen). Auswertungseinheit: Frage mit Systemverhalten (Antwort oder Verweigerung). Aggregationsregel: getrennte Berichte pro Konfiguration und Fragetyp (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Diese Metriken werden für unterschiedliche Retrievalkonfigurationen und Fragetypen ausgewertet, um zu prüfen, ob graphbasierte und provenienzerhaltende Retrievalstrategien zu besseren, nachvollziehbareren und vorsichtigeren Antworten führen als textbasierte oder flache Baselines (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

14.3 Antwortbarkeitsmatrix

Die Antwortbarkeitsmatrix unterscheidet zwischen dem Goldstatus einer Frage und dem Systemverhalten. Sie ordnet jede Frage in eine Kombination aus „Gold: beantwortbar / nicht beantwortbar“ und „System: Antwort / Verweigerung“ ein (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

	System: Antwort	System: Verweigerung
Gold: beantwortbar	Antwortkorrektheit und Evidenzvollständigkeit werden bewertet	Falsche Verweigerung
Gold: nicht beantwortbar	Nicht gestützte Antwort	Korrekte Verweigerung

Diese Matrix macht deutlich, dass nicht nur falsche positive Antworten problematisch sind, sondern auch Fälle, in denen ein System aus übermäßiger Vorsicht Antworten verweigert, obwohl genügend Evidenz vorhanden wäre. Sie bildet die Grundlage, um die Kategorien SUPPORTED, PARTIALLY_SUPPORTED, NOT_ANSWERABLE und korrekte Verweigerung aus Kapitel 3 und 6 empirisch zu messen (vgl. Liu et al., 2026; Abedu et al., 2025).

14.4 Statistische Analyse

Die quantitative Auswertung umfasst in jedem Fall deskriptive Statistiken, Rohzahlen zu richtigen, falschen und fehlenden Evidenzelementen beziehungsweise Antworten, Konfusionsmatrizen für ausgewählte Kategorien sowie Ergebnisse pro Extraktions- und Retrievalkonfiguration und pro Fragetyp. Dadurch werden sowohl globale Trends als auch spezifische Fehlermuster sichtbar (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Formale Signifikanztests, beispielsweise ein Wilcoxon-Test oder ein anderes geeignetes Verfahren, werden nur eingesetzt, wenn die Voraussetzungen hinsichtlich Messniveau, Stichprobengröße, Unabhängigkeit und Vergleichsdesign erfüllt sind. Statistische Verfahren werden nicht vorab als Pflicht festgelegt, sondern erst nach dem finalen experimentellen Design ausgewählt und begründet. Ziel ist eine sinnvolle, aber nicht künstlich überladene statistische Betrachtung (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Für jede Metrik werden zusätzlich Zähler, Nenner, Auswertungseinheit, Aggregationsregel und der Umgang mit Teiltreffern formal spezifiziert und so dokumentiert, dass die Auswertungsskripte und berechneten Kennzahlen später eindeutig nachvollzogen und gegebenenfalls reproduziert werden können (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

15. Fehleranalyse

Kapitelorientierung - Schritt für Schritt

Thema	Warum das System scheitert
Problem	Aggregierte Kennzahlen zeigen nicht, welche technischen Muster problematisch sind.
Lösung	Fehler werden nach Ursache, Artefakttyp, Frameworkmuster und Evidence Gap kategorisiert.
Ergebnis	Konkrete Verbesserungs- und Limitationsaussagen.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Die quantitative Auswertung wird durch eine qualitative Taxonomie ergänzt. Vorgesehene Fehlerklassen sind ungelöste Symbole, Aliase, indirekte Mappings, Hilfsmethoden, interprozedurale Lücken, dynamisches SQL, Stored-Procedure-Parsing, nicht unterstützte ORM-Muster, generierter Code, mehrdeutige READ/WRITE-Klassifikation, ausgelassene Retrievalpfade und falsche Antwortbarkeitsentscheidungen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Für jeden Fehlertyp werden Häufigkeit, repräsentative Fälle, die technische Ursache, der Einfluss auf die Forschungsfragen sowie mögliche Verbesserungen dokumentiert. Die Fehleranalyse betrachtet dabei nicht nur „klassische Bugs“, sondern auch strukturelle Grenzen der statischen Analyse, problematische Artefaktkonstellationen und typische Situationen, in denen Antwortbarkeit eingeschränkt ist oder Verweigerungen auftreten (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

16. Erwartete Beiträge

Kapitelorientierung - Schritt für Schritt

Thema	Welchen wissenschaftlichen und praktischen Wert die Arbeit liefert
Problem	Eine Thesis muss mehr beitragen als Quellcode.
Lösung	Artefakt, Datenmodell, Goldstandard, Evaluation und Designwissen werden als Beiträge getrennt.
Ergebnis	Eine klare Contribution-Argumentation.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Aus der Zielsetzung und den Forschungsfragen ergeben sich mehrere konkrete Beiträge, die am Ende der Arbeit überprüfbar vorliegen sollen.

- C1 - Konzeptioneller Beitrag: Ein speicherunabhängiges, provenienzerhaltendes Evidenzmodell, das Beobachtungen, abgeleitete Relationen und Antworten klar voneinander trennt und für heterogene C#- und SQL-Artefakte anwendbar ist.
- C2 - Methodischer Beitrag: Eine dokumentierte Methodik zur getrennten Evaluation von Extraktion (RQ1) und Retrieval/Antwortbarkeit (RQ2), die Fehlerquellen eindeutig zuordnet und reproduzierbar ist.
- C3 - Artefaktbeitrag: Ein lauffähiger Prototyp Cross-Repository Code Intelligence, der Roslyn-basierte Extraktion, Code-Database Evidence Graph-Konstruktion und mehrere Retrievalstrategien in einem reproduzierbaren Pipeline-Aufbau vereint.
- C4 - Empirischer Beitrag: Ein manuell geprüfter Goldstandard über einem eingefrorenen realen Korpus sowie kontrollierte Experimente, die verschiedene Extraktions- und Retrievalkonfigurationen unter denselben Bedingungen vergleichen.
- C5 - Analytischer Beitrag: Eine qualitative Fehlertaxonomie, die typische Grenzen statischer Analyse in realistischen Datenbankzugriffsmustern systematisch beschreibt.
- C6 - Didaktischer Beitrag: Eine nachvollziehbare, teilweise zweisprachig erklärte Langfassung, die das Vorhaben so aufbereitet, dass es verstanden, verteidigt und später in eine kompakte offizielle Exposéfassung überführt werden kann.

Der praktische Beitrag besteht darin, dass Entwicklerinnen und Architektinnen nachvollziehbare, belegte Aussagen über Datenbankzugriffe erhalten, statt sich auf reine Namenssuche oder unbelegte Vermutungen verlassen zu müssen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

17. Erfolgskriterien

Kapitelorientierung - Schritt für Schritt

Thema	Wann das Projekt als erfolgreich gilt
Problem	Perfektion ist bei statischer Analyse nicht realistisch, aber unklare Ziele sind nicht prüfbar.
Lösung	Mindestfunktion, Messbarkeit, Reproduzierbarkeit und dokumentierte Grenzen werden definiert.
Ergebnis	Objektive Abschlusskriterien.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Die Arbeit gilt als erfolgreich, wenn die folgenden Kriterien erfüllt sind:

- Ein eingefrorener Korpus-Snapshot mit Commit-Hash, Archiv und Korpusmanifest liegt vor und wird für alle Experimente verwendet.
- Ein manuell geprüfter Goldstandard mit positiven, negativen und nicht beantwortbaren Fällen ist erstellt, pilotiert und vor dem finalen Test eingefroren.
- Der Prototyp extrahiert belegte READ- und WRITE-Relationen mit vollständiger Provenienz für mindestens die Entity-Framework-Kernmuster des Korpus.
- Extraktion und Retrieval werden getrennt und unter identischen Bedingungen für mehrere Konfigurationen evaluiert.
- Für RQ1 liegen Precision, Recall, F1 und Kategorieergebnisse vor; für RQ2 liegen Antwortkorrektheit, Evidenz- und Nachvollziehbarkeitsmetriken sowie Verweigerungsraten vor.
- Die Antwortbarkeitslogik unterscheidet nachweisbar zwischen beantwortbaren und nicht beantwortbaren Fragen und liefert begründete Verweigerungen.
- Eine qualitative Fehleranalyse dokumentiert die wichtigsten Fehlerklassen und ihre Ursachen.
- Alle zentralen Aussagen der Arbeit sind durch Evidenz, Quellorte oder dokumentierte Ableitungsregeln gestützt (Claim-Evidence-Audit).

18. Validitätsbedrohungen

Kapitelorientierung - Schritt für Schritt

Thema	Wie die Aussagekraft begrenzt sein kann
Problem	Ein einzelnes Open-Source-System und manuelle Annotation können Verzerrungen erzeugen.
Lösung	Konstrukt-, interne, externe und Schlussfolgerungsvalidität werden systematisch geprüft.
Ergebnis	Transparente Grenzen der Ergebnisse.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Die Aussagekraft der Ergebnisse hängt von mehreren Validitätsdimensionen ab, die bewusst offengelegt und, soweit möglich, kontrolliert werden.

18.1 Konstruktvalidität

Es besteht das Risiko, dass die gemessenen Größen nicht genau das erfassen, was sie messen sollen. Diesem Risiko wird durch präzise Definitionen von READ, WRITE, Evidenzpfad, Antwortbarkeit und den zugehörigen Metriken begegnet. Jede Metrik erhält Zähler, Nenner, Auswertungseinheit und Aggregationsregel.

18.2 Interne Validität

Die interne Validität ist bedroht, wenn Regeln oder Detektoren unbemerkt an den finalen Testfällen optimiert werden. Die Trennung in Entwicklungs-, Pilot- und Testanteile sowie das gemeinsame Einfrieren von Regeln, Konfigurationen, Fragen und Gold-Items vor dem finalen Experiment sollen dies verhindern (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

18.3 Externe Validität

Da nur ein Fallstudienkorpus (Danphe) untersucht wird, sind die Ergebnisse nicht ohne Weiteres verallgemeinerbar. Diese Grenze wird transparent gemacht. Die Aussagen gelten zunächst für das untersuchte System und die tatsächlich vorkommenden Muster, nicht für beliebige C#- oder SQL-Projekte.

18.4 Schlussfolgerungsvalidität

Statistische Schlüsse können durch kleine Stichproben oder ungeeignete Verfahren verzerrt werden. Deshalb werden Signifikanztests nur eingesetzt, wenn ihre Voraussetzungen erfüllt sind, und stets zusammen mit Rohzahlen und deskriptiven Statistiken berichtet.

18.5 Researcher Bias

Da die Arbeit von einer einzelnen Person erstellt wird, besteht die Gefahr systematischer Verzerrungen bei Annotation und Bewertung. Schriftliche Annotationsregeln, ein pilotierter Goldstandard und, soweit realistisch, eine Form des Zweitreviews sollen diesen Einfluss begrenzen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

18.6 Ethisch-rechtliche Aspekte

Der Umgang mit realem Code muss lizenz- und datenschutzkonform erfolgen. Es werden keine sensiblen personenbezogenen Daten aus dem Korpus ausgewertet; der Fokus liegt auf strukturellen Codeeigenschaften und Datenbankzugriffsmustern, nicht auf konkreten Patientendaten (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

19. Ethische und rechtliche Rahmenbedingungen

Kapitelorientierung - Schritt für Schritt

Thema	Welche Verantwortung gilt
Problem	Healthcare-Code kann sensible Domänenlogik enthalten, auch wenn keine Patientendaten verarbeitet werden.
Lösung	Lizenz, Datenschutz, Datenminimierung und Claim-Grenzen werden dokumentiert.
Ergebnis	Ein verantwortbarer Forschungsprozess.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Das untersuchte System stammt aus dem Gesundheitsbereich. Die Analyse beschränkt sich ausdrücklich auf statische Code- und Schemaeigenschaften. Reale Patientendaten werden nicht verarbeitet. Lizenzbedingungen des Korpus werden geprüft und eingehalten, und der eingefrorene Snapshot wird ausschließlich für wissenschaftliche Zwecke im Rahmen dieser Arbeit verwendet.

20. Arbeits- und Zeitplan

Bestätigte Planungsregel

Die frühere 42- oder 48-Wochen-Darstellung wird nicht verwendet. Vor der Anmeldung dient die Zeit bis 17. September 2026 nur der Vorbereitung, dem Pilot und den Kursprojekten. Die eigentliche Thesis-Roadmap wird als 16 aktive Arbeitswochen plus zwei Pufferwochen geplant.

20.1 Vorbereitungsphase bis 17. September 2026

Zeitraum	Fokus	Verbindlicher Output
27.07.-16.08.	Scope verstehen, Corpus Census vorbereiten, Kernliteratur lesen, Mini-Fixtures bauen	Scope Note, Corpus Inventory, erste Evidence Fixtures
17.08.-08.09.	Je Kurssitzung lernen; am Folgetag genau ein wiederverwendbares Modul bauen	Tokenizer, Retrieval-Baselines, Metrics, Query Parser, optionale DL-Experimente
09.09.-17.09.	Module konsolidieren, Pilot-Demo, deutsche und englische Erklärung	Pilot Report, Demo, GitHub-Dokumentation, Fragenliste für Betreuung

20.2 Thesis-Roadmap nach Anmeldung

Phase	Arbeit	Output
W1-W2	Corpus freeze, Reproduzierbarkeit, Use-Case-Verträge	Commit-Hash, Lizenznotiz, Corpus Census, Question Contracts
W3-W5	Roslyn-/EF-Core-Extraktion und Fixtures	Method-, Symbol-, Mapping-, READ-/WRITE-Extractor
W6-W7	SQL/ADO.NET/Stored Procedure und Normalisierung	Evidence Records, Capability Matrix, unresolved states
W8-W9	Code-Database Evidence Graph und Query Layer	Schema, Import, Constraints, Cypher Queries
W10	Pilot-Goldstandard und Annotation Protocol	Pilotset, Guidelines, Konfliktregeln
W11-W12	Hauptexperimente und Baselines	Extraktions-, Retrieval- und Path-Metriken
W13	Role-specific outputs und Answerability	Developer-, Architect-, QA-Templates, refusal tests
W14	Fehleranalyse und Validitätsprüfung	Error Taxonomy, Threats to Validity
W15	Schreiben und Abbildungen	Method, Results, Discussion
W16	Reproduktion, Abschlussprüfung und Präsentation	Replication Package, finaler Build, Vortrag
P1-P2	Puffer	Nur für echte Blocker; kein neuer Scope

Frage für Prof. Schmid

Dürfen Corpus Census, Mini-Fixtures und ein technischer Pilot vor der offiziellen Anmeldung vorbereitet werden, sofern die eigentlichen Experimente und die finale Datenerhebung erst nach Anmeldung beginnen?

21. Vorläufige Gliederung der Bachelorarbeit

Kapitelorientierung - Schritt für Schritt

Thema	Wie die spätere Thesis aufgebaut wird
Problem	Ein Lern-Exposé und eine Abgabe-Thesis haben unterschiedliche Funktionen.
Lösung	Kapitel werden entlang Problem, Methode, Artefakt, Evaluation und Diskussion angeordnet.
Ergebnis	Eine vorbereitete Schreibstruktur.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Die folgende Gliederung ist ein vorläufiger Entwurf und wird im Verlauf der Arbeit mit der Betreuung abgestimmt.

- Einleitung - Motivation, praktisches Wartungs- und Analyseproblem, Zielsetzung und Aufbau der Arbeit.
- Grundlagen und Begriffe - Statische Analyse, Roslyn, Entity Framework, Evidenzmodell, Provenienz, Antwortbarkeit.
- Verwandte Arbeiten - Code Property Graphs, SQL-Lokalisierung, mehrsprachige statische Analyse, repositoryweites Retrieval, KG-gestütztes Repository-QA, Forschungslücke.
- Forschungsdesign - Forschungsfragen, Use Cases, Untersuchungsumfang, Design-Science-Rahmen.
- Artefakt Cross-Repository Code Intelligence - Architektur, Evidenzschema, Extraktion, Graphkonstruktion, Retrieval- und Antwortbarkeitslogik.
- Evaluationsmethodik - Korpus, Goldstandard, Konfigurationen, Metriken, experimentelles Design.
- Ergebnisse - Extraktions- und Retrievalergebnisse, Antwortbarkeitsmatrix, Fehleranalyse.
- Diskussion und Ausblick - Interpretation, Validitätsbedrohungen, Grenzen, zukünftige Arbeit, Fazit.

22. Offene Entscheidungen und Fragen für Prof. Schmid

Leseregel

Rot markierte Punkte sind keine versteckten Entscheidungen. Sie müssen entweder durch Literatur, Pilotdaten oder ein Betreuungsgespräch geschlossen werden.

Frage für Prof. Schmid

Welche konkrete Interpretation von „Cross-Repository“ soll als Mindestanforderung gelten?

Frage für Prof. Schmid

Reicht Danphe als primärer Corpus, oder soll ein zweites unabhängiges C#-System zur externen Validierung ergänzt werden?

Frage für Prof. Schmid

Sind Tabellenoperationen der verpflichtende Kern und Spaltenoperationen eine bedingte Erweiterung?

Frage für Prof. Schmid

Sollen Role-Specific Use Cases nur durch objektive Aufgabenmetriken oder zusätzlich mit realen Nutzerinnen und Nutzern validiert werden?

Frage für Prof. Schmid

Wie viele Rollen sind verbindlich: Developer, Architect und QA/Compliance, oder eine kleinere Menge?

Frage für Prof. Schmid

Soll Neo4j als konkrete Implementierung genannt werden, während das wissenschaftliche Modell speicherunabhängig bleibt?

Frage für Prof. Schmid

Sind LLM-basierte Formulierung, Text-to-Cypher oder Deep-Learning-Reranking ausdrücklich Future Work, optionales Experiment oder Teil der Evaluation?

Frage für Prof. Schmid

Soll die Arbeit zwei oder drei Hauptforschungsfragen enthalten?

Frage für Prof. Schmid

Welcher Anmeldezeitpunkt ist nach Operation und Vorbereitung realistisch und zulässig?

23. Literatur- und Lernressourcen

Dieses Kapitel ordnet die Literatur nicht nach bloßer Ähnlichkeit, sondern nach der Entscheidung, die sie im Projekt unterstützt. Ein Paper wird dann tief gelesen, wenn es eine Forschungsfrage, eine Architekturentscheidung, eine Extraktionsregel, ein Evaluationsdesign oder eine Limitationsaussage trägt.

23.1 Leselogik

Fünf Fragen pro Paper

Problem	Welches präzise Problem adressiert die Arbeit?
Methode	Welche Daten, Repräsentationen und Algorithmen werden verwendet?
Ergebnis	Welche empirischen Befunde werden tatsächlich berichtet?
Grenze	Was kann aus den Daten nicht geschlossen werden?
Transfer	Welche konkrete Entscheidung in diesem Projekt wird dadurch verändert oder begründet?

23.2 Prioritätsgruppen

Priorität	Bedeutung	Leseumfang
★★★★★	Kernquelle	vollständig lesen, Abbildungen und Methodik nacherzählen
★★★★☆	sehr wichtig	Abstract, Methode, Evaluation, Limitations vollständig
★★★☆☆	unterstützend	gezielte Kapitel
★★☆☆☆	optional	nur für Related Work oder Future Work
☆☆☆☆☆	nicht im aktuellen Scope	nur archivieren

23.3 Wissenschaftliches Notizformat

Paper-ID:
 Vollständige Quelle:
 Projektentscheidung, die dieses Paper stützt:
 Problem:
 Methode:
 Datensatz/Corpus:
 Metriken:
 Hauptergebnis:
 Limitations:
 Eigene Erklärung in 5 Sätzen:
 Offene Frage:

24. Präsentationskern

Kapitelorientierung - Schritt für Schritt

Thema	Wie das Projekt mündlich erklärt wird
Problem	Viele Details können den Hauptbeitrag verdecken.
Lösung	Problem, Evidence Gap, Architektur, Evaluation und Beitrag werden auf einen kurzen Kern reduziert.
Ergebnis	Ein verständlicher Vortrag.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Für die mündliche Erklärung und die Verteidigung wird ein Kern definiert, der ohne Vorlage frei erklärbar sein muss: das praktische Problem, die Forschungslücke, die Trennung von RQ1 und RQ2, das Evidenzmodell mit seinen drei Ebenen, die Antwortbarkeitszustände, der Aufbau der Evaluation sowie die bewusst gesetzten Grenzen der statischen Analyse. Ziel ist, jede wissenschaftliche Entscheidung begründen und nicht nur beschreiben zu können.

Anhang A - Relationstypen des Evidenzmodells

Relationstyp	Bedeutung	Typische Evidenz
READ	Belegter Lesezugriff auf ein Datenbankelement.	LINQ-Terminaloperation, DbSet-Abfrage, Find-Operation, SQL-SELECT.
WRITE	Belegter Schreibzugriff (Mutation und Persistenz).	Add/Update/Remove plus SaveChanges bzw. Persistenztrigger.
INVOKES	Aufrufbeziehung zwischen Methoden.	Semantisch aufgelöster Methodenaufruf.
MAPS_TO	Zuordnung von Entity/Property zu Tabelle/Spalte.	Attribute, Fluent API, dokumentierte Konvention.
MUTATES	Veränderung eines Objektzustands.	Property-Änderung, Add/Update/Remove ohne Persistenztrigger.
PERSISTS	Persistierung mutierter Daten.	SaveChanges oder entsprechende Persistenzoperation.
SUPPORTED_BY	Verknüpfung einer Aussage mit ihrer Evidenz.	Beobachtung oder abgeleitete Relation mit Quellort.

Anhang B - Prüf-Checkliste für Evidenz und Antwortbarkeit

- Ist jede zentrale Aussage in einen überprüfbaren Fingerprint übersetzt?
- Sind Beobachtung, abgeleitete Relation und Antwort klar getrennt?
- Besitzt jede Pfadkante eine direkte Beobachtung oder eine dokumentierte Ableitungsregel?
- Ist der Persistenztrigger für WRITE-Aussagen belegt?
- Werden Spaltenaussagen nur bei ausreichender Mapping-Evidenz berichtet?
- Wird bei fehlender Evidenz korrekt PARTIALLY_SUPPORTED oder NOT_ANSWERABLE ausgegeben?
- Ist der Korpus-Snapshot eingefroren und über Commit-Hash reproduzierbar?
- Sind negative und nicht beantwortbare Fälle Teil des Goldstandards?

25. Betreuungshinweise und Bedeutung der Use Cases

Kapitelorientierung - Schritt für Schritt

Thema	Wie Betreuung und Use Cases in Designentscheidungen übersetzt werden
Problem	Use Cases dürfen nicht nur Beispiele sein, sondern müssen Anforderungen und Evaluation strukturieren.
Lösung	Rollen, Fragen, Evidenzbedarf und Ausgabeformat werden operationalisiert.
Ergebnis	Role-specific use cases als verbindlicher Projektbestandteil.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

25.1 Umsetzung der Hinweise der Betreuung

Diese ausführliche Arbeitsfassung berücksichtigt die Anforderungen, die für das Exposé formuliert wurden: einen klaren Titel, eine nachvollziehbare Motivation, eine Einordnung in verwandte Arbeiten mit expliziter Forschungslücke, konkrete Forschungsfragen sowie einen realistischen Arbeits- und Zeitplan. Der Titel benennt Gegenstand, Methode und Ziel der Arbeit: die nachvollziehbare Extraktion und graphbasierte Retrieval-Analyse von Datenbankoperationen in heterogenen C#- und SQL-Artefakten. Das Related Work wird nicht als bloße Liste ähnlicher Systeme verstanden, sondern zeigt, welche Bausteine bereits existieren, welche Annahmen sie treffen und an welcher Stelle Cross-Repository Code Intelligence einen engeren, empirisch prüfbaren Beitrag leistet. Die Forschungsfragen trennen Extraktion und Retrieval, damit Fehlerquellen und Beiträge eindeutig zugeordnet werden können. Die Literatúrauswahl und die endgültige Schwerpunktsetzung werden mit der Betreuung abgestimmt (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

25.2 Bedeutung der Use Cases

Use Cases sind für diese Arbeit notwendig, weil die wissenschaftliche Evaluation nicht bei abstrakten Graphknoten oder technischen Detektoren stehen bleiben darf. Ein Use Case verbindet eine realistische Nutzerfrage mit einer definierten Evidenzanforderung, einem Retrievalpfad, einem Antwortstatus und einer überprüfbaren Goldantwort. Er legt damit fest, wofür das Artefakt praktisch eingesetzt werden soll und welche Leistung tatsächlich gemessen wird (vgl. Yamaguchi et al., 2014; Tao et al., 2025).

Die Use Cases erfüllen fünf Funktionen: Erstens konkretisieren sie das praktische Problem. Zweitens begrenzen sie den Untersuchungsumfang. Drittens helfen sie bei der Auswahl des Goldstandards. Viertens strukturieren sie die Retrieval- und Antwortbarkeitsregeln. Fünftens ermöglichen sie eine verständliche Demonstration gegenüber der Betreuung und in der Verteidigung (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Personas können zur verständlichen Darstellung genutzt werden, dürfen aber nicht mit dem experimentellen Design verwechselt werden. Für die Kern-Evaluation sind primär typisierte Use Cases mit klaren Ein- und Ausgaben vorgesehen; Personas können diese Use Cases illustrieren, ohne selbst als Evaluationskategorie zu dienen (vgl. Olea et al., 2024).

Didaktische Erklärung - einfaches Deutsch

Die Use Cases übersetzen das abstrakte Forschungsthema in konkrete Aufgaben. Jeder Use Case legt fest, wer fragt, welche Evidenz benötigt wird, welches Ergebnis erwartet wird und wann das System nicht antworten darf.

26. Verbindliche Use Cases für Entwicklung und Evaluation

Kapitelorientierung - Schritt für Schritt

Thema	Welche Aufgaben das System tatsächlich beherrschen soll
Problem	Ein universelles Code-QA-System wäre nicht realistisch evaluierbar.
Lösung	Eine begrenzte Menge von tabellen-, spalten-, pfad- und rollenbezogenen Fragen wird festgelegt.
Ergebnis	Konkrete Tests und Gold-Antworten.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

26.1 UC1 - Methoden finden, die eine Tabelle oder Spalte lesen

- Akteur: Entwicklerin oder Wartungsingenieurin.
- Ausgangsfrage: Welche Methoden lesen eine bestimmte Tabelle oder, soweit statisch belegbar, eine bestimmte Spalte?
- Beispielhafte Frage: Welche Methoden lesen Patient.DateOfBirth?
- Erforderliche Evidenz: Methodenaufruf oder LINQ-Ausdruck; semantisch aufgelöstes Symbol; DbSet- oder Kontextzuordnung; Entity-zu-Tabelle-Mapping; gegebenenfalls Property-zu-Spalte-Mapping; genaue Datei- und Positionsangaben.
- Erwartetes Ergebnis: Eine Liste unterstützter Methoden mit READ-Relation, Evidenzpfad, Quellorten und Status. Ist die Spaltenzuordnung nicht belegbar, darf das System höchstens eine tabellenbezogene oder teilweise unterstützte Aussage liefern.
- Negativfall: Ein bloßer Namensfund in Kommentar, Testdatei, DTO oder nicht gemappter Property darf nicht als Datenbankzugriff bewertet werden.

26.2 UC2 - Methoden und Pfade finden, die Daten verändern

- Akteur: Entwicklerin, Architektin oder Reviewer.
- Ausgangsfrage: Welche Methoden oder Codepfade können eine Tabelle oder Spalte verändern?
- Erforderliche Evidenz: Mutationssignal wie Add, Update, Remove oder Property-Änderung; Persistenzsignal wie SaveChanges; Mapping zum Datenbankziel; gegebenenfalls aufgelöste Hilfsmethoden- oder Aufrufkette.
- Erwartetes Ergebnis: Unterstützte WRITE-Relationen mit Trennung zwischen Mutationssignal und Persistenztrigger. Das Ergebnis beschreibt eine statisch unterstützte Möglichkeit innerhalb des Analyseumfangs, nicht den Beweis, dass die Operation zur Laufzeit tatsächlich ausgeführt wurde.
- Negativfall: Das Erzeugen oder Ändern eines reinen In-Memory-Objekts ohne belegten Persistenzpfad darf nicht automatisch als Datenbank-WRITE gelten.

26.3 UC3 - Zugriffsmechanismus bestimmen

- Akteur: Architektin oder Modernisierungsverantwortliche.
- Ausgangsfrage: Erfolgt der Zugriff über Entity Framework, ADO.NET, eingebettetes SQL oder eine Stored Procedure?
- Erforderliche Evidenz: Technologienaher Detektor; aufgelöster API-Aufruf; SQL- oder Prozedurbezug; Mapping- und Provenienzinformationen.

- Erwartetes Ergebnis: Klassifikation des Zugriffsmechanismus mit Belegen und Unsicherheitsstatus.
- Negativfall: Ein importierter Namespace oder ein vorhandenes Paket allein beweist nicht, dass ein konkreter Datenbankzugriff über diese Technologie erfolgt.

26.4 UC4 - Vollständigen Evidenzpfad erklären

- Akteur: Senior-Architektin, Reviewer oder Betreuerin.
- Ausgangsfrage: Durch welche belegten Schritte verbindet das System eine C#-Methode mit einer Datenbanktabelle oder -spalte?
- Erforderliche Evidenz: Einstiegsmethode; Aufrufkette; Operation; Entity- oder DbSet-Bezug; Mapping; Datenbankziel; direkte Beobachtungen; dokumentierte Ableitungsregeln.
- Erwartetes Ergebnis: Ein prüfbarer Pfad, beispielsweise Methode → Invocation → DbSet → Entity → Tabelle → READ oder WRITE, wobei jede Kante entweder eine direkte Beobachtung oder eine explizite Ableitung besitzt.
- Negativfall: Eine Kette mit einer unbelegten Kante darf nicht als vollständig unterstützt erscheinen.

26.5 UC5 - Fehlende Evidenz und begründete Verweigerung

- Akteur: Jede Nutzerrolle.
- Ausgangsfrage: Kann eine konkrete Aussage anhand des statischen Korpus sicher beantwortet werden?
- Erforderliche Evidenz: Frageabhängige Muss-Bedingungen und eine Prüfung, ob alle erforderlichen Pfadteile vorhanden, widerspruchsfrei und innerhalb des Scopes auflösbar sind.
- Erwartetes Ergebnis: SUPPORTED, PARTIALLY_SUPPORTED oder NOT_ANSWERABLE. Bei teilweiser Unterstützung oder Verweigerung nennt das System exakt die fehlende Zuordnung, das ungelöste Symbol, die dynamische Konstruktion oder den nicht zugänglichen Artefakteil.
- Negativfall: Das System darf eine plausible Vermutung nicht als bestätigte Antwort formulieren.

26.6 UC6 - Cross-Repository-/Cross-Repository/Cross-Project- oder Cross-Database-Pfad untersuchen

- Akteur: Architektin oder Maintainerin eines modularen Systems.
- Ausgangsfrage: Führt eine Methode über Projekt- oder Datenbankgrenzen zu einem belegbaren Datenbankzugriff?
- Voraussetzung: Dieser Use Case wird nur verbindlich evaluiert, wenn entsprechende Fälle im eingefrorenen Korpus tatsächlich vorkommen und mit vertretbarem Aufwand annotierbar sind.
- Erforderliche Evidenz: Projektzugehörigkeit; Referenz- oder Aufrufbeziehung; Datenkontext; Datenbankidentität; stabile Quellprovenienz über die Grenze hinweg.
- Erwartetes Ergebnis: Ein grenzüberschreitender Evidenzpfad oder eine begründete Feststellung, dass die Grenze im statischen Umfang nicht auflösbar ist.

26.7 UC7 - Negative Kontrolle gegen Namensähnlichkeit

- Akteur: Evaluatorin.
- Ausgangsfrage: Kann Cross-Repository Code Intelligence einen semantisch relevanten Datenbankzugriff von bloßer textueller Ähnlichkeit unterscheiden?
- Erforderliche Evidenz: Hard-Negative-Beispiele wie gleichnamige Properties, Kommentare, Testdaten, DTOs, Aliasnamen, generierter Code oder [NotMapped]-Properties.
- Erwartetes Ergebnis: Die semantischen Konfigurationen sollen weniger False Positives erzeugen als die Textbaseline. Fehler werden nicht nur gezählt, sondern nach Ursache klassifiziert, zum Beispiel Namensgleichheit ohne Mapping, Kommentare ohne Operation oder Tests ohne Persistenzpfad.

Didaktische Erklärung - einfaches Deutsch

Dieser Abschnitt beschreibt die wissenschaftliche Denkweise des Projekts. Jeder Claim braucht eine überprüfbare Quelle, jede Ableitung eine dokumentierte Regel und jede Antwort einen vollständigen Evidenzpfad.

27. Evidenzprinzipien und methodische Disziplin

Kapitelorientierung - Schritt für Schritt

Thema	Welche Regeln jede Systemaussage erfüllen muss
Problem	Ohne Evidence Contract können Vermutung, Retrieval-Treffer und Beweis verwechselt werden.
Lösung	Jeder Claim benötigt Typ, Provenienz, Source Location und Status.
Ergebnis	Reproduzierbare und überprüfbare Antworten.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

27.1 Jede Behauptung wird zu einer überprüfbaren Suchaufgabe

Jede Aussage über das Korpus muss in überprüfbare Fingerprints übersetzt werden. Die Aussage „das Projekt verwendet C#“ kann durch Dateitypen, Projektdateien und Builds geprüft werden. Die Aussage „Entity Framework wird verwendet“ benötigt konkrete DbContext-, DbSet-, Mapping- oder API-Belege. Die Aussage „ADO.NET wird verwendet“ benötigt konkrete Typ- oder Symbolbelege wie SqlConnection oder SqlCommand. Die Aussage „eine Methode schreibt eine Tabelle“ benötigt einen vollständigen oder klar begrenzten Evidenzpfad und kann nicht allein aus einem Namen abgeleitet werden (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Dieses Prinzip verbindet Literaturarbeit, Corpus Census, Implementierung und Evaluation. Es verhindert, dass Annahmen aus alten Notizen, Demonstrationsdaten oder synthetischen Beispielen unbemerkt zu Fakten des Danphe-Korpus werden.

27.2 Drei Evidenzebenen

- **Beobachtung (Observation):** Eine direkt aus einem konkreten Quellort extrahierte, typisierte Beobachtung.
- **Abgeleitete Relation (Derived Relation):** Eine Beziehung, die aus einer oder mehreren Beobachtungen durch eine dokumentierte Regel entsteht.
- **Antwort (Answer):** Eine Schlussfolgerung, die nur ausgegeben wird, wenn die für die konkrete Frage definierten Evidenzanforderungen vollständig und gültig sind.

Diese Trennung ist verbindlich. Ein Detektortreffer ist noch keine Datenbankrelation, und eine Datenbankrelation ist noch nicht automatisch eine vollständige Antwort auf eine Nutzerfrage (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

27.3 Speicherunabhängiges wissenschaftliches Modell

Der Code-Database Evidence Graph ist die wissenschaftliche Repräsentation für heterogene, mehrstufige und provenienzerhaltende Beziehungen. Ein konkreter Graphspeicher ist lediglich eine austauschbare Implementierung. Das kanonische Evidenzmodell bleibt unabhängig vom Speicher: JSON oder JSONL können für reproduzierbare Rohdaten, SQLite für Auswertungs- und Experimentprotokolle und ein Graphspeicher für Pfadabfragen eingesetzt werden. Keine dieser Technologien ist allein der wissenschaftliche Beitrag (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

27.4 Kategorische Unsicherheit statt unbegründeter Prozentwerte

Die primären Zustände lauten SUPPORTED, PARTIALLY_SUPPORTED und NOT_ANSWERABLE. Numerische Konfidenzwerte werden nur dann verwendet, wenn eine gesonderte Kalibrierung mit gelabelten Daten geplant und ausgewertet wird. Ohne eine solche Studie könnten Prozentwerte eine nicht vorhandene Genauigkeit vortäuschen.

Didaktische Erklärung - einfaches Deutsch

READ und WRITE müssen präzise definiert werden, damit ein Detektor nicht jede ähnliche Codezeile falsch bewertet. Wichtig ist besonders die Trennung zwischen Vorbereitung und tatsächlicher Ausführung einer Abfrage sowie zwischen Änderung im Speicher und belegter Persistenz.

28. Erweiterte Operationalisierung von READ und WRITE

Kapitelorientierung - Schritt für Schritt

Thema	Wie Datenbankoperationen formal erkannt werden
Problem	Framework-Methodennamen allein reichen nicht, um READ oder WRITE sicher zu klassifizieren.
Lösung	Operationen werden über Mutationen, Materialisierung, Mapping und Persistenzereignisse definiert.
Ergebnis	Explizite Regeln für Tabellen- und Spaltenebene.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

28.1 READ

Ein READ liegt im statischen Modell vor, wenn eine Methode einen belegten Pfad zu einer Datenabfrage besitzt. Dazu gehören beispielsweise semantisch aufgelöste LINQ-Terminaloperationen, DbSet-Abfragen, Find-Operationen oder analysierbare SQL-SELECT-Anweisungen. Query-Konstruktion und Query-Ausführung werden getrennt repräsentiert. Ein IQueryable-Ausdruck kann vorbereitet werden, ohne dass seine Ausführung an derselben Stelle belegt ist (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

28.2 WRITE

Ein WRITE liegt im statischen Modell vor, wenn ein belegter Mutationspfad und, sofern für das verwendete Framework erforderlich, ein Persistenztrigger vorliegen. Add, Update, Remove oder Property-Änderungen werden deshalb von SaveChanges beziehungsweise einer entsprechenden Persistenzoperation getrennt modelliert. Erst die Kombination aus Mutationssignal, Kontext, Mapping und Persistenztrigger trägt eine WRITE-Relation.

28.3 Tabellen- und Spaltenebene

Die Tabellenebene bildet den verbindlichen Kern. Die Spaltenebene wird nur berichtet, wenn die Zuordnung statisch durch Attribute, Fluent API, Konventionen mit dokumentierter Regel, SQL-Struktur oder andere überprüfbare Evidenz gestützt ist. Bei konventionsbasierter Zuordnung muss der Ableitungsstatus sichtbar bleiben, sodass Nutzerinnen und Nutzer sehen, ob eine Spaltenaussage direkt oder nur konventionsbasiert abgeleitet wurde.

28.4 Laufzeitgrenze

SUPPORTED_STATIC_READ oder SUPPORTED_STATIC_WRITE bedeutet, dass eine Relation innerhalb des dokumentierten statischen Scopes belegt ist. Es bedeutet nicht, dass die betreffende Operation in einem konkreten Lauf tatsächlich ausgeführt wurde. Reflection, dynamisch erzeugtes SQL, Laufzeitkonfiguration, polymorphe Auflösung und externe Dienste können die statische Sicht begrenzen.

Didaktische Erklärung - einfaches Deutsch

Die Evaluation beantwortet die Frage, ob Cross-Repository Code Intelligence wissenschaftlich funktioniert, nicht nur ob eine Demo gut aussieht. Deshalb werden Daten, Regeln, Fragen und Konfigurationen eingefroren und verschiedene Ansätze unter denselben Bedingungen verglichen.

29. Erweiterte Evaluationslogik

Kapitelorientierung - Schritt für Schritt

Thema	Wie Pilot, Entwicklung und Test getrennt werden
Problem	Wenn dieselben Beispiele zum Entwickeln und Bewerten genutzt werden, entsteht Optimismus-Bias.
Lösung	Pilot- und Testmenge, Baselines, Ablationen und Error Taxonomy werden getrennt.
Ergebnis	Glaubwürdigere Ergebnisse.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

29.1 Entwicklungs-, Pilot- und Testtrennung

Regeln und Detektoren dürfen nicht wiederholt anhand derselben finalen Testfälle optimiert werden. Das Korpus beziehungsweise das annotierte Subset wird deshalb in Entwicklungs-, Pilot- und finale Testanteile getrennt, soweit Umfang und Korpusstruktur dies zulassen. Die endgültigen Regeln, Konfigurationen, Fragen und Gold-Items werden vor dem finalen Experiment gemeinsam eingefroren (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

29.2 Extraktionsvergleich

Die Extraktionsstufe vergleicht mindestens eine Text- oder Namensbaseline, eine syntaxbasierte Roslyn-Konfiguration und eine semantische, mappingbewusste Konfiguration. Optionale heterogene Erweiterungen - etwa SQL-Dateien oder Stored Procedures - werden nur aufgenommen, wenn der Corpus Census ausreichende Fälle und die Pilotstudie eine belastbare Umsetzung zeigen (vgl. Microsoft, 2026a; Shatnawi et al., 2019).

29.3 Retrievalvergleich

Die Retrievalstufe vergleicht dieselben eingefrorenen Fragen über unterschiedliche Repräsentationen. Mindestens erforderlich sind eine Textbaseline, eine flache Suche über normalisierte Evidenzdatensätze und ein graphbasierter Pfadansatz. Eine zusätzliche Konfiguration mit expliziten Antwort- und Verweigerungsregeln prüft den Nutzen der Antwortbarkeitslogik (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

29.4 Goldstandard und Annotation

Ein Gold-Item enthält Subjektmethode, Operationstyp, Zieldatenbankelement, Zugriffsmechanismus, erforderlichen Evidenzpfad, Quellorte, Direktheitsstatus, Antwortstatus und gegebenenfalls eine dokumentierte ungelöste Stelle. Negative und unbeantwortbare Fälle sind gleichwertiger Bestandteil des Goldstandards. Die Annotation wird nach schriftlichen Regeln erstellt, pilotiert, überarbeitet und vor dem finalen Test eingefroren (vgl. Peng et al., 2026; Chen et al., 2025; Du et al., 2025).

29.5 Metriken

Für RQ1 werden Precision, Recall, F1, Rohzahlen und Kategorieergebnisse berichtet. Für RQ2 werden Antwortkorrektheit, Evidenzvollständigkeit, Evidenzpräzision, Vollständigkeit der Nachvollziehbarkeit, Pfadgültigkeit, Coverage, Rate korrekter Verweigerungen, Rate falscher Verweigerungen und Rate nicht

gestützter Antworten verwendet. Eine einzelne Gesamtzahl reicht nicht aus, weil sie schwierige oder seltene Kategorien verdecken kann (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

29.6 Fehleranalyse

Die qualitative Fehleranalyse unterscheidet mindestens ungelöste Symbole, Aliasprobleme, indirekte oder konventionsbasierte Mappings, Hilfsmethoden, interprozedurale Lücken, dynamisches SQL, Stored-Procedure-Grenzen, nicht unterstützte ORM-Muster, generierten Code, mehrdeutige READ/WRITE-Klassifikation, ausgelassene Retrievalpfade und falsche Antwortbarkeitsentscheidungen (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Didaktische Erklärung - einfaches Deutsch

Dieser Plan verbindet Lernen, Forschung, Implementierung, Evaluation und Schreiben. Er ist bewusst ausführlich, damit jede Phase ein sichtbares Ergebnis erzeugt und du jederzeit erklären kannst, warum eine Aufgabe gerade notwendig ist.

30. Integrierter Arbeits-, Lern- und Sprachplan

Planungsprinzip

Dieses Kapitel erzeugt keine zusätzliche tägliche Vollplanung. Für den Kurs vom 17. August bis 17. September gilt nur: Kurstag = verstehen und entwerfen; Folgetag = genau ein Modul bauen, testen und erklären.

Sitzung	Kurstag	Folgetag: genau bauen	Projektproblem
1	Preprocessing / Tokenization	Code-aware Tokenizer	Technische Identifier erhalten und normalisieren.
2	BoW / TF-IDF	TF-IDF Repository Retriever	Reproduzierbare lexikalische Candidate-Baseline.
3	Cosine Similarity	Ranking und Retrieval Metrics	Top-k, Recall@k und MRR messen.
4	Word2Vec / FastText	Identifier Embedding Search	Vocabulary mismatch als Kandidatenproblem untersuchen.
5	RNN	Evidence Sequence Classifier	Nur Course Experiment; gegen Regeln vergleichen.
6	LSTM / GRU	Evidence Status Comparison	Nur bei ausreichenden Daten.
7	Seq2Seq	Role-specific NL-to-Structured Query	Question Contract als JSON erzeugen.
8	Transformer / Attention	Evidence Path Attention Experiment	Erklärendes Experiment, keine Beweisquelle.
9	BERT / GPT / Prompting	CodeBERT Retriever + Agent Output Adapter	Neural Baseline und einheitliches Evidence-Format.
10	RAG / LoRA / Evaluation	Graph-grounded Answer Pipeline	End-to-End-Demo mit Status und Quellen.

30.1 Exakte Sprachautomatisierung pro Sitzung

Für jedes Modul wird dieselbe kurze, überprüfbare Sequenz verwendet. Sie dauert 25 bis 35 Minuten und ersetzt keine technische Arbeit.

Englisch

- | | |
|---|--|
| 1 | Schreibe genau fünf Sätze: problem, input, method, output, limitation. |
| 2 | Sprich diese fünf Sätze ohne Ablesen in 90 Sekunden. |
| 3 | Beantworte exakt die Frage: Why is this module useful but not sufficient evidence? |

Deutsch

- | | |
|---|---|
| 1 | Schreibe genau fünf Sätze mit denselben Rollen. |
| 2 | Erkläre den Modulfluss in 90 Sekunden ohne Volltext. |
| 3 | Beantworte exakt die Frage: Woran erkennt man, dass ein Ergebnis nur Kandidat und noch kein Beweis ist? |

31. Konsolidierte Abgrenzung und Entscheidungsstand

Kapitelorientierung - Schritt für Schritt

Thema	Welche Entscheidungen aktuell gelten
Problem	Viele frühere Varianten können Scope und Terminologie verwirren.
Lösung	Aktuelle Core-Entscheidungen werden an einer Stelle zusammengeführt.
Ergebnis	Eine verlässliche Arbeitsgrundlage.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Verbindlich sind Roslyn-basierte C#-Analyse, tatsächlich im Korpus vorkommende Entity-Framework-Muster, READ- und WRITE-Beziehungen, Provenienz, ein speicherunabhängiges Evidenzmodell, graphbasierter und nicht graphbasierter Vergleich, vordefinierte Use Cases, Antwortbarkeitsregeln, ein Frozen Corpus, ein geprüfter Goldstandard und eine kontrollierte Evaluation (vgl. Microsoft, 2026a; Shatnawi et al., 2019).

Bedingte Erweiterungen sind ausgewählte ADO.NET-Muster, eingebettetes SQL, SQL-Dateien, Stored Procedures, begrenzte interprozedurale Verknüpfung sowie Cross-Repository-/Cross-Repository/Cross-Project- und Cross-Database-Fälle, sofern sie im eingefrorenen Korpus tatsächlich vorkommen und wissenschaftlich evaluierbar sind (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Nicht verbindlich sind GraphRAG, LLM-basierte Antworten, MCP, Multi-Agent-Systeme, eine produktionsreife Benutzeroberfläche, universelle natürliche Sprache, vollständige Laufzeitverifikation, Unterstützung aller ORMs oder SQL-Dialekte und allgemeine Compliance-Aussagen. Diese Themen bleiben Backlog oder zukünftige Arbeit, solange keine dokumentierte Entscheidung den Scope ändert (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

Didaktische Erklärung - einfaches Deutsch

Dieser Abschnitt sammelt Fragen und Entscheidungen, die vor der endgültigen Festlegung mit der Betreuung geklärt werden müssen. Die Liste zeigt nicht Unsicherheit im negativen Sinn, sondern wissenschaftliche Sorgfalt: offene Punkte werden sichtbar gemacht, statt heimlich als feststehend behandelt zu werden.

32. Prüf- und Gesprächsfragen für die Betreuung

Problem

Frage Welche konkrete Information kann ein allgemeiner Coding Agent liefern, die dieses Artefakt bewusst nicht als wissenschaftlichen Beweis akzeptiert?

Eigene Antwort _____

Graph

Frage Warum wird eine eigene DatabaseOperation- oder EvidenceRecord-Ebene benötigt, statt nur Method-Writes_To-Table-Kanten zu speichern?

Eigene Antwort _____

Cross-Repository

Frage Wie werden Symbole, Projekte und Repositories global identifiziert und wie werden unsichere Cross-Boundary-Verbindungen markiert?

Eigene Antwort _____

Evaluation

Frage Welche Unit of Analysis wird für Precision und Recall verwendet?

Eigene Antwort _____

Answerability

Frage Wann ist eine Verweigerung korrekt und wann ist sie ein False Refusal?

Eigene Antwort _____

Use Cases

Frage Worin unterscheiden sich Developer-, Architect- und QA-Ausgaben objektiv?

Eigene Antwort _____

Scope

Frage Welche Funktion wird zuerst gestrichen, wenn Zeit oder Corpus die Planung gefährdet?

Eigene Antwort _____

Contribution

Frage Welche neuen Erkenntnisse bleiben übrig, selbst wenn der Prototyp technisch nur mittelmäßige Recall-Werte erreicht?

Eigene

Antwort —

33. Retrieval- und Antwortlogik

Kapitelorientierung - Schritt für Schritt

Thema	Wie aus einer Frage eine evidenzbasierte Antwort wird
Problem	Relevante Snippets allein garantieren weder Vollständigkeit noch Richtigkeit.
Lösung	Question Contract, Retrieval, Graph Traversal und Verifier werden gestuft ausgeführt.
Ergebnis	Traceable answers oder begründete Verweigerung.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

33.1 Retrieval als eigener Untersuchungsgegenstand

Retrieval bedeutet hier nicht nur, Textstellen zu finden. Es bedeutet, für eine festgelegte Frage genau die Evidenzelemente und Pfade auszuwählen, die zur Beantwortung erforderlich sind. RepoCoder zeigt, dass repositoryweiter Kontext durch Retrieval gezielt nutzbar gemacht werden kann, zugleich aber stark von der Qualität der Retrievalanfrage und des ausgewählten Kontexts abhängt (Zhang et al., 2023). Knowledge-Graph-gestütztes Repository-QA zeigt ebenfalls, dass mehrstufige Fragen über mehrere Beziehungen anspruchsvoll bleiben und fehlerhafte Graphabfragen oder falsche Annahmen zu nicht gestützten Antworten führen können (Abedu et al., 2025). Deshalb wird Retrieval in Cross-Repository Code Intelligence getrennt von der Extraktion evaluiert und mit expliziter Antwortbarkeitsprüfung verbunden.

33.2 Retrievalvarianten

- Textretrieval sucht nach Namen und Textmustern und dient als schwache, aber verständliche Baseline.
- Symbolbewusstes Retrieval nutzt semantisch aufgelöste Programmelemente, ohne zwingend mehrstufige Graphpfade zu verwenden.
- Flaches Evidenzretrieval filtert normalisierte Evidenzdatensätze nach Typen, IDs und Eigenschaften.
- Graphbasiertes Retrieval traversiert belegte Beziehungen und gibt vollständige Pfade zurück.
- Antwortbarkeitsbewusstes Retrieval ergänzt feste Muss-Bedingungen und verweigert unvollständige Schlussfolgerungen.

33.3 Fragevertrag

Jede Evaluationsfrage erhält einen Fragevertrag mit Zieltyp, erlaubtem Scope, erforderlichen Relationstypen, Mindestprovenienz, akzeptierten Unsicherheitszuständen und Regeln für Teilantwort oder Verweigerung. Dadurch wird verhindert, dass die Antwortqualität erst nach Sichtung der Resultate beliebig bewertet wird (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

Beispiel: Die Frage „Welche Methoden schreiben Tabelle T?“ verlangt mindestens Methode, Mutation, Persistenztrigger, Entity- oder SQL-Ziel, Mapping zu T und gültige Provenienz. Fehlt der Persistenztrigger, ist eine vollständige WRITE-Antwort nicht zulässig (vgl. Lewis et al., 2020; Zhang et al., 2023; Gandhi et al., 2025).

34. Architektur- und Forschungsentscheidungen

Kapitelorientierung - Schritt für Schritt

Thema	Warum die Architektur so gewählt wird
Problem	Technologieentscheidungen können stillschweigend zu wissenschaftlichen Annahmen werden.
Lösung	Jede Entscheidung wird mit Ziel, Alternative, Risiko und Evaluation dokumentiert.
Ergebnis	Nachvollziehbares Designwissen.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

- AD-01 - Statische Analyse statt Laufzeitinstrumentierung: Die Arbeit untersucht rekonstruierbare Evidenz aus Artefakten. Laufzeitverhalten bleibt eine Grenze und wird nicht vorgetäuscht. Cross-Repository Code Intelligence zielt bewusst auf statisch belegbare READ- und WRITE-Beziehungen und macht Grenzen der statischen Sicht explizit sichtbar, statt sie mit Laufzeitannahmen zu überdecken.
- AD-02 - Roslyn für C#: Roslyn ermöglicht Syntax-, Symbol- und Semantikanalyse im .NET-Ökosystem. Eine reine Text- oder Regex-Lösung reicht für symbolische Zuordnung nicht aus. Die Wahl von Roslyn stellt sicher, dass Methoden, Properties, Typen und DbContext-/DbSet-Strukturen präzise aufgelöst und als Grundlage für Evidenzdetektoren genutzt werden können.
- AD-03 - Entity Framework als verbindlicher Kern: Entity Framework ist im vorgesehenen Korpus zentral und erlaubt einen realistischen Bachelorumfang. Weitere Mechanismen wie ADO.NET oder Stored Procedures bleiben bedingte Erweiterungen. Dadurch konzentriert sich der Pflichtumfang auf ein klar definiertes ORM, während zusätzliche Technologien nur dann einbezogen werden, wenn sie im Snapshot tatsächlich vorkommen und evaluierbar sind.
- AD-04 - Code-Database Evidence Graph als wissenschaftliche Repräsentation: Mehrstufige, heterogene und provenienzerhaltende Beziehungen werden explizit modelliert. Der Code-Database Evidence Graph ist damit die zentrale wissenschaftliche Struktur, in der Beobachtungen, abgeleitete Relationen und Antworten miteinander verknüpft werden. Er bildet die Grundlage für graphbasierte Retrievalstrategien und für die Messung von Pfadgültigkeit und Nachvollziehbarkeit.
- AD-05 - Keine Bindung an eine spezifische Graphdatenbank: Speichertechnologie ist austauschbar und wird erst durch Prototypvergleich begründet. Das kanonische Evidenzmodell bleibt unabhängig von der konkreten Implementierung (zum Beispiel Neo4j, In-Memory-Graph oder andere Lösungen). Entscheidend ist die wissenschaftliche Repräsentation, nicht das einzelne Produkt.
- AD-06 - Frozen Corpus: Ein eingefrorener Snapshot verhindert, dass sich Testgrundlage und Resultate während der Evaluation unbemerkt verändern. Commit-Hash, Archiv und Korpusmanifest sichern, dass alle Experimente auf derselben Codebasis laufen und später reproduzierbar sind. Änderungen am Korpus erzeugen bewusst neue Versionen, statt stillschweigend in die Ergebnisse einzufließen.
- AD-07 - Goldstandard: Systemqualität wird gegen manuell geprüfte Evidenz gemessen, nicht anhand plausibler Demonstrationen. Ein klar definierter Goldstandard mit annotierten Pfaden, Operationstypen, Zielen und Antwortstatus bildet die Referenz für Precision, Recall, Antwortkorrektheit und Verweigerungsverhalten. Demoprojekte oder synthetische Beispiele gelten nicht als Goldstandard.
- AD-08 - Trennung von Extraktion und Retrieval: Diese Trennung lokalisiert Fehler und verhindert, dass eine Stufe die Schwäche der anderen verdeckt. Zuerst wird gemessen, welche Evidenz überhaupt extrahiert wurde (RQ1), dann wird bewertet, wie gut unterschiedliche Retrievalstrategien

diese Evidenz für Antworten nutzen (RQ2). So können Extraktions- und Retrievalprobleme getrennt analysiert und verbessert werden.

- AD-09 - Antwortbarkeit statt Raten: Eine korrekte Verweigerung ist besser als eine plausible, aber unbelegte Antwort. Cross-Repository Code Intelligence soll Fragen nur dann beantworten, wenn die dafür definierten Muss-Bedingungen im Evidenzpfad erfüllt sind. Fehlen wesentliche Zuordnungen oder bleiben kritische Symbole ungelöst, ist eine begründete Verweigerung das gewünschte Systemverhalten.
- AD-10 - Kategorien statt unbegründeter Prozentwerte: SUPPORTED, PARTIALLY_SUPPORTED und NOT_ANSWERABLE sind verbindliche Antwortzustände. Numerische Konfidenzwerte erfordern eine eigene Kalibrierungsstudie mit gelabelten Daten. Ohne eine solche Studie würden Prozentangaben eine Genauigkeit suggerieren, die empirisch nicht abgesichert ist. Daher bleibt die Unsicherheit kategorisch und erklärbar.
- AD-11 - Tabellenebene als Kern: Spaltenaussagen werden nur bei ausreichender Mapping-Evidenz berichtet. Die Tabellenebene ist der feste Pflichtumfang; Spaltenebene ist optional und an klare Zuordnungsregeln gebunden (zum Beispiel Attribute, Fluent API oder überprüfbare Konventionen). Damit wird vermieden, dass unsichere Spaltenzuordnungen als gesicherte Fakten erscheinen.
- AD-12 - Ein realer Vertical Slice vor Breitenentwicklung: Zuerst wird ein vollständiger, reproduzierbarer Pfad umgesetzt - vom C#-Code über EF-Mapping bis zur READ/WRITE-Antwort - bevor zahlreiche Detektoren parallel entwickelt werden. Dieser Vertical Slice dient als Referenz für Architektur, Evidenzmodell, Graphkonstruktion und Evaluation und verhindert, dass sich das Projekt in zu vielen unvollständigen Teilbausteinen verliert.

35. Integrierte Lernstruktur für jeden Fachabschnitt

Kapitelorientierung - Schritt für Schritt

Thema	Wie das Dokument aktiv gelernt wird
Problem	Lesen allein führt nicht automatisch zu erklärbarem Wissen.
Lösung	Jeder Abschnitt endet mit Recall-Frage, Mini-Erklärung, Beispiel und Transferaufgabe.
Ergebnis	Mündliche und schriftliche Abrufbarkeit.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Diese ausführliche Fassung ist nicht nur ein wissenschaftlicher Plan, sondern auch eine Lernunterlage. Jeder zentrale Abschnitt soll daher nach demselben Muster bearbeitet werden:

- **Wissenschaftliche Aussage:** Was behauptet oder definiert der Abschnitt präzise? Welche zentrale These, Definition oder Regel wird formuliert?
- **Einfache Erklärung:** Wie lässt sich dieselbe Idee ohne Fachjargon erklären? Ziel ist eine Version, die spontan und verständlich weitergegeben werden kann.
- **Bedeutung:** Warum ist dieser Punkt für RQ1, RQ2 oder den Prototyp wichtig? Hier wird die Rolle des Abschnitts im Forschungsdesign klar gemacht.
- **Konkretes Beispiel:** Wie zeigt sich der Punkt an einem realen oder eindeutig synthetisch markierten Danphe-Fall? Beispiele sollen den Zusammenhang zwischen Theorie und Codebasis sichtbar machen.
- **Gegenbeispiel:** Welcher ähnliche Fall darf nicht gleich bewertet werden? Dadurch werden Grenzen der Definition und typische Verwechslungen deutlich.
- **Typischer Fehler:** Welche falsche Schlussfolgerung liegt nahe? Hier werden Missverständnisse gesammelt, die in Implementierung, Annotation oder Präsentation vermieden werden sollen.
- **Merksatz:** Welcher kurze Satz hält den Kern fest? Der Merksatz soll sich leicht einprägen lassen und für die mündliche Erklärung helfen.
- **Betreuungsfrage:** Welche kritische Rückfrage könnte die Betreuung stellen? So wird vorbereitet, wissenschaftliche Entscheidungen zu begründen, statt sie nur zu beschreiben.
- **Mündliche Erklärung:** Kann der Inhalt in zwei Minuten ohne Vorlage erklärt werden? Wenn nicht, muss die Erklärung noch vereinfacht und geübt werden.
- **Lernnachweis:** Welche Datei, Tabelle, Skizze, Testausgabe oder Aufnahme zeigt die tatsächliche Beherrschung? Jeder Abschnitt soll einen konkreten Output haben, der zeigt, dass Wissen angewendet wurde, nicht nur gelesen.

Diese Struktur ersetzt keine wissenschaftliche Argumentation. Sie ergänzt sie so, dass das Dokument zunächst vollständig verstanden und später in eine kurze offizielle Exposéfassung überführt werden kann.

Abschließender Merksatz: Zuerst wird das Vorhaben vollständig verstanden und wissenschaftlich konsistent gemacht. Erst danach wird daraus eine kurze, formale Fassung für die Betreuung erstellt (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

36. Verbindliche Terminologie

Kapitelorientierung - Schritt für Schritt

Thema	Welche Begriffe konstant verwendet werden
Problem	Synonyme und wechselnde Statusnamen erzeugen Missverständnisse.
Lösung	Ein kontrolliertes Glossar legt deutsche und englische Begriffe fest.
Ergebnis	Konsistente Thesis und Präsentation.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

36.1 Zweck

Dieses Kapitel legt die verbindlichen Fachbegriffe für die gesamte Forschungs- und Lernfassung fest. Beim ersten Auftreten kann die englische Fachbezeichnung in Klammern ergänzt werden. Danach wird grundsätzlich die deutsche Form verwendet, sofern es sich nicht um einen etablierten Eigennamen, eine Programmierschnittstelle oder einen Relationstyp des Evidenzmodells handelt. Ziel ist, dass jedes zentrale Konzept im gesamten Dokument eine bevorzugte, konsistente Bezeichnung erhält (vgl. Nagy et al., 2015; Alshemaimri et al., 2021; Microsoft, 2026b).

36.2 Terminologieregeln

Englisch	Deutsch (bevorzugt)
Evidence	Evidenz
Observation	Beobachtung
Derived Relation	abgeleitete Relation
Provenance	Provenienz
Answerability	Antwortbarkeit
Traceability	Nachvollziehbarkeit
Correct Refusal	begründete Verweigerung
Corpus	Korpus
Gold Standard	Goldstandard
Research Artifact	Forschungsartefakt
Evidence Path	Evidenzpfad
Evidence Completeness	Evidenzvollständigkeit
Evidence Precision	Evidenzpräzision
Path Validity	Pfadgültigkeit

Die Relationstypen des Evidenzmodells bleiben als technische Bezeichner in Großbuchstaben erhalten: READ, WRITE, INVOKES, MAPS_TO, MUTATES, PERSISTS, SUPPORTED_BY. Etablierte Eigennamen und Technologien werden in ihrer üblichen Form verwendet, zum Beispiel Roslyn, Entity Framework, Code Property Graph und Stored Procedure (vgl. Microsoft, 2026a; Shatnawi et al., 2019).

Merksatz: Ein Konzept erhält im gesamten Dokument genau eine bevorzugte Bezeichnung. Wenn ein Begriff einmal festgelegt ist, bleibt er in allen Kapiteln gleich.

37. Priorisierung von Zielen, Konfigurationen und Erweiterungen

Kapitelorientierung - Schritt für Schritt

Thema	Was zuerst gebaut wird
Problem	Deep Learning und Agenten können den Kern verdrängen.
Lösung	Core, Baseline, Experiment und Future Work werden strikt getrennt.
Ergebnis	Ein umsetzbarer Entwicklungsweg.
Erklärziel	Erkläre nach dem Lesen in eigenen Worten, warum dieses Kapitel für Cross-Repository Code Intelligence notwendig ist.

Bestätigte Priorität

Deep Learning, LoRA, Multi-Agent-Systeme und freies GraphRAG gehören nicht zum verpflichtenden Kern. Sie dürfen nur als klar getrennte Baseline, Course Experiment oder Future Work erscheinen.

37.1 Verbindlicher Kern

Zum verbindlichen Kern der Arbeit gehören das kanonische Evidenzmodell, die Roslyn-basierte C#-Analyse, die im Korpus tatsächlich vorkommenden Entity-Framework-Muster, die Trennung von Beobachtung, abgeleiteter Relation und Antwort, die Provenienzerhaltung, ein eingefrorener Korpus, ein manuell geprüfter Goldstandard sowie die getrennte Evaluation von Extraktion und Retrieval. Damit ist klar festgelegt, welche Elemente im Bachelorumfang zwingend umgesetzt und wissenschaftlich geprüft werden müssen: Cross-Repository Code Intelligence soll nicht nur ein technischer Prototyp sein, sondern ein nachvollziehbares Forschungsartefakt, das auf einem stabilen Snapshot arbeitet, seine Evidenzpfade offenlegt und seine Qualität gegen einen kontrollierten Goldstandard misst (vgl. Hevner et al., 2004).

37.2 Bedingte Erweiterungen

Bedingte Erweiterungen sind Teile des Projekts, die fachlich sinnvoll und wünschenswert sind, aber nur dann zum verbindlichen Umfang gehören, wenn der eingefrorene Korpus ausreichend Fälle bietet und die Zeit- und Ressourcenlage eine solide Evaluation zulässt. Dazu zählen insbesondere:

- ausgewählte ADO.NET-Muster mit SqlConnection, SqlCommand und passenden Execute-Varianten,
- eingebettetes SQL in C#-Code,
- separate SQL-Dateien,
- Stored Procedures mit ausreichend zugänglicher Definition und Aufrufkontext,
- begrenzte interprozedurale Verknüpfung,
- Cross-Repository-/Cross-Repository/Cross-Project- und Cross-Database-Fälle.

Diese Erweiterungen sollen nur aufgenommen werden, wenn sie im Frozen Snapshot tatsächlich vorkommen, annotierbar sind und eine sinnvolle Ergänzung zum EF-Kern bilden. Andernfalls bleiben sie ausdrücklich als optionale Erweiterung markiert und werden nicht als Teil des Pflichtumfangs betrachtet.

37.3 Nicht verbindliche und zukünftige Erweiterungen

Nicht verbindliche Erweiterungen sind Themen und Technologien, die für Cross-Repository Code Intelligence konzeptionell interessant sind, aber im Rahmen dieser Bachelorarbeit nicht zum Pflichtumfang gehören. Beispiele sind: (vgl. Usai et al., 2026; Shatnawi et al., 2019; Alebachew et al., 2026).

- graphbasierte Retrieval-Augmented-Ansätze mit großen Sprachmodellen,
- LLM-basierte Antwortgenerierung auf Grundlage des Code-Database Evidence Graphs,
- komplexe Multi-Agent-Systeme oder orchestrierte Tools,
- eine produktionsreife Benutzeroberfläche mit umfassender Interaktion,
- universelle natürliche Sprache für alle Frageformen,
- vollständige Laufzeitverifikation von Datenbankzugriffen,
- Unterstützung weiterer ORMs oder SQL-Dialekte über den untersuchten Kern hinaus,
- allgemeine Compliance- oder Security-Aussagen ohne spezifisches Evaluationsdesign.

Diese Punkte bleiben bewusst im Backlog oder als Perspektive für zukünftige Arbeit, solange keine dokumentierte Entscheidung den Scope der Bachelorarbeit erweitert. Sie können in Diskussion und Ausblick erwähnt werden, dürfen aber die Anforderungen an den verbindlichen Kern nicht verwässern (vgl. Ullrich et al., 2025).

Merksatz: Der verbindliche Kern definiert, was im Rahmen der Bachelorarbeit sicher geliefert wird. Bedingte Erweiterungen kommen nur hinzu, wenn sie im Korpus real vorkommen und sauber evaluierbar sind. Nicht verbindliche Erweiterungen bleiben bewusst als Zukunftsthema markiert, damit das Projekt realistisch und wissenschaftlich kontrollierbar bleibt.

Ende der konsolidierten Langfassung (Lernfassung 2026). Diese Fassung dient dem vollständigen Verständnis und der Vorbereitung der Verteidigung. Für die offizielle Einreichung wird daraus eine gekürzte, formale Exposéfassung abgeleitet.

38. Implementierungsmanual: vom Corpus zum überprüfbaren Ergebnis

Orientierung

Problem	Ein großes Forschungsziel muss in kleine, testbare Module zerlegt werden.
Lösung	Jeder Schritt erhält Input, Output, Invariant, Test und Abbruchkriterium.
Ergebnis	Ein reproduzierbarer Implementierungspfad, der nicht von einem Agenten oder LLM abhängt.

38.1 Repository und Reproduzierbarkeit

- Lege einen unveränderlichen Corpus-Snapshot über Commit-Hash und Submodule fest.
- Speichere Lizenz, Sprache, Projektstruktur, Build-Anleitung und Ausschlüsse in corpus-manifest.yaml.
- Trenne src/, tests/, fixtures/, corpus-manifest/, gold/, experiments/ und reports/.
- Jeder Experimentlauf erhält Konfiguration, Git-Hash, Zeitstempel, Toolversionen und Output-Hash.

Prüffrage

Welcher konkrete Test würde zeigen, dass die Aussagen dieses Abschnitts falsch implementiert wurden?

38.2 Minimaler Solution-Aufbau

- Core: domänenunabhängige Evidence- und Graphmodelle.
- Extractors.CSharp: Roslyn Syntax, SemanticModel, Symbols, SourceSpan.
- Extractors.EFCore: DbContext, DbSet, Mappings, LINQ, Mutations, SaveChanges.
- Extractors.Sql: SQL-Dateien, ADO.NET, Raw SQL und Procedures.
- Graph: Schema, Import, Constraints und Query Repository.
- Evaluation: Gold Loader, Metrics, Error Taxonomy und Report Generator.
- CLI: scan, export-evidence, build-graph, query, evaluate.

Prüffrage

Welcher konkrete Test würde zeigen, dass die Aussagen dieses Abschnitts falsch implementiert wurden?

38.3 EvidenceRecord-Vertrag

- Jeder Record besitzt recordId, repositoryId, projectId, filePath, lineStart, lineEnd und symbolId.
- ObservationType beschreibt die lokale Beobachtung; ClaimType beschreibt die mögliche Schlussfolgerung.
- RuleId und RuleVersion machen Ableitungen reproduzierbar.
- Status unterscheidet OBSERVED, DERIVED, INCOMPLETE und UNRESOLVED.
- Kein Record darf eine physische Tabelle behaupten, wenn nur eine Entity ohne Mapping bekannt ist.

Prüffrage

Welcher konkrete Test würde zeigen, dass die Aussagen dieses Abschnitts falsch implementiert wurden?

38.4 Globale Identitäten

- RepositoryId und ProjectId werden explizit gespeichert.
- C#-Symbole verwenden nach Möglichkeit Assembly, Namespace, ContainingType, MethodSignature und Source Location.
- Datenbankobjekte verwenden Database/Schema/Table/Column bzw. eine dokumentierte Teilidentität.
- Unaufgelöste Ziele erhalten keinen erfundenen globalen Schlüssel, sondern einen expliziten unresolved reference record.

Prüffrage

Welcher konkrete Test würde zeigen, dass die Aussagen dieses Abschnitts falsch implementiert wurden?

38.5 EF-Core-READ-Regeln

- Eine DbSet-Referenz ist ein Kandidat, aber noch keine ausgeführte READ-Operation.
- Materialisierung kann über ToList, First, Single, Any, Count, foreach, await foreach oder provider-spezifische Varianten erfolgen.
- Property-Nutzung erlaubt nur dann eine Spaltenaussage, wenn ein Property-to-Column-Mapping aufgelöst ist.
- Deferred Execution und Client Evaluation müssen als Grenzen dokumentiert werden.

Prüffrage

Welcher konkrete Test würde zeigen, dass die Aussagen dieses Abschnitts falsch implementiert wurden?

38.6 EF-Core-WRITE-Regeln

- Add, Update, Remove, Attach plus State-Änderungen erzeugen Mutationskandidaten.
- Ein unterstützter Persistenzclaim benötigt eine nachvollziehbare Verbindung zu SaveChanges/SaveChangesAsync oder einer alternativen Persistenzoperation.
- Bulk-APIs wie ExecuteUpdate oder ExecuteDelete benötigen eigene Regeln.
- Trigger, Interceptors und Datenbankseiteneffekte dürfen nicht ohne separate Evidenz behauptet werden.

Prüffrage

Welcher konkrete Test würde zeigen, dass die Aussagen dieses Abschnitts falsch implementiert wurden?

38.7 SQL- und Procedure-Regeln

- Raw SQL wird geparkt, sofern Dialekt und Stringwert ausreichend auflösbar sind.
- Teilweise dynamische Strings erzeugen Evidence Gaps oder Platzhalter, keine vollständig aufgelösten Claims.
- Procedure-Aufrufe und Procedure-Definitionen werden getrennt extrahiert und erst über den Procedure-Namen verbunden.
- Dynamisches SQL innerhalb von Procedures wird als eigene Unsicherheitsquelle markiert.

Prüffrage

Welcher konkrete Test würde zeigen, dass die Aussagen dieses Abschnitts falsch implementiert wurden?

38.8 Graphschema

- Knoten: Repository, Project, File, Symbol/Method, Entity, DatabaseObject, DatabaseOperation, EvidenceRecord, SourceLocation, UseCase, Role.

- Beziehungen: CONTAINS, CALLS, MAPS_TO, OBSERVES, SUPPORTS, READS_FROM, WRITES_TO, EXECUTES_PROCEDURE, LOCATED_AT, DEFINED_FOR.
- SourceLocation und EvidenceRecord bleiben erstklassig, damit Provenienz nicht in einer Kante verloren geht.
- Constraints schützen globale IDs; Indexe unterstützen Symbol-, Tabellen- und Source-Location-Abfragen.

Prüffrage

Welcher konkrete Test würde zeigen, dass die Aussagen dieses Abschnitts falsch implementiert wurden?

38.9 Query Contract und Ausgabe

- Ein Query Contract benennt Rolle, Frageart, Zielobjekt, erlaubte Evidenz und erforderliche Vollständigkeit.
- Das Retrieval liefert Kandidaten und Pfade; der Verifier prüft Pflichtschritte.
- Die Ausgabe trennt Claim, Evidence, Missing Evidence, Status und Limitation.
- Formulierung durch ein LLM ist nur erlaubt, wenn der strukturierte Claim unverändert bleibt und jede Quelle übernommen wird.

Prüffrage

Welcher konkrete Test würde zeigen, dass die Aussagen dieses Abschnitts falsch implementiert wurden?

38.10 Teststrategie

- Unit Tests pro Extraktionsregel mit minimalen C#- und SQL-Fixtures.
- Integration Tests vom Source File bis EvidenceRecord.
- Graph Tests für Constraints, Import und Query Contracts.
- Golden Tests für JSON-Ausgaben und Evidence Paths.
- Negative Tests für Namensähnlichkeit, tote Pfade, fehlende Mappings und dynamische Strings.

Prüffrage

Welcher konkrete Test würde zeigen, dass die Aussagen dieses Abschnitts falsch implementiert wurden?

38.11 Experimentausführung

- Konfigurationen werden deklarativ in YAML oder JSON gespeichert.
- Jeder Lauf erzeugt predictions.jsonl, metrics.json, errors.csv und run-manifest.json.
- Keine manuelle Nachkorrektur der Testausgaben vor Metrikberechnung.
- Fehleranalyse erfolgt auf einer Kopie der unveränderten Rohoutputs.

Prüffrage

Welcher konkrete Test würde zeigen, dass die Aussagen dieses Abschnitts falsch implementiert wurden?

38.12 Beispiel eines Evidence Records

```
{
  "recordId": "ev-000184",
  "repositoryId": "danphe",
  "projectId": "PatientManagement",
  "symbolId": "PatientRepository.UpdatePatient(Patient)",
  "observationType": "EF_ENTITY_MUTATION",
  "operationCandidate": "WRITE",
  "entity": "Patient",
  "databaseTarget": "dbo.Patient",
  "source": {"file": "PatientRepository.cs", "lineStart": 104, "lineEnd": 110},
  "ruleId": "EF-WRITE-UPDATE-SAVECHANGES",
  "status": "SUPPORTED",
  "evidencePath": ["mutation", "entity-table-mapping", "SaveChangesAsync"]
}
```

39. Fachartikelkatalog mit aktuellen Dateinamen und Bezugsquellen

Namensregel

In diesem Katalog steht ausschließlich der aktuelle Dateiname. Alte Source-Namen oder frühere Umbenennungen werden nicht angezeigt.

39.1 ★★★★★

01_★★★★★_CORE_R06_Nagy_2015_Where_This_SQL_Query_Executed.pdf

Nagy, C., Meurice, L., & Cleve, A. (2015). Where Was This SQL Query Executed? A Static Concept Location Approach. SANER.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://doi.org/10.1109/SANER.2015.7081881>

Warum diese Quelle im Projekt liegt

Status: CORE

Beitrag: Direkter methodischer Bezug zur statischen Lokalisierung von SQL-Ausführungsstellen.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.2 ★★★★★

02_★★★★★_CORE_R09_Usai_2026_LogicLens_Multi_Repository_Semantic_Code_Graph.pdf

Usai, N., Montagnini, D., Iliev, K. I., & Camanzo, R. (2026). LogicLens: Leveraging Semantic Code Graph to explore Multi Repository large systems.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2601.10773>

Warum diese Quelle im Projekt liegt

Status: CORE

Beitrag: Nächste verwandte Arbeit zu semantischem Multi-Repository-Graph und natürlichen Fragen.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.3 ★★★★★

03_★★★★★_CORE_R04_Shawnawi_2019_Static_Analysis_of_Multilanguage_Systems.pdf

Shawnawi, A., et al. (2019). Static Code Analysis of Multilanguage Software Systems.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/1906.00815>

Warum diese Quelle im Projekt liegt

Status: CORE

Beitrag: Begründet heterogene Artefakte, versteckte Framework-Abhängigkeiten und gemeinsame Repräsentation.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.4 ★★★★★

04_★★★★★_CORE_R02_Yamaguchi_2014_Code_Property_Graphs.pdf

Yamaguchi, F., Golde, N., Arp, D., & Rieck, K. (2014). Modeling and Discovering Vulnerabilities with Code Property Graphs. IEEE S&P.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://doi.org/10.1109/SP.2014.44>

Warum diese Quelle im Projekt liegt

Status: CORE

Beitrag: Grundlage für die Verbindung mehrerer Programmdarstellungen in einem Property Graph.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.5 ★★★★★

05_★★★★★_CORE_R05_Xie_2026_CodeFuse_Query_Large_Scale_Code_Analysis.pdf

Xie, X., et al. (2026). Principles and Practices of Large-Scale Code Analysis at Ant Group. ICSE-SEIP.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://doi.org/10.1145/3786583.3786907>

Warum diese Quelle im Projekt liegt

Status: CORE

Beitrag: Beispiel für daten- und logikorientierte großskalige Codeanalyse.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.6 ★★★★★

06_★★★★★_CORE_R15_Abedu_2025_LLM_Knowledge_Graph_Repository_QA.pdf

Abedu, S., Khatoonabadi, S., & Shihab, E. (2025). Synergizing LLMs and Knowledge Graphs for Software Repository-Related Question Answering.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2412.03815>

Warum diese Quelle im Projekt liegt

Status: CORE

Beitrag: Empirische Verbindung von Repository-Fragen, Knowledge Graph und LLM; wichtig für Grenzen und Evaluation.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.7 ★★★★★

07_★★★★★_CORE_R01_Hevner_2004_Design_Science_in_Information_Systems.pdf

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design Science in Information Systems Research. MIS Quarterly, 28(1), 75-105.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://aisel.aisnet.org/misq/vol28/iss1/6/>

Warum diese Quelle im Projekt liegt

Status: CORE

Beitrag: Methodische Grundlage für Build, Evaluate und wissenschaftlichen Artefaktbeitrag.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.8 ★★★★★☆

08_★★★★☆_IMPORTANT_R07_Alshemaimri_2021_Problematic_Database_Code_Fragments_Survey.pdf

Alshemaimri, B., Elmasri, R., Alsahfi, T., & Almotairi, M. (2021). A survey of problematic database code fragments in software systems. Engineering Reports, 3, e12441.

Offizielle oder öffentlich zugängliche Bezugsquelle:

<https://onlinelibrary.wiley.com/doi/full/10.1002/eng2.12441>

Warum diese Quelle im Projekt liegt

Status: IMPORTANT

Beitrag: Ordnet SQL-, ORM- und Architekturformen des Datenbankzugriffs.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.9 ★★★★★☆

09_★★★★☆_IMPORTANT_R13_Lekssays_2026_Bridging_CPGs_and_Language_Models.pdf

Lekssays, A. (2026). Bridging Code Property Graphs and Language Models for Program Analysis. SVM.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2603.24837>

Warum diese Quelle im Projekt liegt

Status: IMPORTANT

Beitrag: Zeigt, wie High-Level-Tools CPG-Analysen für LLMs kontrollierbar zugänglich machen.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.10 ★★★★★☆

10_★★★★☆_IMPORTANT_R17_Shah_2025_RANGER_Graph_Enhanced_Repository_Retrieval.pdf

Shah, P., Ghosh, R., Singhal, A., & Dutta, D. (2025). RANGER: Repository-Level Agent for Graph-Enhanced Retrieval.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2509.25257>

Warum diese Quelle im Projekt liegt

Status: IMPORTANT

Beitrag: Vergleichspunkt für graphgestütztes Repository-Retrieval und natürliche Fragen.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.11 ★★★★★☆

11_★★★★☆_IMPORTANT_R10_Gandhi_2025_Repository_Level_Code_Search_Neural_Retrieval.pdf

Gandhi, S., Gao, L., & Callan, J. (2025). Repository-level Code Search with Neural Retrieval Methods.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2502.07067>

Warum diese Quelle im Projekt liegt

Status: IMPORTANT

Beitrag: Liefert BM25- und CodeBERT-Baselines sowie Ranking-Metriken.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.12 ★★★★★☆

12_★★★★☆_IMPORTANT_R29_Peng_2026_SWE_QA_Repository_Level_Code_Questions.pdf

Peng, W., et al. (2026). SWE-QA: Can Language Models Answer Repository-level Code Questions?

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2509.14635>

Warum diese Quelle im Projekt liegt

Status: IMPORTANT

Beitrag: Taxonomie und Validierungsprozess für repositoryweite Fragen.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.13 ★★★★★☆

13_★★★★☆_IMPORTANT_R28_Alebachew_2026_StackRepoQA_Repository_Level_QA.pdf

Alebachew, Y. B., Leary, H., Vaishampayan, S., & Brown, C. (2026). Beyond Code Snippets: Benchmarking LLMs on Repository-Level Question Answering.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2603.26567>

Warum diese Quelle im Projekt liegt

Status: IMPORTANT

Beitrag: Zeigt Nutzen struktureller Signale und Risiken von Memorization bei Repository-QA.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.14 ★★★★★☆

14_★★★★☆_IMPORTANT_R31_Du_2025_DependEval_Repository_Dependency_Understanding.pdf

Du, J., et al. (2025). DependEval: Benchmarking LLMs for Repository Dependency Understanding.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2503.06689>

Warum diese Quelle im Projekt liegt

Status: IMPORTANT

Beitrag: Vorbild für hierarchische Abhängigkeitsaufgaben und Multi-File-Evaluation.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.15 ★★★★★☆

15_★★★★☆_IMPORTANT_R23_Olea_Persona_Prompting_for_Question_Answering.pdf

Olea, C., et al. (2024). Evaluating Persona Prompting for Question Answering Tasks.

Offizielle oder öffentlich zugängliche Bezugsquelle:

<https://aircconline.com/csit/abstract/v14n11/csit141106.html>

Warum diese Quelle im Projekt liegt

Status: IMPORTANT

Beitrag: Empirische Grundlage, um Rollen- oder Persona-Effekte nicht nur intuitiv zu behaupten.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.16 ★★★★★☆

16_★★★★☆_IMPORTANT_R30_Chen_2025_CoReQA_Code_Repository_Question_Answering.pdf

Chen, J., et al. (2025). CoReQA: Uncovering Potentials of Language Models in Code Repository Question Answering.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2501.03447>

Warum diese Quelle im Projekt liegt

Status: IMPORTANT

Beitrag: Liefert Repository-QA-Datensatz und mehrdimensionale Antwortbewertung.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.17 ★★★★★☆

17_★★★★☆_SUPPORT_R27_Tao_2026_Retrieval_Augmented_Code_Generation_Survey.pdf

Tao, Y., Li, Y., Qin, Y., & Liu, Y. (2026). Retrieval-Augmented Code Generation: A Survey with Focus on Repository-Level Approaches.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2510.04905>

Warum diese Quelle im Projekt liegt

Status: SUPPORT

Beitrag: Orientierung zu Retrieval-Substraten, Agenten, Benchmarks und Necessity Boundary.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.18 ★★★★★

18_★★★★★_SUPPORT_R16_Tao_2025_Code_Graph_Model_CGM_NeurlPS.pdf

Tao, H., et al. (2025). Code Graph Model: A Graph-Integrated Large Language Model for Repository-Level Software Engineering Tasks. NeurIPS.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2505.16901>

Warum diese Quelle im Projekt liegt

Status: SUPPORT

Beitrag: Zeigt eine mögliche spätere Deep-Learning-Erweiterung, nicht den Pflichtkern.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.19 ★★★★★

19_★★★★★_SUPPORT_R14_Lekssays_2025_LLMxCPG_Context_Aware_Program_Analysis.pdf

Lekssays, A., Mouhcine, H., Tran, K., Yu, T., & Khalil, I. (2025). LLMxCPG.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2507.16585>

Warum diese Quelle im Projekt liegt

Status: SUPPORT

Beitrag: Beispiel für CPG-basiertes Slicing und fokussierten Modellkontext.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.20 ★★★★★

20_★★★★★_SUPPORT_R11_Zhang_2023_RepoCoder_Iterative_Retrieval_and_Generation.pdf

Zhang, F., et al. (2023). RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. EMNLP.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://aclanthology.org/2023.emnlp-main.151.pdf>

Warum diese Quelle im Projekt liegt

Status: SUPPORT

Beitrag: Belegt Nutzen iterativer Retrieval-Generierungs-Schleifen; Completion bleibt außerhalb des Kerns.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.21 ★★★★★

21_★★★★☆_SUPPORT_R21_Ullrich_2025_Requirements_to_Code_with_LLMs.pdf

Ullrich, J., Koch, M., & Vogelsang, A. (2025). From Requirements to Code.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2507.07548>

Warum diese Quelle im Projekt liegt

Status: SUPPORT

Beitrag: Hilft, Use Cases in implementierbare Aufgaben und Architekturvorgaben zu zerlegen.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.22 ★★★★★

22_★★★★☆_SUPPORT_R03_Lewis_2020_Retrieval_Augmented_Generation.pdf

Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2005.11401>

Warum diese Quelle im Projekt liegt

Status: SUPPORT

Beitrag: Grundlage für Trennung von Retriever und Generator sowie externe Provenienz.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.23 ★★★★★

23_★★★★☆_SUPPORT_R24_Hou_2024_LLMs_for_Software_Engineering_Systematic_Review.pdf

Hou, X., et al. (2024). Large Language Models for Software Engineering: A Systematic Literature Review.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2308.10620>

Warum diese Quelle im Projekt liegt

Status: SUPPORT

Beitrag: Breiter Forschungsüberblick zu LLM4SE, Daten, Metriken und Risiken.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.24 ★★☆☆☆

24_★★☆☆☆_OPTIONAL_R12_Edge_2025_GraphRAG_Query_Focused_Summarization.pdf

Edge, D., et al. (2025). From Local to Global: A GraphRAG Approach to Query-Focused Summarization.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2404.16130>

Warum diese Quelle im Projekt liegt

Status: OPTIONAL

Beitrag: Wichtig für Future Work, aber nicht direkt auf statische Code-Datenbank-Evidenz zugeschnitten.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.25 ★★☆☆☆

25_★★☆☆☆_OPTIONAL_R19_Pan_2026_ArchAgent_Cross_Repository_Architecture_Recovery.pdf

Pan, R., Mao, B., Ma, T., & Ling, Z. (2026). ArchAgent.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2601.13007>

Warum diese Quelle im Projekt liegt

Status: OPTIONAL

Beitrag: Vergleich zu Cross-Repository-Architekturrekonstruktion.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.26 ★★☆☆☆

26_★★☆☆☆_OPTIONAL_R25_Liu_2025_LLM_Agents_for_Software_Engineering_Survey.pdf

Liu, J., et al. (2025). Large Language Model-Based Agents for Software Engineering: A Survey.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2409.02977>

Warum diese Quelle im Projekt liegt

Status: OPTIONAL

Beitrag: Zur Einordnung von Agenten, Tools, Memory und Action; nicht Pflichtarchitektur.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.27 ★★★★★

27_★★★★★_OPTIONAL_R26_Zhang_2024_Survey_on_LLMs_for_Software_Engineering.pdf

Zhang, Q., et al. (2024). A Survey on Large Language Models for Software Engineering.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2312.15223>

Warum diese Quelle im Projekt liegt

Status: OPTIONAL

Beitrag: Breite Einordnung von Code-LLMs und SE-Aufgaben.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

39.28 ★★★★★

28_★★★★★_OPTIONAL_R34_Liu_2026_Beyond_Functional_Correctness_Code_Hallucinations.pdf

Liu, F., et al. (2026). Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code.

Offizielle oder öffentlich zugängliche Bezugsquelle: <https://arxiv.org/pdf/2404.00971>

Warum diese Quelle im Projekt liegt

Status: OPTIONAL

Beitrag: Nützlich für unsupported claims und Zuverlässigkeitsdiskussion, nicht Kern der Extraktion.

Leseauftrag: Formuliere Problem, Methode, Ergebnis, Limitation und eine konkrete Projektentscheidung in eigenen Worten.

40. Technische Lernressourcen und konkrete Übungen

Die Ressourcen werden nicht als Link-Sammlung verwendet. Jede Ressource ist mit genau einer projektbezogenen Aufgabe verbunden. Erst wenn der Output gespeichert und erklärt wurde, gilt die Ressource als bearbeitet.

40.1 DeepLearning.AI: Knowledge Graphs for RAG

Link: <https://www.deeplearning.ai/courses/knowledge-graphs-rag>

Exakte Übung

Nach jeder Lektion exakt eine Cypher-Abfrage auf dem Projekt-Schema formulieren; Abschluss: Query, die Method -> DatabaseOperation -> Table -> SourceLocation zurückgibt.

40.2 DeepLearning.AI: Building and Evaluating Advanced RAG

Link: <https://www.deeplearning.ai/courses/building-evaluating-advanced-rag>

Exakte Übung

Die RAG-Triad auf ein projektbezogenes Beispiel übertragen und Groundedness ausdrücklich gegen Evidence-Path-Validity abgrenzen.

40.3 DeepLearning.AI: Deep Learning Specialization

Link: <https://www.deeplearning.ai/specializations/deep-learning>

Exakte Übung

Nur Course 5 Sequence Models für RNN, LSTM, Attention und Transformer priorisieren; Ergebnis: ein klar als Experiment markiertes Evidence-Sequence-Notebook.

40.4 DeepLearning.AI: Machine Learning Specialization

Link: <https://www.deeplearning.ai/specializations/machine-learning>

Exakte Übung

Evaluation, Train/Test-Trennung und Overfitting wiederholen; Ergebnis: reproduzierbare Baseline-Auswertung.

40.5 DeepLearning.AI: Mathematics for Machine Learning and Data Science

Link: <https://www.deeplearning.ai/specializations/mathematics-for-machine-learning-and-data-science>

Exakte Übung

Nur lineare Algebra, Wahrscheinlichkeitsgrundlagen und Statistik gezielt für Metriken und Embeddings nutzen.

40.6 Neo4j GraphAcademy: Neo4j Fundamentals

Link: <https://graphacademy.neo4j.com/>

Exakte Übung

Exakte Übung: das minimale Graphschema mit fünf Knotenarten und sechs Relationsarten modellieren.

40.7 Neo4j GraphAcademy: Cypher Fundamentals / Sharpen your Cypher

Link: <https://www.graphacademy.neo4j.com/categories/sharpen-cypher>

Exakte Übung

Exakte Übung: drei Query Contracts als Cypher implementieren und mit positiven sowie negativen Fixtures testen.

40.8 Neo4j GraphAcademy: Neo4j and Generative AI Workshop

Link: <https://www.graphacademy.neo4j.com/courses/workshop-genai>

Exakte Übung

Nur nach Abschluss des deterministischen Kerns; Ergebnis: optionaler text-to-Cypher Pilot mit Validierung.

40.9 Microsoft Learn: Roslyn SDK

Link: <https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/>

Exakte Übung

Exakte Übung: einen Analyzer schreiben, der InvocationExpression, Symbol und SourceSpan eines EF-Aufrufs extrahiert.

40.10 Microsoft Learn: Entity Framework Core

Link: <https://learn.microsoft.com/en-us/ef/core/>

Exakte Übung

Exakte Übung: Add/Update/Remove, LINQ-Materialisierung und Mappingformen in kleinen Testprojekten katalogisieren.

40.11 Joern: Joern Documentation

Link: <https://docs.joern.io/>

Exakte Übung

Nur als Vergleichswerkzeug oder Future Work; eine CPG-Traversierung einem eigenen Evidence Path gegenüberstellen.

40.12 CodeQL: CodeQL Documentation

Link: <https://codeql.github.com/docs/>

Exakte Übung

Eine einfache C#-Query als Baseline formulieren und dokumentieren, was CodeQL ohne projektspezifische Mappinglogik nicht auflöst.

40.13 Tree-sitter: Tree-sitter Documentation

Link: <https://tree-sitter.github.io/tree-sitter/>

Exakte Übung

Als Alternative für heterogene Syntaxextraktion prüfen; Ergebnis: Capability-Matrix gegenüber Roslyn.

Anhang C - Dreisprachiges Fachglossar Deutsch - Englisch -

فارسی

Das Glossar dient der mündlichen Automatisierung. Für jede Zeile wird zuerst der englische Begriff gelesen, danach die deutsche Definition erklärt und schließlich die persische Bedeutung zur schnellen Bedeutungsprüfung genutzt.

English term	Deutscher Begriff	فارسی	Einfache deutsche Erklärung
Query contract	Abfragevertrag	قرارداد پرسوجو	Definiert Eingabe, benötigte Evidenz, Ausgabeformat und Verweigerungsregel einer Frage.
Derived relation	Abgeleitete Relation	رابطه استنتاج شده	Beziehung, die aus mehreren Beobachtungen geschlossen und nicht direkt an einer einzelnen Quellstelle gefunden wird.
Abstract syntax tree (AST)	Abstract Syntax Tree	درخت نحو انتزاعی	Baumdarstellung der syntaktischen Struktur eines Programms.
Answerability	Antwortbarkeit	پاسخ پذیری	Entscheidung, ob die vorhandene Evidenz für eine belastbare Antwort ausreicht.
Baseline system	Baselinesystem	سامانه مبنا	Einfaches Vergleichssystem, gegen das der Mehrwert einer neuen Methode gemessen wird.
Codebase	Codebase	پایه کد	Gesamtheit des Quellcodes und zugehöriger Artefakte eines Systems oder Projekts.
Code-Database Evidence Graph	Code-Database Evidence Graph	گراف شواهد کد-پایگاه داده	Interne Graphrepräsentation für Codeelemente, Datenbankoperationen, Datenbankobjekte, Evidenz und Provenienz.
Code property graph (CPG)	Code Property Graph	گراف ویژگی کد	Gemeinsame Graphrepräsentation von Syntax, Kontrollfluss und Abhängigkeiten.
Corpus census	Corpus Census	سرشماری پیکره	Dokumentierte Bestandsaufnahme des Evaluationskorpus.
Cross-repository	Cross-Repository	بین مخزنی	Beziehungen oder Fragen, die mehr als ein Repository oder logisch getrennte Codeprojekte betreffen.
Database operation	Datenbankoperation	عملیات پایگاه داده	Typisierte READ-, WRITE- oder EXECUTE-Beziehung zwischen Code und Datenbankobjekt.
Direct evidence	Direkte Evidenz	شاهد مستقیم	Beobachtung, die unmittelbar aus einer konkreten Quellstelle extrahiert wird.
Entity Framework Core	Entity Framework Core	انتیتی فریم ورک کور	ORM für .NET, das Objekte, LINQ, Mappings und Persistenzoperationen verbindet.
Evidence gap	Evidence Gap	شکاف شواهد	Fehlender Zwischenschritt,

English term	Deutscher Begriff	فارسی	Einfache deutsche Erklärung
			der eine sichere Schlussfolgerung verhindert.
Evidence path	Evidence Path	مسیر شواهد	Geordnete Folge von Beobachtungen und Relationen, die einen Claim trägt.
False negative	False Negative	منفی کاذب	Vorhandene relevante Evidenz, die das System nicht erkennt.
False positive	False Positive	مثبت کاذب	Vom System gemeldete Evidenz, die im Goldstandard nicht bestätigt ist.
F1 score	F1-Score	F1 امتیاز	Harmonisches Mittel aus Precision und Recall.
Frozen snapshot	Frozen Snapshot	عکس ثابت پیکره	Unveränderlich referenzierter Stand des Korpus, meist über einen Commit-Hash.
Gold standard	Goldstandard	استاندارد طلایی	Manuell geprüfte Referenzdaten für Evaluation.
Graph database	Graphdatenbank	پایگاه داده گراف	Datenbanksystem, das Knoten, Kanten und Eigenschaften speichert und traversiert.
Hard negative	Hard Negative	نمونه منفی دشوار	Negatives Beispiel, das oberflächlich stark wie ein positives Beispiel aussieht.
Confidence	Konfidenz	اطمینان	Grad der Sicherheit einer Vorhersage; darf nicht ohne Kalibrierung als Wahrscheinlichkeit interpretiert werden.
Query materialization	Materialisierung	اجرای نهایی پرسوجو	Operation, durch die eine LINQ- oder ORM-Abfrage tatsächlich ausgeführt wird.
Method symbol	Methodensymbol	نماد متد	Semantisch aufgelöste Identität einer Methode, nicht nur ihr Textname.
Mean reciprocal rank	MRR	میانگین رتبه معکوس	Retrieval-Metrik, die die Position des ersten relevanten Ergebnisses bewertet.
Neo4j	Neo4j	ننو ۴جی	Property-Graph-Datenbank mit der Abfragesprache Cypher.
Unsupported answer	Nicht gestützte Antwort	پاسخ بدون پشتوانه	Antwort, deren Claim nicht vollständig durch erlaubte Evidenz belegt ist.
Normalization	Normalisierung	نرمال سازی	Überführung heterogener Extractor-Ausgaben in ein gemeinsames Datenmodell.
Precision	Precision	دقت مثبت	Anteil der gemeldeten positiven Ergebnisse, die korrekt sind.
Provenance	Provenienz	منشأ	Information darüber, aus welcher Datei, Zeile, Regel oder Ableitung ein Ergebnis stammt.
Recall	Recall	بازیابی	Anteil der tatsächlich

English term	Deutscher Begriff	فارسی	Einfache deutsche Erklärung
			relevanten Ergebnisse, die gefunden werden.
Repository	Repository	مخزن کد	Versionierter Speicher für Quellcode und begleitende Artefakte.
Retrieval	Retrieval	بازیابی	Auswahl relevanter Dokumente, Knoten, Pfade oder Evidence Records zu einer Frage.
Role-specific use case	Role-Specific Use Case	مورد کاربرد وابسته به نقش	Aufgabe, deren Informationsbedarf und Ausgabeformat von einer Nutzerrolle abhängen.
Roslyn	Roslyn	رازلین	Compilerplattform und API für syntaktische und semantische C#-Analyse.
Source location	Source Location	مکان منبع	Datei, Zeilenbereich und gegebenenfalls Symbol, an dem Evidenz gefunden wurde.
Static analysis	Statische Analyse	تحلیل ایستا	Analyse von Programmcode ohne konkrete Programmausführung.
Traceability	Traceability	ردیابی پذیری	Möglichkeit, jeden Claim bis zu seinen Quellstellen und Ableitungen zurückzuverfolgen.
Verifier	Verifizierer	تأییدگر	Komponente, die prüft, ob Kandidaten die Evidence-Regeln erfüllen.
WRITE	WRITE	نوشتن	Operation, die Daten persistiert oder eine persistierbare Mutation vorbereitet und ausführt.
READ	READ	خواندن	Operation, die Daten aus einer Datenquelle abfragt oder materialisiert.
EXECUTES_PROCEDURE	EXECUTES_PROCEDURE	اجرای رویه ذخیره شده	Aufruf einer gespeicherten Datenbankprozedur.
Ablation study	Ablation	مطالعه حذف مؤلفه	Vergleich, bei dem eine Komponente entfernt wird, um ihren Beitrag zu messen.
Calibration	Kalibrierung	کالیبراسیون	Übereinstimmung zwischen vorhergesagter Sicherheit und beobachteter Korrektheit.
Refusal	Refusal	امتناع از پاسخ	Kontrollierte Entscheidung, keine inhaltliche Antwort zu erzeugen, wenn Evidenz fehlt.
False refusal	False Refusal	امتناع کاذب	Unnötige Verweigerung trotz ausreichender Evidenz.
Path validity	Path Validity	اعتبار مسیر	Anteil der ausgegebenen Evidence Paths, deren Knoten und Kanten im Goldstandard korrekt sind.
Evidence completeness	Evidence Completeness	کامل بودن شواهد	Grad, zu dem alle erforderlichen Belege einer Antwort vorhanden sind.

English term	Deutscher Begriff	فارسی	Einfache deutsche Erklärung
Identifier	Identifier	شناسه	Name eines Programmsymbols, zum Beispiel Klasse, Methode, Property oder Variable.
Mapping	Mapping	نگاشت	Zuordnung zwischen Codeobjekten und physischen Datenbankobjekten.
Semantic model	Semantic Model	مدل معنایی	Compilerrepräsentation, die Syntax mit aufgelösten Symbolen und Typen verbindet.
Stored procedure	Stored Procedure	رویه ذخیره شده	In der Datenbank gespeichertes ausführbares Programm oder SQL-Modul.
Cypher	Cypher	سایفر	Deklarative Abfragesprache für Property Graphs.
BM25	BM25	بی ام ۲۵	Lexikalische Rankingfunktion für Information Retrieval.
CodeBERT	CodeBERT	کدبرت	Vortrainiertes Modell für gemeinsame Repräsentation von natürlicher Sprache und Code.
Retrieval-augmented generation	RAG	تولید تقویت شده با بازیابی	Verbindung eines Retrievers mit einem generativen Modell.
Graph retrieval-augmented generation	GraphRAG	گراف رگ	RAG-Variante, die Graphstrukturen für Indexierung oder Retrieval nutzt.
Evidence record	Evidence Record	رکورد شواهد	Normalisierte Einheit mit Beobachtung, Typ, Quelle, Regel und Status.
Query contract	Query Contract	قرارداد پرسش	Formale Beschreibung, welche Evidenz für eine Frage notwendig ist.

Anhang D - Vollständiges Literaturverzeichnis

Die folgende Liste enthält die im Dokument verwendeten Kern- und Ergänzungsquellen sowie offizielle technische Dokumentation. Bei Preprints wird der Preprint-Status in der finalen Thesis transparent angegeben.

- Nagy, C., Meurice, L., & Cleve, A. (2015). Where Was This SQL Query Executed? A Static Concept Location Approach. SANER. [Quelle](#)
- Usai, N., Montagnini, D., Iliev, K. I., & Camanzo, R. (2026). LogicLens: Leveraging Semantic Code Graph to explore Multi Repository large systems. [Quelle](#)
- Shatnawi, A., et al. (2019). Static Code Analysis of Multilanguage Software Systems. [Quelle](#)
- Yamaguchi, F., Golde, N., Arp, D., & Rieck, K. (2014). Modeling and Discovering Vulnerabilities with Code Property Graphs. IEEE S&P. [Quelle](#)
- Xie, X., et al. (2026). Principles and Practices of Large-Scale Code Analysis at Ant Group. ICSE-SEIP. [Quelle](#)
- Abedu, S., Khatoonabadi, S., & Shihab, E. (2025). Synergizing LLMs and Knowledge Graphs for Software Repository-Related Question Answering. [Quelle](#)
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design Science in Information Systems Research. MIS Quarterly, 28(1), 75-105. [Quelle](#)
- Alshemaimri, B., Elmasri, R., Alsahfi, T., & Almotairi, M. (2021). A survey of problematic database code fragments in software systems. Engineering Reports, 3, e12441. [Quelle](#)
- Lekssays, A. (2026). Bridging Code Property Graphs and Language Models for Program Analysis. SVM. [Quelle](#)
- Shah, P., Ghosh, R., Singhal, A., & Dutta, D. (2025). RANGER: Repository-Level Agent for Graph-Enhanced Retrieval. [Quelle](#)
- Gandhi, S., Gao, L., & Callan, J. (2025). Repository-level Code Search with Neural Retrieval Methods. [Quelle](#)
- Peng, W., et al. (2026). SWE-QA: Can Language Models Answer Repository-level Code Questions? [Quelle](#)
- Alebachew, Y. B., Leary, H., Vaishampayan, S., & Brown, C. (2026). Beyond Code Snippets: Benchmarking LLMs on Repository-Level Question Answering. [Quelle](#)
- Du, J., et al. (2025). DependEval: Benchmarking LLMs for Repository Dependency Understanding. [Quelle](#)
- Olea, C., et al. (2024). Evaluating Persona Prompting for Question Answering Tasks. [Quelle](#)
- Chen, J., et al. (2025). CoReQA: Uncovering Potentials of Language Models in Code Repository Question Answering. [Quelle](#)
- Tao, Y., Li, Y., Qin, Y., & Liu, Y. (2026). Retrieval-Augmented Code Generation: A Survey with Focus on Repository-Level Approaches. [Quelle](#)
- Tao, H., et al. (2025). Code Graph Model: A Graph-Integrated Large Language Model for Repository-Level Software Engineering Tasks. NeurIPS. [Quelle](#)
- Lekssays, A., Mouhcine, H., Tran, K., Yu, T., & Khalil, I. (2025). LLMxCPG. [Quelle](#)
- Zhang, F., et al. (2023). RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. EMNLP. [Quelle](#)
- Ullrich, J., Koch, M., & Vogelsang, A. (2025). From Requirements to Code. [Quelle](#)
- Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. [Quelle](#)
- Hou, X., et al. (2024). Large Language Models for Software Engineering: A Systematic Literature Review. [Quelle](#)
- Edge, D., et al. (2025). From Local to Global: A GraphRAG Approach to Query-Focused Summarization. [Quelle](#)
- Pan, R., Mao, B., Ma, T., & Ling, Z. (2026). ArchAgent. [Quelle](#)
- Liu, J., et al. (2025). Large Language Model-Based Agents for Software Engineering: A Survey. [Quelle](#)
- Zhang, Q., et al. (2024). A Survey on Large Language Models for Software Engineering. [Quelle](#)
- Liu, F., et al. (2026). Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code. [Quelle](#)
- Microsoft. (2026a). .NET Compiler Platform SDK (Roslyn) documentation. [Quelle](#)
- Microsoft. (2026b). Entity Framework Core documentation. [Quelle](#)

Neo4j. (2026). GraphAcademy and Cypher learning paths. [Quelle](#)

GitHub. (2026). CodeQL documentation. [Quelle](#)

Joern Project. (2026). Joern documentation. [Quelle](#)

Abschluss-Checkliste für die Nutzung dieser Lernfassung

- Kann ich Problem, Lösung, Architektur und Evaluation in jeweils höchstens fünf Sätzen erklären?
- Kann ich für jeden Claim mindestens eine Source Location und den vollständigen Evidence Path zeigen?
- Sind bestätigte Entscheidungen, Annahmen und offene Fragen farblich eindeutig getrennt?
- Sind alle Paper-Notizen paraphrasiert und mit Quellen versehen?
- Sind Core, Baseline, Experiment und Future Work getrennt?
- Ist der Corpus über Commit, Lizenz und Census reproduzierbar?
- Sind Pilotdaten und Testdaten getrennt?
- Kann ein anderer Mensch die Experimente aus den Manifests wiederholen?
- Sind False Positives, False Negatives, False Refusals und Evidence Gaps dokumentiert?
- Ist jede rote Frage vor Einreichung geschlossen oder ausdrücklich als offene Entscheidung markiert?